

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Muhammad Bukhari Fitri

NIM. 2310817210015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Bukhari Fitri

NIM : 2310817210015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL	7
MODUL 1 : ANDROID BASIC WITH KOTLIN	10
SOAL 1	10
A. Source Code.....	12
B. Output Program.....	19
C. Pembahasan	24
MODUL 2 : ANDROID LAYOUT	35
SOAL 1	35
A. Source Code.....	37
B. Output Program.....	47
C. Pembahasan	52
SOAL 2	63
A. Pembahasan	63
MODUL 3 : BUILD A SCROLLABLE LIST.....	66
SOAL 1	66
A. Source Code.....	68
B. Output Program.....	93
C. Pembahasan	101
SOAL 2	121
A. Pembahasan	121

MODUL 4 : VIEWMODEL AND DEBUGGING.....	122
SOAL 1	122
A. Source Code.....	122
B. Output Program.....	149
C. Pembahasan	157
SOAL 2	181
A. Pembahasan	181
MODUL 5 : CONNECT TO THE INTERNET	182
SOAL 1	182
A. Source Code.....	182
B. Output Program.....	205
C. Pembahasan	209
Tautan Git.....	237

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1	10
Gambar 2. Tampilan Dadu Setelah Di Roll Soal 1 Modul 1	11
Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1	12
Gambar 4. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 XML	19
Gambar 5. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 XML	20
Gambar 6. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 XML	21
Gambar 7. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 Compose	22
Gambar 8. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 Compose	23
Gambar 9. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 Compose	24
Gambar 10. Tampilan Awal Aplikasi Soal 1 Modul 2	35
Gambar 11. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2	36
Gambar 12. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2	36
Gambar 13. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 XML	47
Gambar 14. Screenshot Hasil Tampilan Pilihan Soal 1 Modul 2 XML	48
Gambar 15. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 XML	49
Gambar 16. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 Compose	50
Gambar 17. Tampilan Pilihan Persentasi Tip Compose	51
Gambar 18. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 Compose	52
Gambar 19. Contoh UI List Soal 1 Modul 3	67
Gambar 20. Contoh UI Detail Soal 1 Modul 3	68
Gambar 21. Tampilan Awal UI Soal 1 Modul 3 XML	93
Gambar 22. Tampilan Awal UI Landscape Soal 1 Modul 3 XML	93
Gambar 23. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 3 XML	94
Gambar 24. Tampilan Ketika Discroll Soal 1 Modul 3 XML	95
Gambar 25. Tampilan Tombol Detail Ditekan Soal 1 Modul 3 XML	96
Gambar 26. Tampilan Tombol Detail Ditekan Landscape Soal 1 Modul 3 XML	96
Gambar 27. Tampilan Awal UI Soal 1 Modul 3 Compose	97

Gambar 28. Tampilan Ketika Discroll Soal 1 Modul 3 Compose	98
Gambar 29. Tampilan Awal UI Landscape Soal 1 Modul 3 Compose.....	98
Gambar 30. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 3 Compose.....	99
Gambar 31. Tampilan Detail Soal 1 Modul 3 Compose	100
Gambar 32. Tampilan Detail Landscape Soal 1 Modul 3 Compose.....	101
Gambar 33. Tampilan Awal UI Soal 1 Modul 4 XML	149
Gambar 34. Tampilan Awal UI Landscape Soal 1 Modul 4 XML	149
Gambar 35. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 4 XML	150
Gambar 36. Tampilan Ketika Discroll Soal 1 Modul 4 XML	151
Gambar 37. Tampilan Tombol Detail Ditekan Soal 1 Modul 4 XML.....	152
Gambar 38. Tampilan Tombol Detail Ditekan Landscape Soal 1 Modul 4 XML....	152
Gambar 39. Tampilan Awal UI Soal 1 Modul 4 Compose.....	153
Gambar 40. Tampilan Ketika Discroll Soal 1 Modul 4 Compose	154
Gambar 41. Tampilan Awal UI Landscape Soal 1 Modul 4 Compose.....	154
Gambar 42. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 4 Compose	155
Gambar 43. Tampilan Detail Soal 1 Modul 4 Compose.....	156
Gambar 44. Tampilan Detail Landscape Soal 1 Modul 4 Compose.....	157
Gambar 45. Penggunaan Debugger Soal 1 Modul 4.....	157
Gambar 46. Tampilan Layar Detail Soal 1 Modul 5.....	205
Gambar 47. Tampilan Awal Soal 1 Modul 5	206
Gambar 48. Tampilan Awal Landscape Soal 1 Modul 5	207
Gambar 49. Tampilan Awal Dark Mode Soal 1 Modul 5	209

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Soal 1 Modul 1 XML.....	12
Tabel 2. Source Code DiceRollerApp.kt Soal 1 Modul 1 XML	13
Tabel 3. Source Code activity_main.xml Soal 1 Modul 1 XML	14
Tabel 4. Source Code MainActivity.kt Soal 1 Modul 1 Compose.....	15
Tabel 5. Source Code DiceRollerApp.kt Soal 1 Modul 1 Compose.....	16
Tabel 6. Source Code DiceWithButtonAndImage.kt Soal 1 Modul 1 Compose.....	16
Tabel 7. Source Code MainActivity.kt Soal 1 Modul 2 XML.....	37
Tabel 8. Source Code activity_main.xml Soal 1 Modul 2 XML	39
Tabel 9. Source Code MainActivity.kt Soal 1 Modul 2 Compose.....	41
Tabel 10. Source Code MainActivity.kt Soal 1 Modul 3 XML.....	68
Tabel 11. Source Code Book.kt Soal 1 Modul 3 XML.....	69
Tabel 12. Source Code DetailFragment.kt Soal 1 Modul 3 XML	71
Tabel 13. Source Code MyItemRecyclerViewAdapter.kt Soal 1 Modul 3 XML.....	72
Tabel 14. Source Code activity_main.xml Soal 1 Modul 3 XML	74
Tabel 15. Source Code fragment_book.xml Soal 1 Modul 3 XML.....	75
Tabel 16. Source Code fragment_detail.xml Soal 1 Modul 3 XML.....	76
Tabel 17. Source Code item_book.xml Soal 1 Modul 3 XML	80
Tabel 18. Source Code MainActivity.kt Soal 1 Modul 3 Compose.....	84
Tabel 19. Source Code Datasource.kt Soal 1 Modul 3 Compose	86
Tabel 20. Source Code Book.kt Soal 1 Modul 3 Compose	86
Tabel 21. Source Code BookDetailScreen.kt Soal 1 Modul 3 Compose.....	87
Tabel 22. Source Code BookListScreen.kt Soal 1 Modul 3 Compose	89
Tabel 23. Source Code MainActivity.kt Soal 1 Modul 4 XML.....	122
Tabel 24. Source Code Book.kt Soal 1 Modul 4 XML.....	123
Tabel 25. Source Code BookFragment.kt Soal 1 Modul 4 XML	123
Tabel 26. Source Code DetailFragment.kt Soal 1 Modul 4 XML	125
Tabel 27. Source Code BookViewModel.kt Soal 1 Modul 4 XML.....	126
Tabel 28. Source Code MyItemRecyclerViewAdapter.kt Soal 1 Modul 4 XML.....	127
Tabel 29. Source Code MyApp.kt Soal 1 Modul 4 XML.....	129

Tabel 30. Source Code activity_main.xml Soal 1 Modul 4 XML	130
Tabel 31. Source Code fragment_book.xml Soal 1 Modul 4 XML.....	131
Tabel 32. Source Code fragment_detail.xml Soal 1 Modul 4 XML.....	131
Tabel 33. Source Code item_book.xml Soal 1 Modul 4 XML	134
Tabel 34. Source Code MainActivity.kt Soal 1 Modul 4 Compose.....	138
Tabel 35. Source Code Datasource.kt Soal 1 Modul 4 Compose	140
Tabel 36. Source Code Book.kt Soal 1 Modul 4 Compose	140
Tabel 37. Source Code BookDetailScreen.kt Soal 1 Modul 4 Compose.....	141
Tabel 38. Source Code BookListScreen.kt Soal 1 Modul 4 Compose	143
Tabel 39. Source Code BookListViewModel.kt Soal 1 Modul 4 Compose	146
Tabel 40. Source Code BookListViewModelFactory.kt Soal 1 Modul 4 Compose .	147
Tabel 41. Source Code MyApplication.kt Soal 1 Modul 4 Compose.....	148
Tabel 42. Source Code presentation/MainActivity.kt Soal 1 Modul 5	183
Tabel 43. Source Code AppContainer.kt Soal 1 Modul 5	185
Tabel 44. Source Code MyApplication.kt Soal 1 Modul 5.....	186
Tabel 45. Source Code core/common/NetworkResult Soal 1 Modul 5	186
Tabel 46. Source Code data/local/dao/MovieDao.kt Soal 1 Modul 5.....	187
Tabel 47. Source Code data/local/model/MovieEntity.kt Soal 1 Modul 5	187
Tabel 48. Source Code data/local/AppDatabase.kt Soal 1 Modul 5	188
Tabel 49. Source Code Data/mapper/MovieMapper.kt Soal 1 Modul 5	188
Tabel 50. Source Code Data/remote/api/ApiConstants.kt Soal 1 Modul 5.....	189
Tabel 51. Source Code data/remote/api/KtroApiClient.kt Soal 1 Modul 5	190
Tabel 52. Source Code data/remote/model/MovieDto.kt Soal 1 Modul 5.....	190
Tabel 53. Source Code data/repository/MovieRepositoryImpl.kt Soal 1 Modul 5...	191
Tabel 54. Source Code domain/model/Movie.kt Soal 1 Modul 5.....	192
Tabel 55. Source Code domain/repository/MovieRepository.kt Soal 1 Modul 5.....	193
Tabel 56. Source Code Domain/usecase/GetMovieUseCase.kt Soal 1 Modul 5.....	193
Tabel 57. Source Code presentation/ui/component/MovieCard.kt Soal 1 Modul 5 .	194
Tabel 58. Source Code presentation/ui/navigation/Screen.kt Soal 1 Modul 5	196

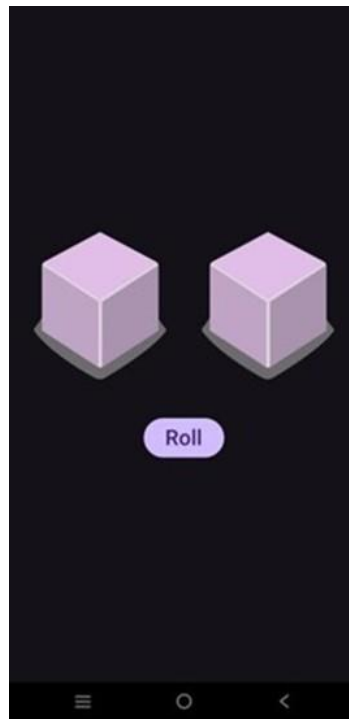
Tabel 59. Source Code presentation/ui/screens/detail/MovieDetailScreen.kt Soal 1 Modul 5	196
Tabel 60. Source Code presentation/ui/screens/list/MovieListScreen.kt Soal 1 Modul 5	198
Tabel 61. Source Code presentation/ui/screens/list/MovieListViewModel.kt Soal 1 Modul 5	202
Tabel 62. Source Code presentation/ui/screens/list/MovieListViewModelFactory.kt Soal 1 Modul 5	204

MODUL 1 : ANDROID BASIC WITH KOTLIN

SOAL 1

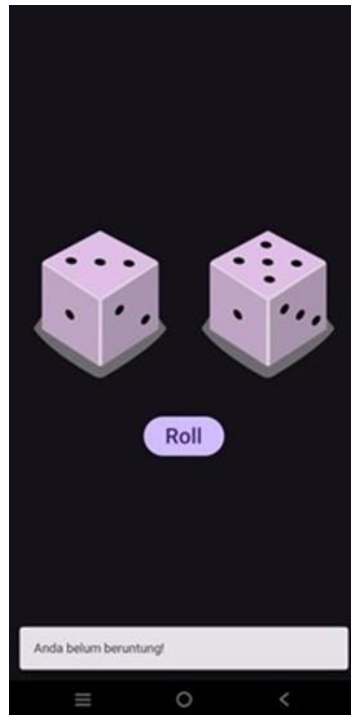
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “*Roll Dice*”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



Gambar 1. Tampilan Awal Aplikasi Soal 1 Modul 1

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di Roll Soal 1 Modul 1

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3. Tampilan Roll Dadu Double Soal 1 Modul 1

A. Source Code

XML:

MainActivity.kt

Tabel 1. Source Code MainActivity.kt Soal 1 Modul 1 XML

1	package com.example.dicexml
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import
7	com.example.dicexml.databinding.ActivityMainBinding
8	
9	class MainActivity : AppCompatActivity() {
10	private lateinit var binding: ActivityMainBinding
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	enableEdgeToEdge()
14	

15	binding	=
	ActivityMainBinding.inflate(layoutInflater)	
16	setContentView(binding.root)	
17		
18	binding.button.setOnClickListener {	
19	diceRollerApp(binding)	
20	}	
21	}	
22	}	

DiceRollerApp.kt

Tabel 2. Source Code DiceRollerApp.kt Soal 1 Modul 1 XML

1	package com.example.dicexml
2	
3	import
	com.example.dicexml.databinding.ActivityMainBinding
4	import com.google.android.material.snackbar.Snackbar
5	
6	fun diceRollerApp(binding: ActivityMainBinding) {
7	val result1 = randomDice()
8	val result2 = randomDice()
9	
10	val imageResource1 = getDiceImage(result1)
11	val imageResource2 = getDiceImage(result2)
12	
13	binding.imageView.setImageResource(imageResource1)
14	binding.imageView2.setImageResource(imageResource2)
15	
16	if (result1 == result2) {
17	Snackbar.make(binding.button, "Selamat, anda dapat
	dadu double", Snackbar.LENGTH_LONG).show()
18	}
19	else if (result1 != result2) {
20	Snackbar.make(binding.button, "Anda belum
	beruntung!", Snackbar.LENGTH_LONG).show()
21	}
22	}
23	
24	fun randomDice() : Int {
25	return (1..6).random()
26	}
27	
28	fun getDiceImage(result: Int) : Int {
29	return when (result) {
30	1 -> R.drawable.dice_1
31	2 -> R.drawable.dice_2

32	3 -> R.drawable.dice_3
33	4 -> R.drawable.dice_4
34	5 -> R.drawable.dice_5
35	else -> R.drawable.dice_6
36	}
37	}

activity_main.xml

Tabel 3. Source Code activity_main.xml Soal 1 Modul 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:background="#2B2A2A"
9	tools:context=".MainActivity">
10	
11	<Button
12	android:id="@+id/button"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content"
15	android:layout_marginStart="162dp"
16	android:layout_marginTop="450dp"
17	android:layout_marginEnd="162dp"
18	android:layout_marginBottom="234dp"
19	android:backgroundTint="#D460ED"
20	android:text="@string/roll"
21	app:layout_constraintBottom_toBottomOf="parent"
22	app:layout_constraintEnd_toEndOf="parent"
23	app:layout_constraintStart_toStartOf="parent"
24	app:layout_constraintTop_toTopOf="parent" />
25	
26	
27	<ImageView
28	android:id="@+id/imageView"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:layout_marginTop="171dp"
32	android:layout_marginEnd="4dp"
33	android:layout_marginBottom="29dp"
34	android:contentDescription="@string/image_description"
35	app:layout_constraintBottom_toTopOf="@+id/button"

36	app:layout_constraintEnd_toEndOf="parent"
	app:layout_constraintStart_toEndOf="@+id/imageView2"
37	app:layout_constraintTop_toTopOf="parent"
38	app:srcCompat="@drawable/dice_0" />
39	
40	<ImageView
41	android:id="@+id/imageView2"
42	android:layout_width="wrap_content"
43	android:layout_height="wrap_content"
44	android:layout_marginTop="171dp"
45	android:layout_marginBottom="29dp"
46	
	android:contentDescription="@string/image_description"
47	app:layout_constraintBottom_toTopOf="@+id/button"
48	
	app:layout_constraintEnd_toStartOf="@+id/imageView"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintTop_toTopOf="parent"
51	app:srcCompat="@drawable/dice_0" />
52	</androidx.constraintlayout.widget.ConstraintLayout>
53	

Compose:

MainActivity.kt

Tabel 4. Source Code MainActivity.kt Soal 1 Modul 1 Compose

1	package com.example.dicecompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.material3.Scaffold
9	import androidx.compose.ui.Modifier
10	import com.example.dicecompose.component.DiceRollerApp
11	import com.example.dicecompose.ui.theme.DiceComposeTheme
12	
13	class MainActivity : ComponentActivity() {
14	override fun onCreate(savedInstanceState: Bundle?) {
15	super.onCreate(savedInstanceState)
16	enableEdgeToEdge()
17	setContent {
18	DiceComposeTheme {
19	

	Scaffold(	modifier	=
20	Modifier.fillMaxSize())		
21	{ innerPadding ->		
22	DiceRollerApp()		
23	}		
24	}		
25	}		
26	}		
	}		

DiceRollerApp.kt

Tabel 5. Source Code DiceRollerApp.kt Soal 1 Modul 1 Compose

1	package com.example.dicecompose.component
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.runtime.Composable
5	import androidx.compose.ui.Alignment
6	import androidx.compose.ui.Modifier
7	import androidx.compose.ui.tooling.preview.Preview
8	
9	@Composable
10	fun DiceRollerApp() {
11	DiceWithButtonAndImage(
12	modifier = Modifier
13	.fillMaxSize()
14	.wrapContentSize(Alignment.Center)
15	)
16	}
17	
18	@Preview(showBackground = true)
19	@Composable
20	fun DiceRollerAppPreview() {
21	DiceRollerApp()
22	}

DiceWithButtonAndImage.kt

Tabel 6. Source Code DiceWithButtonAndImage.kt Soal 1 Modul 1 Compose

1	package com.example.dicecompose.component
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.Column
5	import androidx.compose.foundation.layout.Row
6	import androidx.compose.foundation.layout.Spacer
7	import androidx.compose.foundation.layout.height


```

8 import androidx.compose.material3.Button
9 import androidx.compose.material3.Scaffold
10 import androidx.compose.material3.SnackbarDuration
11 import androidx.compose.material3.SnackbarHost
12 import androidx.compose.material3.SnackbarHostState
13 import androidx.compose.material3.Text
14 import androidx.compose.runtime.Composable
15 import androidx.compose.runtime.getValue
16 import androidx.compose.runtime.mutableStateOf
17 import androidx.compose.runtime.remember
18 import androidx.compose.runtime.rememberCoroutineScope
19 import androidx.compose.runtime.setValue
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.res.painterResource
23 import androidx.compose.ui.res.stringResource
24 import androidx.compose.ui.unit.dp
25 import com.example.dicecompose.R
26
27 import kotlinx.coroutines.launch
28
29 @Composable
30 fun DiceWithButtonAndImage(modifier: Modifier = Modifier)
31 {
32     var result1 by remember { mutableStateOf(0) }
33     var result2 by remember { mutableStateOf(0) }
34     val imageResource1 = getDiceImage(result1)
35     val imageResource2 = getDiceImage(result2)
36
37     val scope = rememberCoroutineScope()
38     val snackbarHostState = remember {
39         SnackbarHostState()
40     }
41     Scaffold(
42         snackbarHost = { SnackbarHost(hostState =
43             snackbarHostState) }
44     ) {
45         paddingValues ->
46         Column (
47             modifier = modifier,
48             horizontalAlignment =
49                 Alignment.CenterHorizontally
50         ) {
51             Row {
52                 Image (
53                     painter =
54                         painterResource(imageResource1),
55                     contentDescription =
56                         result1.toString()

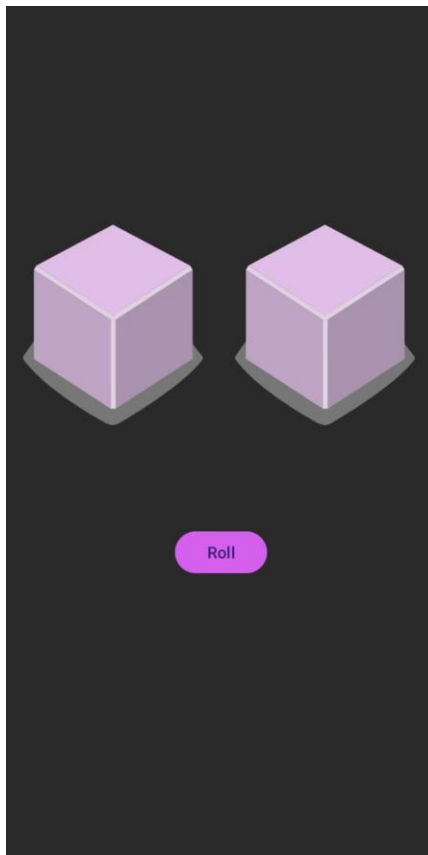
```

```

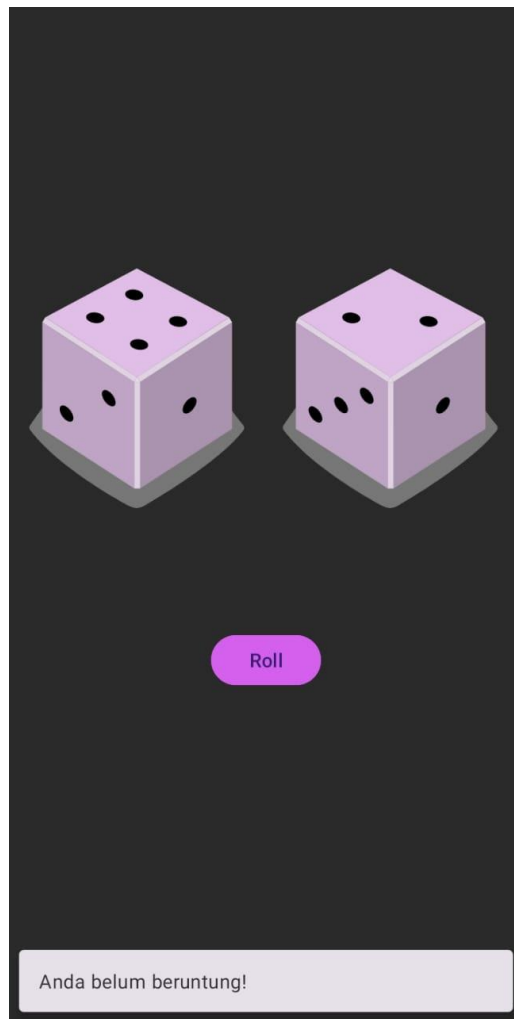
52         )
53         Image(
54             painter                                     =
painterResource(imageResource2),
55             contentDescription                         =
result2.toString()
56         )
57     }
58
59     Spacer(modifier = Modifier.height(12.dp))
60     Button(onClick = {
61         result1 = (1..6).random()
62         result2 = (1..6).random()
63
64         scope.launch {
65             val message = if (result1 == result2)
66             {
67                 "Selamat, anda dapat dadu double!"
68             } else {
69                 "Anda belum beruntung!"
70             }
71             snackHostState.showSnackbar(
72                 message,
73                 duration = SnackbarDuration.Short
74             )
75         }
76     }) {
77         Text(stringResource(R.string.roll))
78     }
79 }
80
81 }
82
83 fun getDiceImage(result: Int) : Int {
84     return when (result) {
85         0 -> R.drawable.dice_0
86         1 -> R.drawable.dice_1
87         2 -> R.drawable.dice_2
88         3 -> R.drawable.dice_3
89         4 -> R.drawable.dice_4
90         5 -> R.drawable.dice_5
91         else -> R.drawable.dice_6
92     }
93 }

```

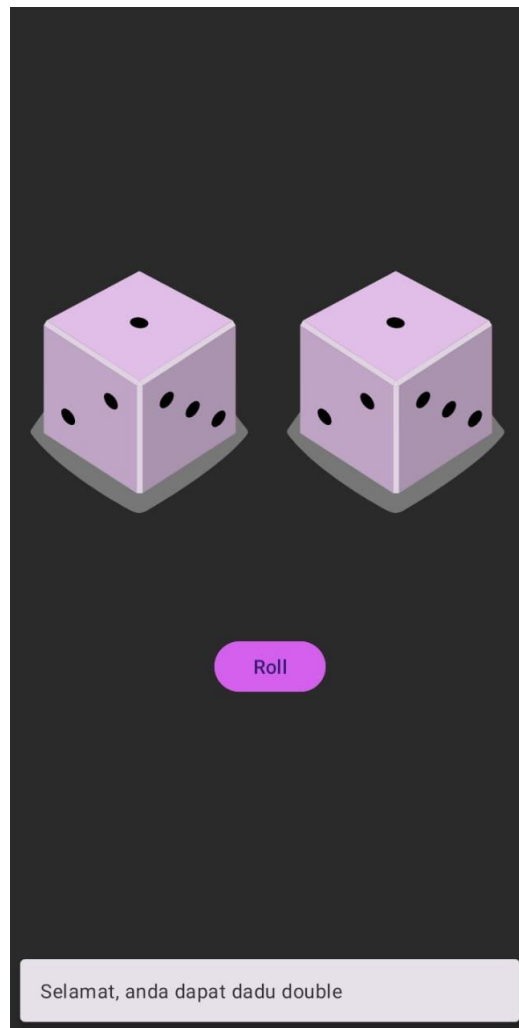
B. Output Program



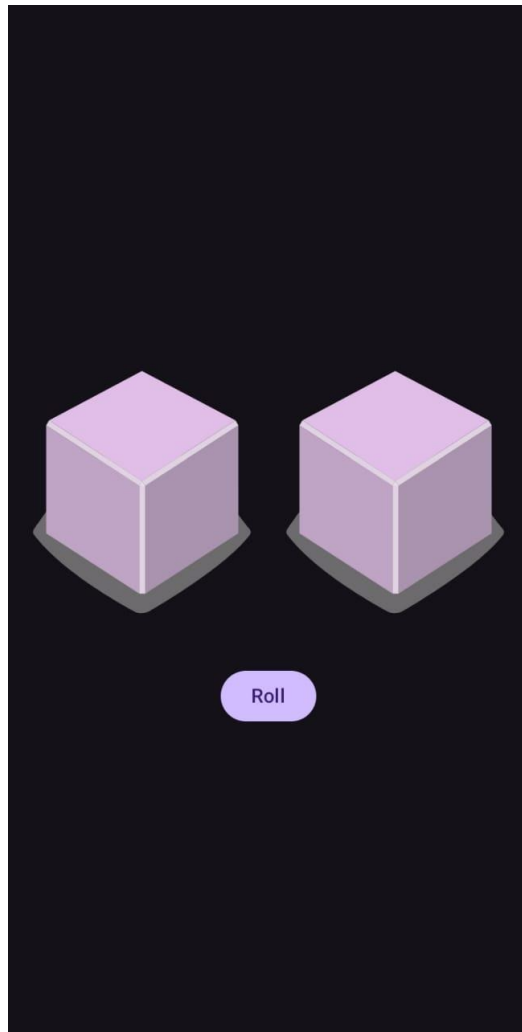
Gambar 4. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 XML



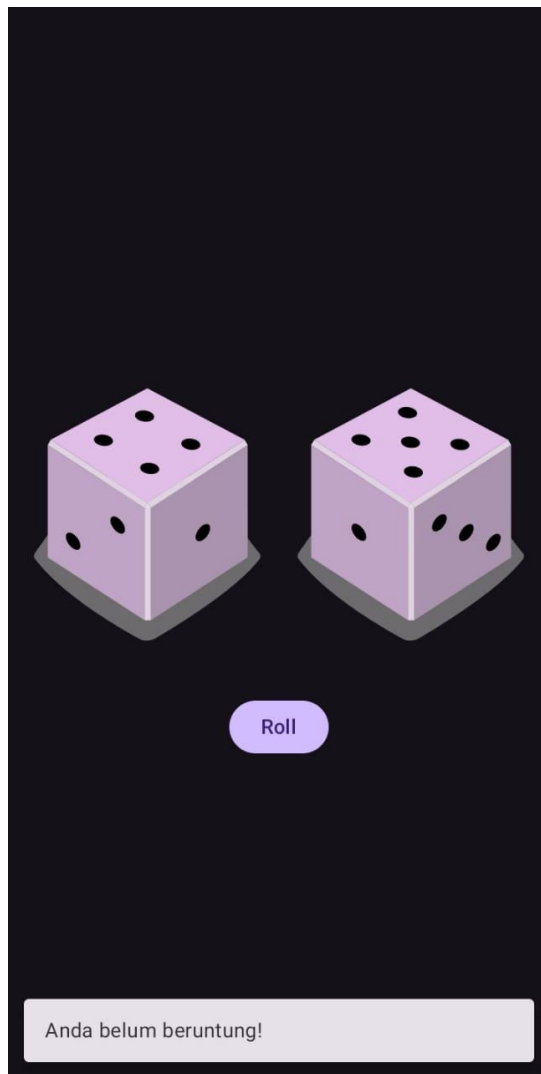
Gambar 5. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 XML



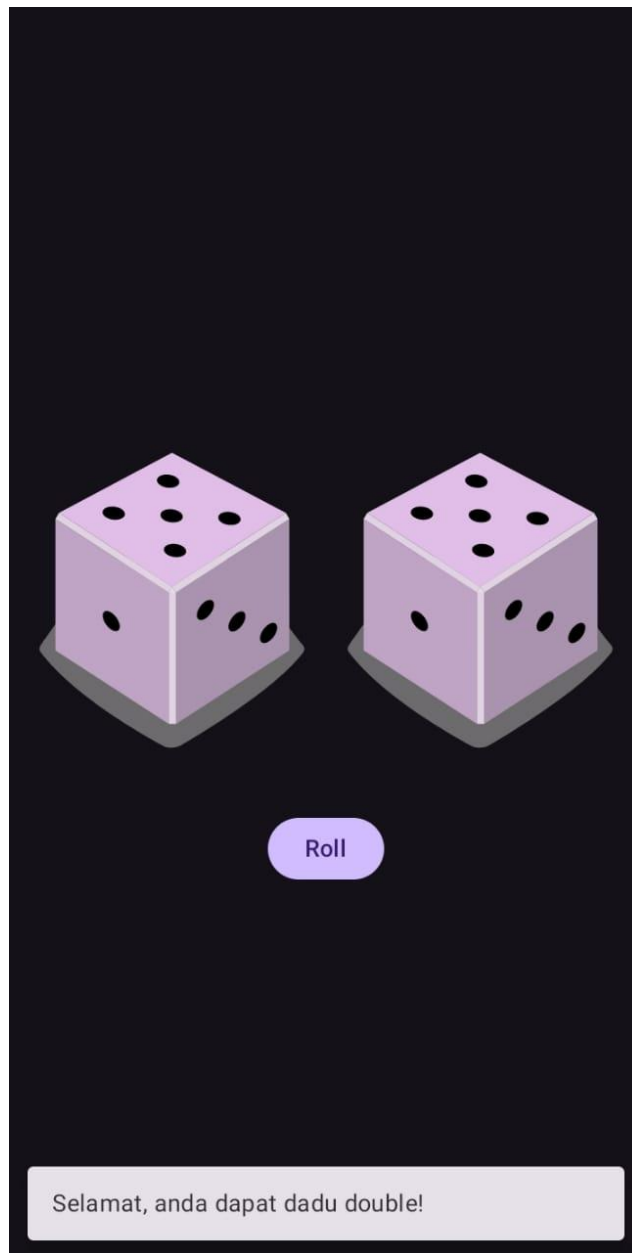
Gambar 6. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 XML



Gambar 7. Screenshot Hasil Tampilan Awal Soal 1 Modul 1 Compose



Gambar 8. Screenshot Hasil Tampilan Dadu Tidak Sama Soal 1 Modul 1 Compose



Gambar 9. Screenshot Hasil Tampilan Dadu Sama Soal 1 Modul 1 Compose

C. Pembahasan

A. XML:

MainActivity.kt:

Pada baris 1, dideklarasikan nama package file Kotlin

Pada baris 3, diimport Bundle untuk mengkomunikasikan data antar aktivitas

Pada baris 4, diimport enableEdgeToEdge untuk menampilkan konten dengan layar penuh

Pada baris 5, diimport AppCompatActivity untuk base class activity yang mendukung versi terdahulu Android

Pada baris 6, diimport ActivityMainBinding untuk menghubungkan layout XML dengan kode kotlin

Pada baris 9, dideklarasikan kelas bernama MainActivity yang extend dari AppCompatActivity

Pada baris 10, terdapat variabel binding bertipe ActivityMainBinding dengan deklarasi lateinit untuk diinisialisasi nanti

Pada baris 11, terdapat override pada fungsi onCreate yang dipanggil saat activity dimulai

Pada baris 12, akan dipanggil superclass dari onCreate

Pada baris 13, akan dipanggil enableEdgeToEdge() agar tampilan memenuhi hingga tepi layar

Pada baris 15, inisialisasi binding untuk menghubungkan layout XML ke Kotlin menggunakan View Binding

Pada baris 16, setContentView(binding.root) untuk menampilkan tampilan XML ke layar

Pada baris 18, menetapkan aksi saat tombol dengan id button ditekan

Pada baris 19, memanggil function diceRollerApp(binding)

DiceRollerApp.kt:

Pada baris 1, dideklarasikan nama package file Kotlin

Pada baris 3, diimport ActivityMainBinding untuk menghubungkan layout XML dengan kode Kotlin

Pada baris 3, diimport Snackbar untuk menampilkan notifikasi

Pada baris 6, dideklarasikan fungsi diceRollerApp dengan parameter binding yang akan menjalankan logika dari aplikasi

Pada baris 7, dideklarasikan variable result1 yang berisi hasil acak dari fungsi randomDice()

Pada baris 8, dideklarasikan variable result2 yang berisi hasil acak dari fungsi randomDice()

Pada baris 10, dideklarasikan variable imageResource1 yang akan mendapatkan gambar dadu sesuai dengan nilai result1 melalui fungsi getDiceImage()

Pada baris 11, dideklarasikan variable imageResource2 yang akan mendapatkan gambar dadu sesuai dengan nilai result1 melalui fungsi getDiceImage()

Pada baris 13, gambar dadu pertama akan ditampilkan di komponen ImageView dengan id imageView menggunakan fungsi setImageResource()

Pada baris 14, gambar dadu kedua akan ditampilkan di komponen ImageView dengan id imageView2 menggunakan fungsi setImageResource()

Pada baris 15, inisialisasi binding untuk menghubungkan layout XML ke Kotlin menggunakan View Binding

Pada baris 16, terdapat pengecekan kondisi if jika nilai result1 dan result2 sama (dadu double)

Pada baris 17, jika kondisi if terpenuhi, maka akan ditampilkan Snackbar dengan pesan "Selamat, anda dapat dadu double" selama LENGTH_LONG

Pada baris 19, terdapat pengecekan kondisi else if jika result1 dan result2 tidak sama

Pada baris 20, jika kondisi tersebut terpenuhi, maka akan ditampilkan Snackbar dengan pesan "Anda belum beruntung!" selama LENGTH_LONG

Pada baris 24, dideklarasikan fungsi randomDice() yang akan mengembalikan nilai Int acak dari 1 sampai 6 menggunakan (1..6).random()

Pada baris 28, dideklarasikan fungsi getDiceImage() dengan parameter result bertipe Int yang akan mengembalikan resource gambar dadu yang sesuai

Pada baris 29-35, digunakan struktur when untuk menentukan resource gambar dadu berdasarkan nilai result

Pada baris 30, jika nilai 1, maka dikembalikan R.drawable.dice_1

Pada baris 31, jika nilai 2, maka dikembalikan R.drawable.dice_2

Pada baris 32, jika nilai 3, maka dikembalikan R.drawable.dice_3
Pada baris 33, jika nilai 4, maka dikembalikan R.drawable.dice_4
Pada baris 34, jika nilai 5, maka dikembalikan R.drawable.dice_5
Pada baris 35, jika tidak ada yang cocok, maka akan dikembalikan gambar default R.drawable.dice_6

activity_main.xml:

Pada baris 1, terdapat versi XML dan encoding yang digunakan
Pada baris 2, dideklarasikan root dari file layout, yakni ConstraintLayout dan namespace XML untuk atribut Android
Pada baris 3, dideklarasikan namespace untuk atribut kustom
Pada baris 4, dideklarasikan namespace untuk atribut tools dalam aplikasi
Pada baris 5, terdapat atribut android:id dengan nilai @+id/main yang menjadi id dari ConstraintLayout
Pada baris 6, layout_width diatur match_parent, artinya lebar layout mengikuti lebar layar perangkat
Pada baris 7, layout_height diatur match_parent, artinya tinggi layout mengikuti tinggi layar perangkat
Pada baris 8, background layout diatur dengan warna #2B2A2A, yaitu abu gelap
Pada baris 10, terdapat tag deklarasi element Button
Pada baris 9, tools:context mengaitkan layout ini dengan kelas MainActivity.kt
Pada baris 11, terdapat deklarasi elemen Button
Pada baris 12, dideklarasikan id dari Button, yaitu @+id/button
Pada baris 13, layout_width diatur wrap_content, menyesuaikan dengan lebar isi tombol
Pada baris 14, layout_height diatur wrap_content, menyesuaikan dengan tinggi isi tombol
Pada baris 15, layout_marginStart diatur sebesar 162dp untuk memberi jarak dari sisi kiri

Pada baris 16, `layout_marginTop` diatur sebesar 450dp sebagai jarak dari atas layar

Pada baris 17, `layout_marginEnd` diatur sebesar 162dp untuk memberi jarak dari sisi kanan

Pada baris 18, `layout_marginBottom` diatur sebesar 234dp sebagai jarak dari bawah

Pada baris 19, `backgroundTint` diatur dengan warna ungu #D460ED sebagai warna latar tombol

Pada baris 20, `android:text` diatur menggunakan string resource `@string/roll` (misalnya: "Roll")

Pada baris 21, `app:layout_constraintBottom_toBottomOf="parent"` artinya bagian bawah tombol dirapatkan ke bawah parent

Pada baris 22, `app:layout_constraintEnd_toEndOf="parent"` artinya bagian kanan tombol dirapatkan ke kanan parent

Pada baris 23, `app:layout_constraintStart_toStartOf="parent"` artinya bagian kiri tombol dirapatkan ke kiri parent

Pada baris 24, `app:layout_constraintTop_toTopOf="parent"` artinya bagian atas tombol dirapatkan ke atas parent

Pada baris 27, terdapat deklarasi elemen `ImageView` pertama

Pada baris 28, id dari `ImageView` adalah `@+id/imageView`

Pada baris 29, lebar `ImageView` diatur `wrap_content`

Pada baris 30, tinggi `ImageView` diatur `wrap_content`

Pada baris 31, `layout_marginTop` diatur sebesar 171dp sebagai jarak dari atas

Pada baris 32, `layout_marginEnd` diatur sebesar 4dp sebagai jarak dari sebelah kanan

Pada baris 33, `layout_marginBottom` diatur sebesar 29dp sebagai jarak dari bawah

Pada baris 34, `contentDescription` menggunakan string resource

`@string/image_description` sebagai deskripsi gambar untuk aksesibilitas

Pada baris 35, `app:layout_constraintBottom_toTopOf="@+id/button"` artinya bagian bawah `ImageView` ditempel ke atas tombol

Pada baris 36, `app:layout_constraintEnd_toEndOf="parent"` artinya sisi kanan `ImageView` ditempel ke sisi kanan `parent`

Pada baris 37, `app:layout_constraintStart_toEndOf="@+id/imageView2"` artinya sisi kiri `ImageView` ditempel ke sisi kanan `imageView2`

Pada baris 38, `app:layout_constraintTop_toTopOf="parent"` artinya bagian atas `ImageView` ditempel ke atas `parent`

Pada baris 39, gambar awal `ImageView` ditampilkan menggunakan `@drawable/dice_0`, gambar dadu kosong

Pada baris 41, terdapat deklarasi elemen `ImageView` kedua

Pada baris 42, id dari `ImageView` adalah `@+id/imageView2`

Pada baris 43, lebar `ImageView` diatur `wrap_content`

Pada baris 44, tinggi `ImageView` diatur `wrap_content`

Pada baris 45, `layout_marginTop` diatur sebesar `171dp`

Pada baris 46, `layout_marginBottom` diatur sebesar `29dp`

Pada baris 47, `contentDescription` menggunakan string resource `@string/image_description`

Pada baris 48, `app:layout_constraintBottom_toTopOf="@+id/button"` artinya bagian bawah `ImageView` kedua ditempel ke atas tombol

Pada baris 49, `app:layout_constraintEnd_toStartOf="@+id/imageView"` artinya sisi kanan `imageView2` ditempel ke kiri `ImageView`

Pada baris 50, `app:layout_constraintStart_toStartOf="parent"` artinya sisi kiri `imageView2` ditempel ke sisi kiri `parent`

Pada baris 51, `app:layout_constraintTop_toTopOf="parent"` artinya bagian atas `imageView2` ditempel ke atas `parent`

Pada baris 52, `app:srcCompat="@drawable/dice_0"` digunakan untuk menampilkan gambar awal dadu kosong

Pada baris 53, merupakan penutup tag dari elemen root `ConstraintLayout`

B. Jetpack Compose:

MainActivity.kt:

Pada baris 1, dideklarasikan nama package file Kotlin

Pada baris 3, diimport Bundle untuk mengkomunikasikan data antar aktivitas

Pada baris 4, diimport ComponentActivity sebagai base activity khusus untuk Jetpack Compose

Pada baris 5, diimport setContent, yang digunakan untuk menetapkan tampilan berbasis Compose

Pada baris 6, diimport enableEdgeToEdge agar tampilan aplikasi bisa memenuhi seluruh layar termasuk area status bar dan navigation bar

Pada baris 7, diimport fillMaxSize yang merupakan modifier Compose agar elemen memenuhi ukuran layar penuh

Pada baris 8, diimport Scaffold, yang digunakan sebagai struktur dasar layout di Jetpack Compose

Pada baris 9, diimport Modifier dari Jetpack Compose untuk mengatur tampilan layout

Pada baris 10, diimport DiceRollerApp dari package component, yaitu fungsi utama yang menampilkan UI aplikasi dadu

Pada baris 11, diimport DiceComposeTheme dari package ui.theme untuk mengatur tema tampilan aplikasi

Pada baris 13, deklarasi kelas MainActivity sebagai tempat logika utama aplikasi dijalankan

Pada baris 14, terdapat override function yaitu onCreate untuk dipanggil saat aktivitas pertama kali dibuat

Pada baris 15, memanggil function super.onCreate untuk memanggil konstruktor dari parent

Pada baris 16, memanggil function enableEdgeToEdge agar tampilan aplikasi bisa memenuhi seluruh layar

Pada baris 17, memanggil function setContent untuk menetapkan tampilan UI dengan Jetpack Compose

Pada baris 18, memanggil DiceComposeTheme, tema kustom yang membungkus tampilan aplikasi

Pada baris 19, memanggil Scaffold untuk layout utama aplikasi

Pada baris 20, terdapat parameter `innerPadding` untuk mengatur padding dalam elemen (belum digunakan)

Pada baris 21, memanggil function `DiceRollerApp()` untuk menampilkan komponen UI utama dari aplikasi

DiceRollerApp.kt:

Pada baris 1, dideklarasikan nama package file Kotlin

Pada baris 3, diimport `layout.*` yang berisi fungsi-fungsi pengaturan layout

Pada baris 4, diimport `Composable` untuk fungsi UI berbasis compose

Pada baris 5, diimport `Alignment` untuk mengatur kesejajaran pada layout

Pada baris 7, diimport `Preview` untuk menampilkan pratinjau UI

Pada baris 9, dideklarasikan anotasi `Composable` untuk memberitahu function sebagai elemen UI compose

Pada baris 10, dideklarasikan function `DiceRollerApp()` untuk memanggil function `DiceWithButtonAndImage()`

Pada baris 11-14, dideklarasikan function yang terdapat parameter modifier dengan properti `fillMaxSize()` untuk memenuhi seluruh layar dan `wrapContentSize(Alignment.Center)` untuk memusatkan isi konten jadi di tengah

Pada baris 18, dideklarasikan anotasi `Preview(showBackground = true)` untuk menampilkan pratinjau UI dengan tampilan background

Pada baris 19, dideklarasikan anotasi `Composable` untuk memberitahu function sebagai elemen UI compose

Pada baris 21-21, dideklarasikan function `DiceRollerAppPreview` yang digunakan untuk memanggil function `DiceRollerApp()` untuk menampilkan preview dari function `DiceRollerApp`

DiceWithButtonAndImage.kt:

Pada baris 1, dideklarasikan nama package file Kotlin

Pada baris 3, diimport `Image` untuk menampilkan gambar pada UI

Pada baris 4, diimport `Column` untuk membuat kolom pada layout UI

Pada baris 5, diimport Row untuk membuat Row pada layout UI

Pada baris 6, diimport Spacer untuk membuat ruang kosong di antara 2 komponen

Pada baris 7, diimport height untuk mengatur tinggi layout

Pada baris 8, diimport Button untuk menyediakan komponen Button

Pada baris 9, diimport Scaffold untuk struktur dasar layout di Jetpack Compose

Pada baris 10, diimport SnackbarDuration untuk mengatur durasi notifikasi Snackbar

Pada baris 11-12, diimport SnackbarHost untuk menampilkan komponen Snackbar

Pada baris 13, diimport SnackbarHostState

Pada baris 13, diimport Text untuk menampilkan teks pada UI

Pada baris 14, diimport Composable untuk fungsi UI berbasis compose

Pada baris 15, diimport getValue untuk mengakses nilai dari properti

Pada baris 16, diimport mutableStateOf untuk membuat state count yang dimulai dari 0

Pada baris 17, diimport remember untuk menyimpan nilai counter

Pada baris 18, diimport rememberCoroutineScope untuk membuat CoroutineScope

Pada baris 19, diimport setValue untuk mengubah nilai dari mutableStateOf dengan by

Pada baris 20, diimport Alignment untuk mengatur kesejajaran pada layout

Pada baris 21, diimport Modifier untuk mengatur tampilan layout

Pada baris 22, diimport painterResource untuk mengambil/meload gambar dari sumber folder

Pada baris 23, diimport stringResource untuk mengambil/meload string dari sumber folder

Pada baris 24, diimport dp untuk mengatur ukuran dalam satuan density-independent pixels (dp)

Pada baris 25, diimport R untuk menyediakan resources (sumber)

Pada baris 27, diimport launch untuk menjalankan kode secara asinkronus dalam Coroutine Scope

Pada baris 29, dideklarasikan anotasi Composable untuk memberitahu function sebagai elemen UI compose

Pada baris 30, dideklarasikan function DiceWithButtonAndImage(modifier: Modifier = Modifier) untuk untuk membuat UI button dan dadu. Function ini memiliki parameter Modifier dengan nilai defaultnya modifier untuk modifikasi layout

Pada baris 31-32, dideklarasikan variable (mutable) state result1 dan result2 untuk menyimpan hasil roll dua dadu yang nilai awalnya 0

Pada baris 33-34, dideklarasikan variable (immutable) imageResource1 dan imageResource2 yang memanggil function getDiceImage(result) untuk menampilkan dadu apa yang muncul sesuai dengan nilai result

Pada baris 36, dideklarasikan variable immutable scope yang memanggil function rememberCoroutineScope() untuk menampilkan snackbar secara asynchronous

Pada baris 37, dideklarasikan variable immutable snackbarHostState untuk menyimpan state dan konten snackbar

Pada baris 39-40, memanggil function Scaffold untuk layout utama aplikasi dan tempat untuk menampilkan snackbar melalui snackbarHost

Pada baris 42, terdapat parameter innerPadding untuk mengatur padding dalam elemen (belum digunakan)

Pada baris 43-45, menggunakan Column untuk membuat kolom pada layout dengan parameter modifier untuk modifikasi dan horizontalAlignment untuk mengatur isi konten ke Tengah secara horizontal.

Pada baris 47-54, menggunakan Row untuk membuat baris pada layout yang didalamnya memanggil komponen compose Image untuk menampilkan gambar menggunakan function painterResource() dan deskripsi konten menggunakan function toString()

Pada baris 58, memanggil Spacer ruang kosong dengan parameter modifier untuk memodifikasi tinggi pada layout sebesar 12dp

Pada baris 59, memanggil Button dengan parameter onClick untuk melakukan aksi saat diklik

Pada baris 60-61, dideklarasikan variable result1 dan result2 yang menghasilkan nilai antara 1 sampai 6 secara acak menggunakan function random()

Pada baris 63, menggunakan scope.launch untuk menampilkan snackbar secara asinkron

Pada baris 64, jika notifikasi snackbar sebelumnya masih tampil, function dismiss() digunakan untuk menyembunyikan notifikasi sebelumnya

Pada baris 66-69, dideklarasikan variable message immutable untuk menampilkan pesan "Selamat, anda dapat dadu double!" jika nilai dari variable result1 dan result2 sama, jika selain itu (nilai tidak sama) maka menampilkan pesan "Anda belum beruntung!"

Pada baris 71, memanggil function showSnackbar() untuk menampilkan Snackbar dengan parameter argument variable message dan deklarasi variable duration untuk mengatur durasi Snackbar secara singkat

Pada baris 77, menggunakan komponen Text untuk menampilkan teks "Roll" yang diambil dari file strings.xml

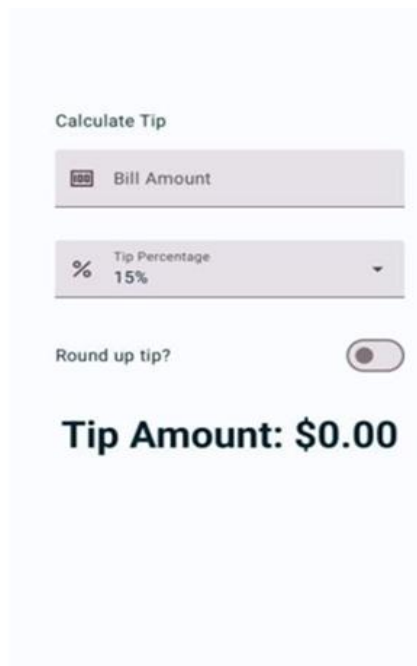
Pada baris 82-90, mendefinisikan function getDiceImage() dengan parameter result bertipe data Int dengan return type Int untuk mengembalikan resource ID gambar berdasarkan nilai result. Jika result adalah 0, akan mengembalikan gambar awal dice_0 (saat belum roll). Nilai 1-6 akan mengembalikan gambar dice_1 sampai dice_6 sesuai hasil roll. Jika selain itu, akan mengembalikan gambar dice_6 sebagai default.

MODUL 2 : ANDROID LAYOUT

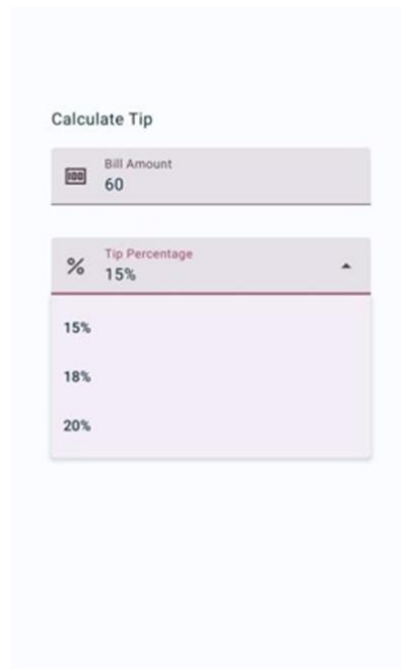
SOAL 1

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

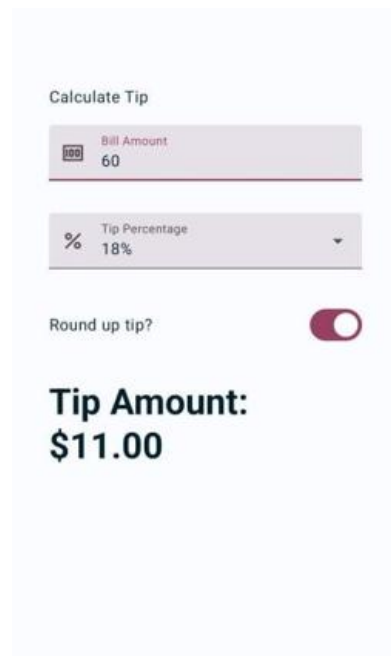
- Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
- Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 10. Tampilan Awal Aplikasi Soal 1 Modul 2



Gambar 11. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2



Gambar 12. Tampilan Aplikasi Setelah Dijalankan Soal Modul 2

A. Source Code

MainActivity.kt

Tabel 7. Source Code MainActivity.kt Soal 1 Modul 2 XML

1	package com.example.calculatortip
2	
3	import android.os.Bundle
4	import android.text.Editable
5	import android.text.TextWatcher
6	import android.view.View
7	import androidx.appcompat.app.AppCompatActivity
8	import
	com.example.calculatortip.databinding.ActivityMainBinding
9	import java.text.NumberFormat
10	import kotlin.math.ceil
11	
12	class MainActivity : AppCompatActivity() {
13	
14	private lateinit var binding: ActivityMainBinding
15	private var amount: String = ""
16	private var tipOption: String = "15%"
17	private var roundUp: Boolean = false
18	
19	override fun onCreate(savedInstanceState: Bundle?) {
20	super.onCreate(savedInstanceState)
21	binding
	ActivityMainBinding.inflate(layoutInflater)
22	setContentView(binding.root)
23	
24	val tipOptions = listOf("15%", "18%", "20%")
25	val adapter = android.widget.ArrayAdapter(this,
	android.R.layout.simple_dropdown_item_1line, tipOptions)
26	binding autoCompleteTextView.setAdapter(adapter)
27	binding autoCompleteTextView.setText(tipOption,
	false)
28	
29	
30	binding.textInputEditText.addTextChangedListener(object :
	TextWatcher {
31	override fun afterTextChanged(s: Editable?) {
32	amount = s.toString()
33	updateTip()
34	}
35	override fun beforeTextChanged(s:
	CharSequence?, start: Int, count: Int, after: Int) {}
36	

```

37         override fun onTextChanged(s: CharSequence?,
38 start: Int, before: Int, count: Int) {}
39     })
40
41
42     binding autoCompleteTextView.setOnItemClickListener {
43 parent, _, position, _ ->
44         tipOption
45         parent.getItemAtPosition(position) as String
46         updateTip()
47     }
48
49     binding.switch1.setOnCheckedChangeListener { _,
50 isChecked ->
51         roundUp = isChecked
52         updateTip()
53     }
54
55     updateTip()
56
57     private fun updateTip() {
58         val amountDouble = amount.toDoubleOrNull() ?: 0.0
59         val tipPercent = when (tipOption) {
60             "15%" -> 0.15
61             "18%" -> 0.18
62             "20%" -> 0.20
63             else -> 0.15
64         }
65
66         var tip = amountDouble * tipPercent
67         if (roundUp) tip = ceil(tip)
68
69         val formattedTip
70         NumberFormat.getCurrencyInstance().format(tip)
71
72         if (formattedTip == "$0.00") {
73             binding.tipAmount.text = "Tip Amount: $0.00"
74             binding.tipAmount.textAlignment
75             View.TEXT_ALIGNMENT_CENTER
76         } else {
77             binding.tipAmount.text
78             Amount:\n$formattedTip"
79             binding.tipAmount.textAlignment
80             View.TEXT_ALIGNMENT_VIEW_START
81         }
82     }
83 }

```

activity_main.xml

Tabel 8. Source Code activity_main.xml Soal 1 Modul 2 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/andro id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<TextView
11	android:id="@+id/textView"
12	android:layout_width="wrap_content"
13	android:layout_height="wrap_content"
14	android:layout_marginStart="48dp"
15	android:layout_marginTop="148dp"
16	android:text="Calculate Tip"
17	android:textSize="16sp"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintTop_toTopOf="parent" />
20	
21	<com.google.android.material.textfield.TextInputLayout
22	android:id="@+id/billAmount"
23	style="@style/Widget.MaterialComponents.TextInputLayout .FilledBox"
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:layout_marginStart="46dp"
27	android:layout_marginTop="12dp"
28	android:layout_marginEnd="46dp"
29	android:hint="@string/bill_amount"
30	app:startIconDrawable="@drawable/money"
31	app:layout_constraintTop_toBottomOf="@id/textView"
32	app:layout_constraintStart_toStartOf="parent"
33	app:layout_constraintEnd_toEndOf="parent">
34	
35	<com.google.android.material.textfield.TextInputEditTex t
36	android:id="@+id/textInputEditText"
37	android:layout_width="match_parent"
38	android:layout_height="wrap_content"
39	android:inputType="numberDecimal" />
40	</com.google.android.material.textfield.TextInputLayout >

41	
42	<com.google.android.material.textfield.TextInputLayout
43	android:id="@+id/tipPercentage"
44	style="@style/Widget.MaterialComponents.TextInputLayout
	.FilledBox.ExposedDropdownMenu"
	android:layout_width="match_parent"
45	android:layout_height="wrap_content"
46	android:layout_marginStart="46dp"
47	android:layout_marginTop="16dp"
48	android:layout_marginEnd="46dp"
49	android:hint="@string/how_was_the_service"
50	app:startIconDrawable="@drawable/percent"
51	app:layout_constraintTop_toBottomOf="@id/billAmount"
52	app:layout_constraintStart_toStartOf="parent"
53	app:layout_constraintEnd_toEndOf="parent">
54	
55	<AutoCompleteTextView
56	android:id="@+id/autoCompleteTextView"
57	android:layout_width="match_parent"
58	android:layout_height="wrap_content"
59	android:inputType="none"/>
60	</com.google.android.material.textfield.TextInputLayout
61	>
62	<TextView
63	android:id="@+id/textView3"
64	android:layout_width="wrap_content"
65	android:layout_height="wrap_content"
66	android:layout_marginStart="44dp"
67	android:layout_marginTop="34dp"
68	android:layout_marginBottom="453dp"
69	android:text="@string/round_up_tip"
70	app:layout_constraintBottom_toBottomOf="parent"
71	app:layout_constraintStart_toStartOf="parent"
72	app:layout_constraintTop_toBottomOf="@+id/tipPercentage
73	" />
74	<androidx.appcompat.widget.SwitchCompat
75	android:id="@+id/switch1"
76	android:layout_width="wrap_content"
77	android:layout_height="wrap_content"
78	android:layout_marginTop="20dp"
79	android:layout_marginEnd="46dp"
80	android:layout_marginBottom="438dp"
81	android:thumb="@drawable/thumb"
82	app:track="@drawable/track"
83	app:layout_constraintBottom_toBottomOf="parent"
84	app:layout_constraintEnd_toEndOf="parent"

85	app:layout_constraintTop_toBottomOf="@+id/tipPercentage
86	" />
87	<TextView
88	android:id="@+id/tipAmount"
89	android:layout_width="0dp"
90	android:layout_height="wrap_content"
91	android:layout_marginStart="48dp"
92	android:layout_marginTop="84dp"
93	android:layout_marginEnd="80dp"
94	android:layout_marginBottom="292dp"
95	android:gravity="start"
96	android:text="Tip Amount: \$0.00"
97	android:textAlignment="viewStart"
98	android:textSize="34sp"
99	android:textStyle="bold"
100	app:layout_constraintBottom_toBottomOf="parent"
101	app:layout_constraintEnd_toEndOf="parent"
102	app:layout_constraintStart_toStartOf="parent"
103	app:layout_constraintTop_toBottomOf="@+id/tipPercentage
104	" />
105	</androidx.constraintlayout.widget.ConstraintLayout>

Compose:

MainActivity.kt

Tabel 9. Source Code MainActivity.kt Soal 1 Modul 2 Compose

1	package com.example.tiptime
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.annotation.DrawableRes
8	import androidx.compose.foundation.layout.Arrangement
9	import androidx.compose.foundation.layout.Column
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxSize
12	import androidx.compose.foundation.layout.fillMaxWidth
13	import androidx.compose.foundation.layout.height
14	import androidx.compose.foundation.layout.padding
15	import
	androidx.compose.foundation.layout.safeDrawingPadding
16	import
	androidx.compose.foundation.layout.statusBarsPadding

```

17 import androidx.compose.foundation.rememberScrollState
18 import androidx.compose.foundation.text.KeyboardOptions
19 import androidx.compose.foundation.verticalScroll
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Surface
22 import androidx.compose.material3.Text
23 import androidx.compose.material3.TextField
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.mutableStateOf
26 import androidx.compose.runtime.remember
27 import androidx.compose.runtime.setValue
28 import androidx.compose.runtime.getValue
29 import androidx.compose.ui.Alignment
30 import androidx.compose.ui.Modifier
31 import androidx.compose.ui.res.stringResource
32 import androidx.compose.ui.text.input.KeyboardType
33 import androidx.compose.ui.tooling.preview.Preview
34 import androidx.compose.ui.unit.dp
35 import com.example.tiptime.ui.theme.TipTimeTheme
36 import java.text.NumberFormat
37 import androidx.annotation.StringRes
38 import androidx.compose.foundation.layout.Row
39 import androidx.compose.ui.text.input.ImeAction
40 import androidx.compose.material3.ExposedDropdownMenuBox
41 import
    androidx.compose.material3.ExposedDropdownMenuDefaults
42 import androidx.compose.material3.Switch
43 import androidx.compose.material3.Icon
44 import androidx.compose.ui.res.painterResource
45 import
    androidx.compose.material3.ExperimentalMaterial3Api
46
47
48 class MainActivity : ComponentActivity() {
49     override fun onCreate(savedInstanceState: Bundle?) {
50         enableEdgeToEdge()
51         super.onCreate(savedInstanceState)
52         setContent {
53             TipTimeTheme {
54                 Surface(
55                     modifier = Modifier.fillMaxSize(),
56                 ) {
57                     TipTimeLayout()
58                 }
59             }
60         }
61     }
62 }
63

```

```

64 @OptIn(ExperimentalMaterial3Api::class)
65 @Composable
66 fun TipTimeLayout() {
67     var amountInput by remember { mutableStateOf("") }
68     val amount = amountInput.toDoubleOrNull() ?: 0.0
69
70     var dropDownTipPercent by remember {
71         mutableStateOf(15.0) }
72
73     var roundUp by remember { mutableStateOf(false) }
74     val tip = calculateTip(amount, dropDownTipPercent,
75         roundUp)
76     Column(
77         modifier = Modifier
78             .statusBarsPadding()
79             .padding(horizontal = 40.dp)
80             .verticalScroll(rememberScrollState())
81             .safeDrawingPadding(),
82         horizontalAlignment = Alignment.CenterHorizontally,
83         verticalArrangement = Arrangement.Center
84     ) {
85         Text(
86             text = stringResource(R.string.calculate_tip),
87             modifier = Modifier
88                 .padding(bottom = 16.dp, top = 40.dp)
89                 .align(alignment = Alignment.Start)
90         )
91         EditNumberField(
92             label = R.string.bill_amount,
93             leadingIcon = R.drawable.money,
94             keyboardOptions = KeyboardOptions.Default.copy(
95                 keyboardType = KeyboardType.Number,
96                 imeAction = ImeAction.Next
97             ),
98             value = amountInput,
99             onValueChange = { amountInput = it },
100             modifier = Modifier
101                 .padding(bottom = 32.dp)
102                 .fillMaxWidth()
103         )
104         TipPercentageDropdown(
105             selectedTip = dropDownTipPercent,
106             leadingIcon = R.drawable.percent,
107             onTipSelected = { dropDownTipPercent = it },
108             modifier = Modifier
109                 .padding(bottom = 32.dp)

```

```

108         .fillMaxWidth()
109     )
110     RoundTheTipRow(
111         roundUp = roundUp,
112         onRoundUpChanged = { roundUp = it },
113         modifier = Modifier.padding(bottom = 32.dp)
114     )
115     val numericTip = tip.replace(Regex("[^\\d.]"),
116     "").toDoubleOrNull() ?: 0.0
117     if (numericTip == 0.0) {
118         Text(
119             text
120             =
121             stringResource(R.string.tip_amount, tip),
122             style
123             =
124             MaterialTheme.typography.displaySmall
125         )
126     }
127     else {
128         Column
129         (modifier
130         =
131         Modifier.align(Alignment.Start)) {
132             Text(
133                 text = "Tip Amount: ",
134                 style
135                 =
136                 MaterialTheme.typography.displaySmall
137             )
138             Text(
139                 text = tip,
140                 style
141                 =
142                 MaterialTheme.typography.displaySmall
143             )
144         }
145     }
146     Spacer(modifier = Modifier.height(150.dp))
147 }
148
149 @Composable
150 fun EditNumberField (
151     @StringRes label: Int,
152     @DrawableRes leadingIcon: Int,
153     keyboardOptions: KeyboardOptions,
154     value: String,
155     onValueChange: (String) -> Unit,
156     modifier: Modifier = Modifier
157 ) {
158     TextField(
159         label = { Text(stringResource(label)) },
160         singleLine = true,
161         keyboardOptions = keyboardOptions,

```

```

151         value = value,
152         leadingIcon = { Icon(painter = painterResource(id
= leadingIcon), null) },
153         onValueChange = onValueChange,
154         modifier = modifier
155     )
156 }
157
158 @OptIn(ExperimentalMaterial3Api::class)
159 @Composable
160 fun TipPercentageDropdown(
161     @DrawableRes leadingIcon: Int,
162     selectedTip: Double,
163     onTipSelected: (Double) -> Unit,
164     modifier: Modifier = Modifier
165 ) {
166     val tipOptions = listOf(15.0, 18.0, 20.0)
167     var expanded by remember { mutableStateOf(false) }
168     val selectedText = "${selectedTip.toInt()}%"
169
170
171     ExposedDropdownMenuBox(
172         expanded = expanded,
173         onExpandedChange = { expanded = !expanded },
174         modifier = modifier
175     ) {
176         TextField(
177             readOnly = true,
178             value = selectedText,
179             leadingIcon = { Icon(painter =
painterResource(id = leadingIcon), null) },
180             onValueChange = {},
181             label = { Text(stringResource(R.string.how_was_the_service)) },
182             trailingIcon = {
183                 ExposedDropdownMenuDefaults.TrailingIcon(
184                     expanded = expanded
185                 )
186             },
187             modifier = Modifier
188                 .menuAnchor()
189                 .fillMaxWidth()
190         )
191         ExposedDropdownMenu(
192             expanded = expanded,
193             onDismissRequest = { expanded = false }
194         ) {
195             tipOptions.forEach { tip ->
196                 androidx.compose.material3.DropdownMenuItem(

```

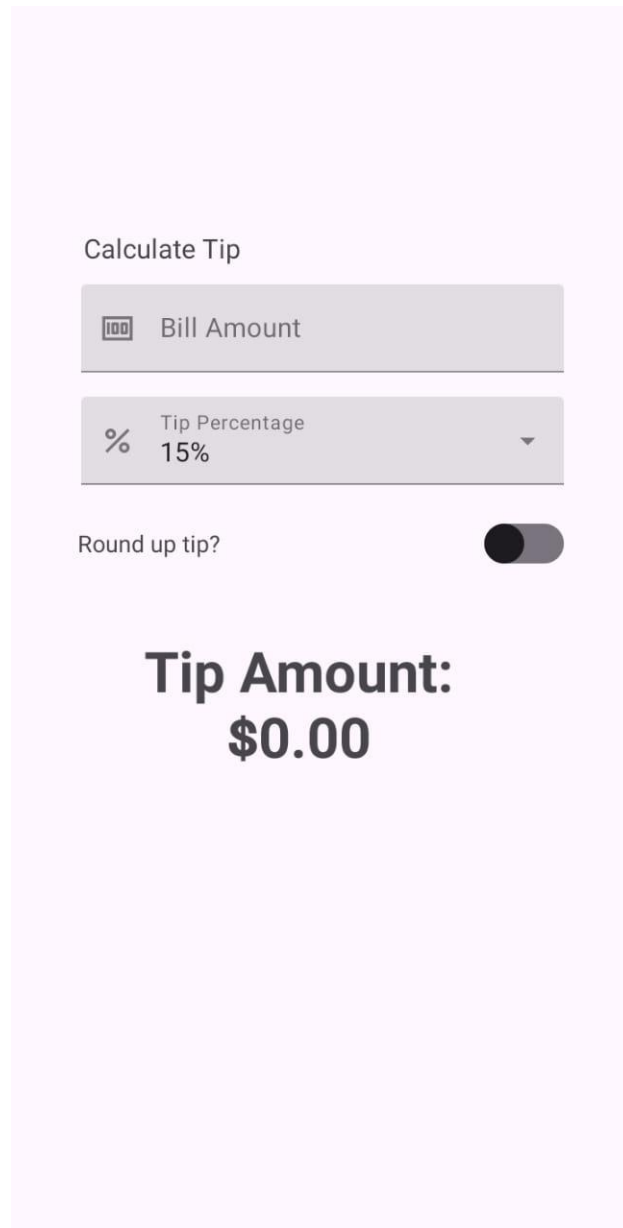
```

197         text = { Text("${tip.toInt()}%") },
198         onClick = {
199             onTipSelected(tip)
200             expanded = false
201         }
202     )
203 }
204 }
205 }
206 }
207 @Composable
208 fun RoundTheTipRow(
209     roundUp: Boolean,
210     onRoundUpChanged: (Boolean) -> Unit,
211     modifier: Modifier = Modifier) {
212     Row(
213         modifier = modifier
214         .fillMaxWidth()
215         .padding(vertical = 8.dp),
216         verticalAlignment = Alignment.CenterVertically,
217         horizontalArrangement = Arrangement.SpaceBetween
218     ) {
219         Text(text = stringResource(R.string.round_up_tip))
220         Switch(
221             checked = roundUp,
222             onCheckedChange = onRoundUpChanged
223         )
224     }
225 }
226
227
228
229 private fun calculateTip(
230     amount: Double,
231     tipPercent: Double,
232     roundUp: Boolean): String {
233     var tip = tipPercent / 100 * amount
234     if(roundUp) {
235         tip = kotlin.math.ceil(tip)
236     }
237     return
238     NumberFormat.getCurrencyInstance().format(tip)
239 }
240 @Preview(showBackground = true)
241 @Composable
242 fun TipTimeLayoutPreview() {

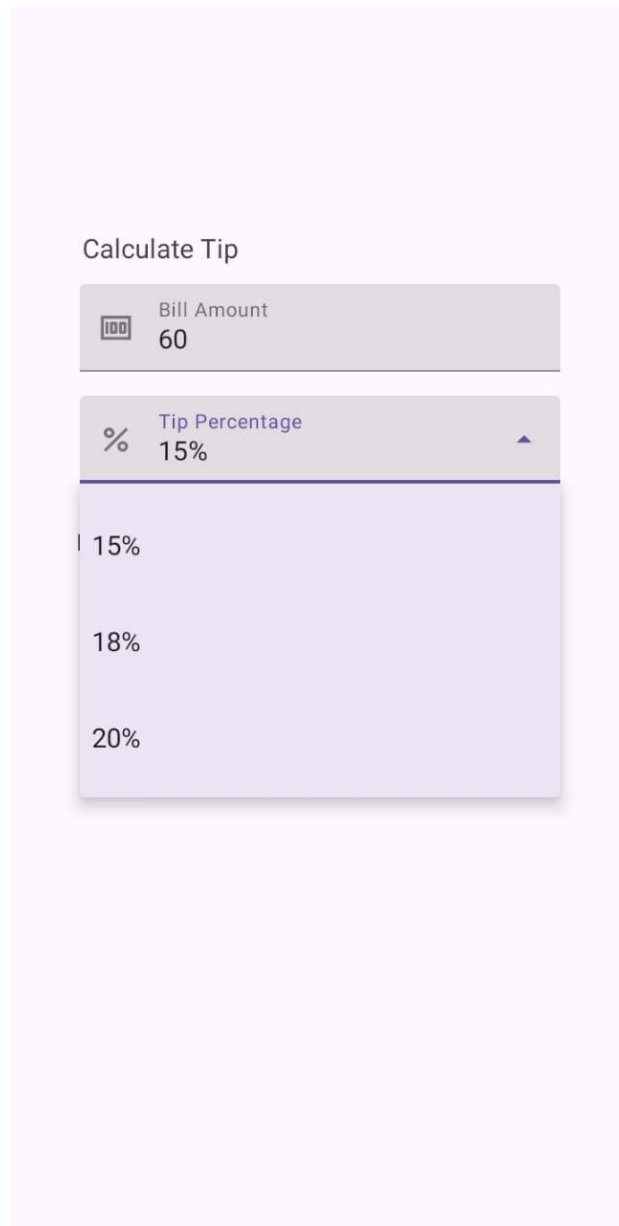
```

243	TipTimeTheme {
244	TipTimeLayout()
245	}
246	}

B. Output Program



Gambar 13. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 XML



Gambar 14. Screenshot Hasil Tampilan Pilihan Soal 1 Modul 2 XML

Calculate Tip

 Bill Amount

60

%

Tip Percentage

18%

▼

Round up tip? ☒

Tip Amount:
\$11.00

Gambar 15. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 XML

Calculate Tip



Bill Amount



Tip Percentage

15%



Round up tip?



Tip Amount: \$0.00

Gambar 16. Screenshot Hasil Tampilan Awal Soal 1 Modul 2 Compose

Calculate Tip

	Bill Amount
	60

%	Tip Percentage	▲
	15%	

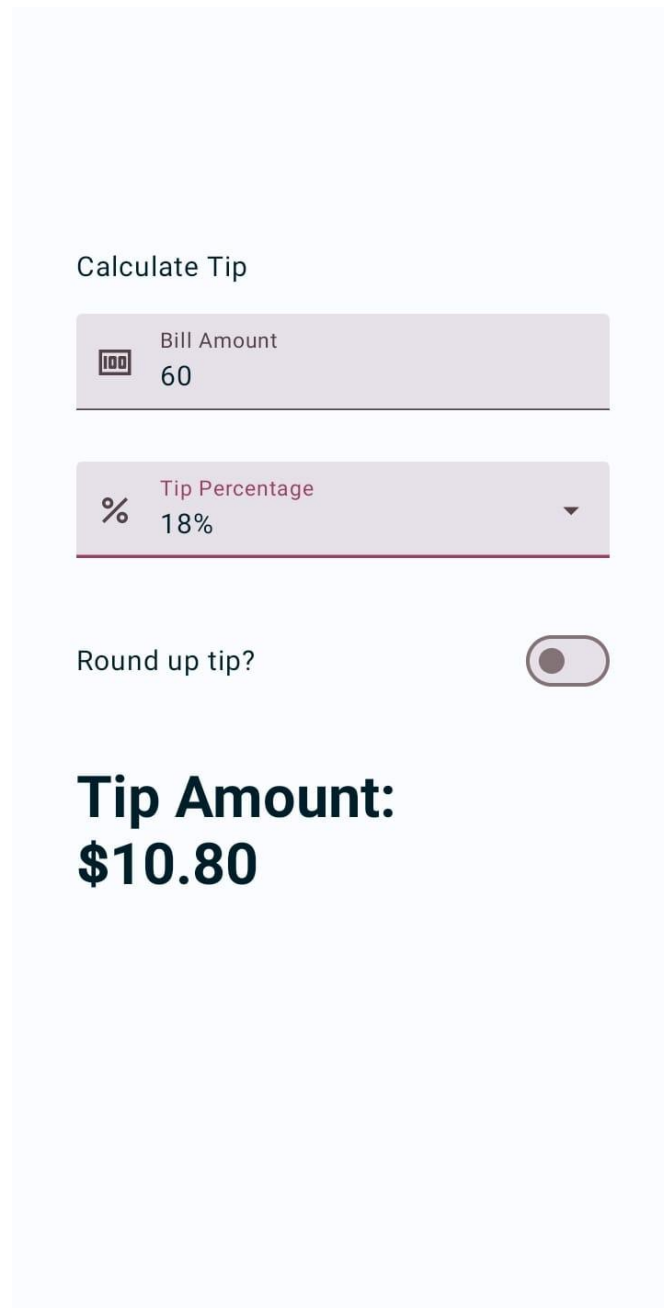
15%

18%

20%

\$9.00

Gambar 17. Tampilan Pilihan Persentasi Tip Compose



Gambar 18. Screenshot Hasil Tampilan Setelah Dijalankan Soal 1 Modul 2 Compose

C. Pembahasan

XML:

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin

Pada line 3, diimportt bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada line 4, diimportt Editable untuk mendeteksi perubahan teks pada input (EditText).

Pada line 5, diimportt View untuk mengatur property tampilan seperti textAlignment.

Pada line 6, diimportt AppCompatActivity yang merupakan superclass dari activity yang menyediakan fitur-fitur android.

Pada line 7, diimportt ActivityMainBinding, kelas binding otomatis yang dihasilkan dari layout XML oleh fitur View Binding.

Pada line 8, diimportt NumberFormat, untuk memformat angka jadi format mata uang.

Pada line 10, diimportt math.ceil untuk membulatkan angka ke atas.

Pada line 12, MainActivity adalah activity utama (layar utama) yang merupakan turunan dari AppCompatActivity sehingga mewarisi fitur-fitur activity android.

Pada line 14, variabel binding menyambungkan elemen-elemen dari XML ke Kotlin menggunakan View Binding, variabel ini tidak langsung diinisialisasi dan hanya bisa diakses dalam class MainActivity.

Pada line 15, deklarasi variabel amount menyimpan input jumlah tagihan dari pengguna sebagai String, variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 16, deklarasi variabel tipOption menyimpan persentase tip yang dipilih dari dropdown (default 15%), variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 17, deklarasi variabel roundup Boolean yang menunjukkan apakah tip harus dibulatkan ke atas, variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 19-22, function onCreate merupakan method utama yang dipanggil saat activity pertama kali dibuat. Binding untuk menghubungkan layout dengan file Kotlin. setContentView(binding.root) menentukan layout utama dari activity ini.

Pada line 24-27, inialisasi dropdown persentase tip. Variabel tipOptions berisi pilihan persentasi tip menggunakan function listOf(). ArrayAdapter digunakan untuk menghubungkan data list ke autoCompleteTextView. setAdapter() menghubungkan dropdown dengan list. setText(tipOption, false) untuk menampilkan pilihan default yaitu 15%.

Pada line 29-36, terdapat listener perubahan input tagihan. Menggunakan TextWatcher untuk mendeteksi perubahan input. Setelah teks berubah (afterTextChanged), nilai amount diperbarui dan updateTip() dipanggil untuk menghitung ulang tip.

Pada line 38-41, terdapat listener pemilihan dropdown, saat user memilih persentase dari dropdown, tipOption diupdate dan tip dihitung ulang.

Pada line 43-48, terdapat listener switch pembulatan tip, mengatur apakah tip akan dibulatkan ke atas atau tidak. Lalu, langsung memanggil updateTip() setelah status switch berubah..

Pada line 51-70, terdapat function update() sebagai logika inti untuk update tagihan, variable; amountDouble mengonversi input tagihan dari String ke Double. Jika input kosong atau bukan angka, maka return 0.0. lalu variabel tipPercent menentukan nilai persentase tip sesuai pilihan user. Variabel tip menghitung tip berdasarkan jumlah tagihan dan persentase. Jika roundup aktif, gunakan ceil() untuk membulatkan angka ke atas. Lalu format nilai tip ke dalam bentuk mata uang, jika hasil tip ada %0.00, posisikan teks di Tengah, jika selain itu maka tampilkan rata kiri.

activity_main.xml:

Pada line 1, terdapat versi XML dan encoding yang digunakan

Pada line 2, dideklarasikan root dari file layout, yakni ConstraintLayout dan namespace XML untuk atribut Android

Pada line 3, dideklarasikan namespace untuk atribut kustom

Pada line 4, dideklarasikan namespace untuk atribut tools dalam aplikasi

Pada line 5-8, android:id: ID layout (jarang dipakai langsung, tapi bisa untuk referensi jika dibutuhkan). layout_width dan layout_height: match_parent berarti layout mengisi seluruh layar. tools:context: memberitahu Android Studio bahwa layout ini milik MainActivity (digunakan untuk preview).

Pada line 10-19, terdapat element <TextView> adalah label yang menampilkan teks "Calculate Tip". Atribut android:id="@+id/textView" memberikan ID agar bisa diakses dari kode Kotlin. layout_width dan layout_height di-set ke wrap_content, artinya ukuran elemen akan mengikuti kontennya (teks). Margin kiri layout_marginStart="48dp" dan margin atas layout_marginTop="148dp" memberikan jarak terhadap tepi parent layout. Atribut android:text="Calculate Tip" adalah teks yang ditampilkan. textSize="16sp" mengatur ukuran teks dalam skala yang mengikuti preferensi ukuran font pengguna. Dua atribut terakhir adalah constraint: app:layout_constraintStart_toStartOf="parent" membuat sisi kiri TextView sejajar dengan sisi kiri ConstraintLayout, dan app:layout_constraintTop_toTopOf="parent" berarti bagian atas TextView sejajar dengan atas layout.

Pada line 21-40, terdapat element <TextInputLayout> untuk input teks dengan label dan ikon. android:id="@+id/billAmount" adalah ID dari element ini. style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox" artinya menggunakan style filledBox. android:layout_width="match_parent" artinya selebar layar. android:layout_height="wrap_content" artinya tinggi mengikuti isinya. android:layout_marginStart="46dp" maka jarak dari sisi kiri 46dp. android:layout_marginTop="12dp" artinya jarak atas dari elemen sebelumnya adalah 12dp. android:layout_marginEnd="46dp" artinya jarak dari sisi kanan adalah 46dp. android:hint="@string/bill_amount" maka hint akan muncul di dalam input field (kotak input). app:startIconDrawable="@drawable/money" untuk menampilkan ikon uang di sebelah kiri input. app:layout_constraintTop_toBottomOf="@id/textView" maka element ditaruh di bawah elemen dengan ID textView. app:layout_constraintStart_toStartOf="parent" untuk sejajar kiri parent.

app:layout_constraintEnd_toEndOf="parent" untuk sejajar kanan parent. Di dalamnya terdapat `<TextInputEditText />` yang dimana tempat user mengetik jumlah uang. `android:id="@+id/textInputEditText"` adalah ID untuk diakses dari Kotlin. `android:layout_width="match_parent"` maka selebar kotak input. `android:layout_height="wrap_content"` maka tinggi otomatis sesuai dengan isi konten. `android:inputType="numberDecimal"` maka hanya menerima angka desimal.

Pada line 42-61, terdapat element `<TextInputLayout>` kedua, untuk memilih persentase tip. ID-nya `@+id/tipPercentage`, menggunakan style `ExposedDropdownMenu` agar input bisa menampilkan dropdown. Lebar `match_parent`, tinggi `wrap_content`, margin kiri dan kanan sama yaitu `46dp`, margin atas dari elemen sebelumnya adalah `16dp`. Hint-nya adalah teks dari string resource `@string/how_was_the_service`. Ikon `@drawable/percent` ditampilkan di kiri input. Elemen ini di-constraint ke bawah `billAmount`, dan kiri-kanannya sejajar parent. Di dalamnya ada `AutoCompleteTextView` dengan ID `autoCompleteTextView`, tipe input-nya diset ke `none` karena pengguna memilih dari dropdown, bukan mengetik langsung.

Pada line 63-73, terdapat element `<TextView>` untuk label switch "Round up tip". ID-nya `textView3`, ukuran mengikuti konten. Margin kiri `44dp`, margin atas `34dp`, dan margin bawah `453dp` (cukup besar agar tidak mepet ke bawah layar). Teks diambil dari string resource `@string/round_up_tip`. Elemen ini di-constraint ke bawah `tipPercentage`, sisi kiri sejajar parent, dan bagian bawah diset ke bawah parent (supaya tetap berada di area bawah layar).

Pada line 75-86, terdapat element `<SwitchCompat>` yaitu switch modern dari AndroidX. ID-nya `@+id/switch1`. Ukuran mengikuti konten. Margin atas `20dp`, margin kanan `46dp`, margin bawah `438dp`. Atribut `android:thumb="@drawable/thumb"` dan `app:track="@drawable/track"` mengatur gambar thumb (lingkaran pada switch) dan track (jalur switch) menggunakan drawable custom. Constraint top-nya di bawah `tipPercentage`, kanan sejajar parent, bawah ke bawah parent.

Pada line 88-105, terdapat element <TextView> untuk menampilkan hasil perhitungan tip. ID-nya tipAmount, lebar di-set 0dp (dalam ConstraintLayout, ini artinya lebar akan mengikuti constraint antara kiri dan kanan), tinggi wrap_content. Margin kiri 48dp, atas 84dp, kanan 80dp, bawah 292dp. Atribut gravity="start" mengatur posisi teks agar sejajar ke kiri. text="Tip Amount: \$0.00" adalah nilai default. textAlignment="viewStart" untuk memastikan alignment teks mengikuti arah tampilan. Ukuran teks besar (34sp) dan textStyle="bold" untuk menonjolkan hasil. Constraint-nya menyambung dari bawah tipPercentage, lalu kiri-kanan dan bawah sejajar dengan parent.

Jetpack Compose:

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin.

Pada line 3, diimport Bundle untuk mengkomunikasikan data antar aktivitas.

Pada line 4, diimport AppCompatActivity, superclass untuk aktivitas di Jetpack Compose yang menggantikan AppCompatActivity. Pada line 5, diimport setContent untuk menetapkan konten berbasis Compose dalam activity.

Pada line 6, diimport enableEdgeToEdge untuk memungkinkan tampilan UI mengisi seluruh layar, termasuk area status bar. Pada line 7, diimport DrawableRes untuk menandai parameter sebagai resource gambar drawable.

Pada line 8-16, diimport berbagai komponen layout (Arrangement, Column, Spacer, dll.) untuk mengatur tampilan UI dalam Compose. Pada line 17, diimport rememberScrollState untuk membuat dan mengingat status scroll dalam UI.

Pada line 18, diimport KeyboardOptions untuk mengonfigurasi opsi keyboard pada input field.

Pada line 19, diimport verticalScroll untuk memungkinkan elemen UI di-scroll secara vertikal.

Pada line 20-23, diimport berbagai komponen Material 3 (MaterialTheme, Surface, Text, TextField, dll.) untuk membangun UI berbasis Material Design 3. Pada line 24-28, diimport komponen reaktivitas Compose seperti mutableStateOf, remember, dan setValue untuk menangani state dalam UI. Pada line 29-35, diimport komponen UI (Alignment, Modifier, dll.) untuk penataan dan pengaturan posisi elemen UI. Pada line 32, diimport KeyboardType untuk menentukan jenis keyboard yang digunakan dalam input field.

Pada line 31, diimport stringResource untuk mengambil resource string dalam UI. Pada line 33, diimport Preview untuk melihat preview komponen Compose dalam editor.

Pada line 34, diimport dp untuk mendefinisikan ukuran dalam satuan density-independent pixels.

Pada line 35, diimport tema TipTimeTheme untuk menentukan tema aplikasi.

Pada line 36, diimport NumberFormat untuk memformat angka menjadi format mata uang.

Pada line 37, diimport StringRes untuk menandai parameter sebagai resource string yang diambil dari file strings.xml.

Pada line 38, diimport Row untuk menata elemen secara horizontal dalam layout Compose.

Pada line 39, diimport ImeAction untuk menentukan aksi keyboard seperti "Next" atau "Done".

Pada line 40, diimport ExposedDropdownMenuBox untuk menampilkan dropdown menu yang dapat diekspos.

Pada line 41, diimport ExposedDropdownMenuDefaults untuk mengatur pengaturan default pada ExposedDropdownMenu.

Pada line 42, diimport Switch untuk menampilkan komponen switch (aktif/tidak aktif) dalam UI.

Pada line 43, diimport Icon untuk menampilkan ikon dalam komponen UI.

Pada line 44, diimport painterResource untuk mengambil resource gambar sebagai

objek

Painter.

Pada line 45, diimport `ExperimentalMaterial3Api` untuk menandai penggunaan API eksperimental dari Material 3.

Pada line 48-62, kelas `MainActivity` meng-extend `ComponentActivity`, yang merupakan kelas dasar untuk activity di Jetpack Compose. Di dalam method `onCreate`, fungsi `enableEdgeToEdge()` digunakan untuk mengaktifkan tampilan layar penuh hingga ke area status bar. Setelah itu, `super.onCreate(savedInstanceState)` dipanggil untuk menjalankan logika bawaan dari `ComponentActivity`. Kemudian, `setContent` digunakan untuk menentukan tampilan Compose dengan `TipTimeTheme`, yang membungkus komponen UI dalam tema yang telah ditentukan sebelumnya. Di dalamnya, `Surface` digunakan sebagai pembungkus elemen UI untuk memberikan latar belakang dan memastikan seluruh layar terisi dengan `Modifier.fillMaxSize()`. Selanjutnya, fungsi `TipTimeLayout()` dipanggil untuk menampilkan layout utama aplikasi.

Pada line 64-136, `TipTimeLayout` adalah fungsi komponen utama dalam aplikasi ini yang menggunakan deklarasi UI berbasis Jetpack Compose. Fungsi ini dimulai dengan mendeklarasikan tiga state penting yang mengontrol input dan pilihan pengguna. Pertama, `amountInput` adalah variabel yang menyimpan input dari pengguna untuk jumlah tagihan, yang didefinisikan menggunakan `remember` dan `mutableStateOf("")`. `remember` memastikan bahwa nilai variabel ini akan dipertahankan saat tampilan diperbarui (misalnya, jika pengguna mengubah input). Nilai awalnya adalah string kosong, yang menunjukkan bahwa belum ada jumlah yang dimasukkan. `amount` adalah variabel turunan yang menghitung nilai numerik dari `amountInput` dengan mengonversi string ke tipe data `Double` menggunakan fungsi `toDoubleOrNull()`. Jika input tidak valid, nilai `amount` akan menjadi 0.0. Variabel `dropDownTipPercent` menyimpan persentase tip yang dipilih pengguna, dengan nilai default 15.0. `roundUp` adalah sebuah boolean yang menentukan apakah tip harus dibulatkan ke atas. Variabel ini dikelola menggunakan `remember` dan `mutableStateOf(false)` untuk menyimpan status pembulatan tip. Kemudian, fungsi `calculateTip` digunakan untuk menghitung nilai tip berdasarkan jumlah tagihan, persentase tip yang dipilih, dan apakah

pembulatan harus diterapkan. Di dalam UI, berbagai elemen ditampilkan menggunakan Column yang memiliki beberapa modifier untuk mengatur layout seperti `statusBarsPadding()` yang memberi ruang di bagian atas untuk status bar, `padding(horizontal = 40.dp)` untuk memberi jarak horizontal, dan `verticalScroll(rememberScrollState())` yang memungkinkan konten di dalam Column discroll secara vertikal jika diperlukan. Di dalam Column, terdapat elemen-elemen UI seperti Text untuk menampilkan judul, `EditNumberField` untuk input jumlah tagihan, `TipPercentageDropdown` untuk memilih persentase tip, dan `RoundTheTipRow` untuk memilih apakah tip harus dibulatkan. Hasil perhitungan tip kemudian ditampilkan dengan format mata uang menggunakan Text, dan jika tip yang dihitung adalah 0.0, hanya pesan "Tip Amount: 0" yang akan ditampilkan. Jika tip valid, maka dua baris teks ditampilkan, yaitu label "Tip Amount" dan nilai tip itu sendiri.

Pada line 138-156, `EditNumberField` adalah fungsi komponen yang mengatur tampilan input untuk jumlah tagihan. Fungsi ini menerima beberapa parameter: label, `leadingIcon`, `keyboardOptions`, `value`, `onValueChange`, dan modifier. Parameter label digunakan untuk menunjukkan teks yang menggambarkan input, yang akan diambil dari string resource. `leadingIcon` menunjukkan ikon yang muncul di sebelah kiri input, yang dalam hal ini adalah ikon uang yang diambil dari drawable resource. `keyboardOptions` memungkinkan penyesuaian keyboard yang digunakan saat pengguna berinteraksi dengan `TextField`. Di sini, keyboard dikonfigurasi untuk menerima input angka dengan tipe `KeyboardType.Number` dan aksi `ImeAction.Next`, yang memungkinkan pengguna untuk melanjutkan ke elemen input berikutnya. Parameter `value` berisi nilai input yang dimasukkan oleh pengguna, dan `onValueChange` adalah fungsi callback yang digunakan untuk memperbarui nilai `amountInput` saat pengguna mengubah teks. Modifier padding ditambahkan untuk memberi jarak vertikal dan horizontal di sekitar elemen input agar tampilan menjadi lebih rapi dan mudah dibaca. Modifier `fillMaxWidth()` memastikan bahwa `TextField` akan mengisi lebar layar secara penuh. Ini memungkinkan elemen input untuk responsif dan mudah diakses oleh pengguna.

Pada line 158-206, `TipPercentageDropdown` adalah fungsi komponen untuk menampilkan dropdown menu bagi pengguna untuk memilih persentase tip yang ingin dihitung. Fungsi ini menerima beberapa parameter, termasuk `leadingIcon`, `selectedTip`, `onTipSelected`, dan `modifier`. Parameter `leadingIcon` adalah ikon yang ditampilkan di samping dropdown, yang berfungsi sebagai petunjuk visual bagi pengguna mengenai opsi yang dapat dipilih. `selectedTip` berisi nilai persentase tip yang saat ini dipilih, yang dalam hal ini didefinisikan dengan tipe `Double`. Fungsi callback `onTipSelected` digunakan untuk memperbarui nilai `dropDownTipPercent` ketika pengguna memilih persentase tip yang diinginkan. Di dalam fungsi ini, terdapat `ExposedDropdownMenuBox`, sebuah elemen UI yang memungkinkan pembuatan dropdown yang dapat dibuka dan ditutup. `ExposedDropdownMenuBox` mengontrol status terbuka atau tertutupnya dropdown dengan mengubah nilai `expanded`, yang dapat berubah ketika pengguna berinteraksi dengan dropdown. Di dalam dropdown menu, terdapat sejumlah opsi persentase tip (15%, 18%, 20%) yang ditampilkan menggunakan `DropdownMenuItem`. Ketika pengguna memilih salah satu item, nilai `dropDownTipPercent` diperbarui, dan dropdown ditutup. `TextField` di dalam `ExposedDropdownMenuBox` berfungsi hanya untuk menampilkan nilai persentase yang dipilih, dan pengaturan `readOnly = true` memastikan bahwa pengguna tidak dapat mengedit teks secara langsung. Ikon di bagian belakang dropdown ditambahkan menggunakan `ExposedDropdownMenuDefaults.TrailingIcon`, yang menunjukkan status dropdown apakah sedang terbuka atau tertutup.

Pada line 207-225, `RoundTheTipRow` adalah fungsi komponen yang menampilkan elemen `Row` untuk menampilkan pilihan apakah pengguna ingin membulatkan jumlah tip. Di dalam `Row`, terdapat elemen teks yang menunjukkan pilihan "Round Up Tip" diikuti dengan `Switch`. `Switch` adalah elemen UI yang memungkinkan pengguna untuk mengubah status dari `false` ke `true`, yang akan memperbarui nilai `roundUp`. Parameter `onCheckedChange` adalah callback yang digunakan untuk memperbarui status `roundUp` saat pengguna mengubah nilai switch. Modifier `padding` ditambahkan untuk memberi jarak vertikal di sekitar elemen switch agar tampil lebih rapi. Dengan menggunakan `Row`, elemen-elemen ini ditata secara horizontal dan diposisikan agar

ada jarak antar elemen dengan `Arrangement.SpaceBetween`, yang memastikan bahwa teks dan switch memiliki ruang yang cukup di antara mereka.

Pada line 229-238, `calculateTip` adalah fungsi utilitas yang menghitung jumlah tip berdasarkan jumlah tagihan yang dimasukkan, persentase tip, dan status pembulatan. Fungsi ini menerima tiga parameter: `amount` (jumlah tagihan), `tipPercent` (persentase tip), dan `roundUp` (boolean yang menentukan apakah tip harus dibulatkan). Pertama, nilai tip dihitung dengan rumus $\text{tipPercent} / 100 * \text{amount}$. Jika `roundUp` bernilai `true`, maka nilai tip dibulatkan ke atas menggunakan `kotlin.math.ceil()`. Fungsi `ceil` akan membulatkan angka tip ke angka bulat terdekat yang lebih besar atau sama dengan angka tersebut. Terakhir, hasil perhitungan tip diformat sebagai mata uang menggunakan `NumberFormat.getCurrencyInstance().format(tip)`. Fungsi ini mengembalikan string yang berisi nilai tip dalam format mata uang yang sesuai dengan lokal perangkat.

Pada line 240-246, `TipTimeLayoutPreview` adalah fungsi `@Preview` untuk melihat pratinjau dari tampilan UI di dalam IDE tanpa perlu menjalankan aplikasi pada perangkat. Dengan menambahkan anotasi `@Preview`, pengembang dapat melihat bagaimana tampilan `TipTimeLayout` akan muncul dengan tema `TipTimeTheme`. Ini membantu pengembang untuk memverifikasi dan menyempurnakan tampilan UI secara langsung dalam lingkungan pengembangan sebelum aplikasi dijalankan secara penuh.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Dengan pendekatan ini Jetpack Compose, UI dibuat sebagai komposisi (compose) atau komponen dari elemen-elemen fungsional yang ditulis dalam Kotlin. Setiap komponen UI dihubungkan langsung dengan logika aplikasi menggunakan state-driven programming.

Sedangkan dengan pendekatan XML, UI dan logikanya dibuat secara terpisah, UI dibuat dalam file xml (main_activity.xml) dan untuk logikanya di file Kotlin. Untuk menghubungkannya dilakukan di file Kotlin menggunakan findViewById atau View Binding (direkomendasikan).

Beberapa contoh perbedaan implementasi:

1. Menampilkan teks “Calculate Tip”:
 - Pada Jetpack Compose, menggunakan komponen Text yang disertai dengan modifier yang berupa objek Modifier untuk memodifikasi layout seperti mengatur padding.
 - Sedangkan pada XML, menggunakan element TextView, yang di mana posisi dan style ditentukan oleh atribut XML, misalnya atribut android:text dan android:layout_marginStart digunakan untuk menentukan teks dan margin. Namun, tidak ada keterikatan langsung antara UI dan state aplikasi dalam file XML.
2. Input (EditText) seperti Bill Amount:
 - Pada Jetpack Compose, untuk memasukkan jumlah tagihan menggunakan komponen TextField dalam bentuk fungsi compose EditNumberField(). Pengguna dapat langsung memasukkan nilai melalui UI ini. Fungsi mengelola input dengan menggunakan state

(amountInput) yang di-update setiap kali pengguna memasukkan angka (nilai).

- Sedangkan Pada XML, menggunakan element `TextInputLayout` dan `TextInputEditText`, dengan pendekatan ini UI didefinisikan di file XML, namun perubahan input akan ditangani di kode Kotlin dengan `TextWatcher`

3. Dropdown untuk Persentase Tip:

- Pada Jetpack Compose, menggunakan `ExposedDropdownMenuBox`. UI diperbarui secara otomatis setiap kali pilihan persentase tip berubah.
- Sedangkan pada XML, menggunakan `AutoCompleteTextView` di dalam `TextInputLayout`. Interaksi ditangani di file Kotlin menggunakan `setOnClickListener` untuk memperbarui pilihan.

4. Switch untuk Pembulatan Tip:

- Pada Jetpack Compose, menggunakan komponen `Switch()`. Pilihan langsung terikat/terhubung langsung ke state `roundup` yang mengontrol apakah tip dibulatkan atau tidak
- Sedangkan pada XML, mendeklarasikan elemen `SwitchCompat` dalam XML. Interaksi pengguna dengan switch ditangani pengguna menggunakan `setOnCheckedChangeListener` di dalam kode Kotlin untuk mengubah status `roundup`.

Kelebihan dan Kekurangan Jetpack Compose

Kelebihan:

1. Bersifat deklaratif dan terintegrasi dengan logika aplikasi, sehingga lebih mudah untuk mengelola interaksi antara UI dan logika aplikasi.
2. Memungkinkan penulisan kode menjadi lebih simpel dan ringkas
3. UI lebih responsif dan dinamis, karena UI secara otomatis terupdate ketika state berubah

4. Pengelolaan layout yang lebih mudah dimodifikasi, karena dengan menggunakan fungsi-fungsi compose memungkinkan penggunaan kembali komponen UI secara lebih fleksibel.

Kekurangan:

1. Memerlukan sedikit waktu untuk memahami konsep-konsep baru seperti Modifier, state, dan remember, dll, bagi yang baru mempelajari pendekatan deklaratif.
2. Kurangnya kompatibilitas pendekatan sebelumnya sehingga membuat transisi penggunaan pendekatan menjadi lebih sulit
3. Ketergantungan pada versi Android dan Tools/Fitur karena terbilang masih baru dan belum sepenuhnya matang (masih perlu banyak pembaruan)

Kelebihan dan Kekurangan Jetpack Compose

Kelebihan:

1. Kompatibilitas lebih luas, karena banyak library UI dan komponen yang sudah tersedia sebelumnya dan stabil untuk digunakan
2. Lebih familiar bagi pengembang berpengalaman, karena termasuk pendekatan imperative (lebih duluan banyak digunakan sebelum declarative).
3. Tools dan Dokumentasi yang lengkap dan matang, karena sudah ada sejak lama.

Kekurangan:

1. Lebih sulit dikelola, karena UI dan logika aplikasi dibuat secara terpisah sehingga perlu mengelola secara lebih manual.
2. UI tidak terupdate secara otomatis (tidak responsive).
3. Kode lebih panjang dan rumit.

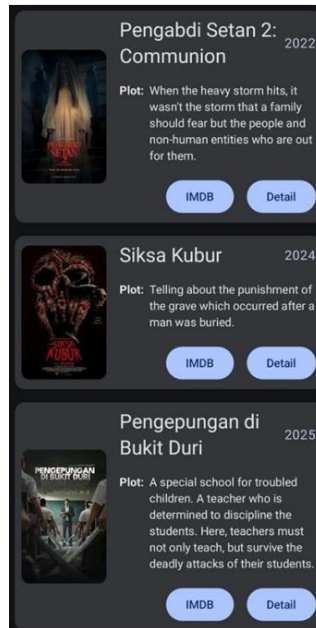
MODUL 3 : BUILD A SCROLLABLE LIST

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 19. Contoh UI List Soal 1 Modul 3

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 20. Contoh UI Detail Soal 1 Modul 3

A. Source Code

XML:

MainActivity.kt

Tabel 10. Source Code MainActivity.kt Soal 1 Modul 3 XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import
	com.example.booklist.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	val binding =
14	ActivityMainBinding.inflate(layoutInflater)
	setContentView(binding.root)
15	}
16	}

Book.kt

Tabel 11. Source Code Book.kt Soal 1 Modul 3 XML

1	package com.example.booklist
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,
8	@StringRes val yearResourceId: Int,
9	@StringRes val aboutResourceId: Int,
10	@StringRes val webUrlResourceId: Int,
11	@DrawableRes val imageResourceId: Int
12	)

BookFragment.kt

```

1 package com.example.booklist
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.findNavController
9 import androidx.recyclerview.widget.LinearLayoutManager
10 import
    com.example.booklist.databinding.FragmentBookBinding
11
12 class BookFragment : Fragment() {
13
14     private var _binding: FragmentBookBinding? = null
15     private val binding get() = _binding!!
16
17     private val bookList = listOf(
18         Book(R.string.book_title1, R.string.book_year1,
19 R.string.book_about1, R.string.book_webUrl1,
20 R.drawable.image1),
21         Book(R.string.book_title2, R.string.book_year2,
22 R.string.book_about2, R.string.book_webUrl2,
23 R.drawable.image2),
24         Book(R.string.book_title3, R.string.book_year3,
25 R.string.book_about3, R.string.book_webUrl3,
26 R.drawable.image3),
27         Book(R.string.book_title4, R.string.book_year4,
28 R.string.book_about4, R.string.book_webUrl4,
29 R.drawable.image4),
30         Book(R.string.book_title5, R.string.book_year5,
31 R.string.book_about5, R.string.book_webUrl5,
32 R.drawable.image5),
33         Book(R.string.book_title6, R.string.book_year6,
34 R.string.book_about6, R.string.book_webUrl6,
35 R.drawable.image6),
36         Book(R.string.book_title7, R.string.book_year7,
37 R.string.book_about7, R.string.book_webUrl7,
38 R.drawable.image7),
39         Book(R.string.book_title8, R.string.book_year8,
40 R.string.book_about8, R.string.book_webUrl8,
41 R.drawable.image8)
42     )
43
44     override fun onCreateView(inflater: LayoutInflater,
45 container: ViewGroup?, savedInstanceState: Bundle?): View {
46 {
47     _binding = FragmentBookBinding.inflate(inflater,
48 container, false)

```

30	return binding.root
31	}
32	
33	override fun onCreateView(view: View, savedInstanceState: Bundle?) {
34	val recyclerView = binding.recyclerView
35	recyclerView.layoutManager =
36	LinearLayoutManager(requireContext())
37	recyclerView.adapter =
38	BookAdapter(requireContext(), bookList) { selectedBook ->
39	val bundle = Bundle().apply {
40	putInt("titleRes",
41	selectedBook.titleResourceId)
42	putInt("yearRes",
43	selectedBook.yearResourceId)
44	putInt("aboutRes",
45	selectedBook.aboutResourceId)
46	putInt("imageResId",
47	selectedBook.imageResourceId)
48	}
49	findNavController().navigate(R.id.action_bookFragment_to_detailFragment, bundle)
50	}
51	}

DetailFragment.kt

Tabel 12. Source Code DetailFragment.kt Soal 1 Modul 3 XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import
9	com.example.booklist.databinding.FragmentDetailBinding
10	
11	class DetailFragment : Fragment() {
12	private var _binding: FragmentDetailBinding? = null
13	private val binding get() = _binding!!

```

14
15     override fun onCreateView(
16         inflater: LayoutInflater,
17         container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View? {
20         _binding =
21         FragmentDetailBinding.inflate(inflater, container, false)
22         return binding.root
23     }
24
25     override fun onViewCreated(view: View,
26         savedInstanceState: Bundle?) {
27         val args = arguments
28         val titleRes = args?.getInt("titleRes") ?: 0
29         val yearRes = args?.getInt("yearRes") ?: 0
30         val aboutRes = args?.getInt("aboutRes") ?: 0
31         val imageResId = args?.getInt("imageResId") ?: 0
32
33         binding.bookTitle.setText(titleRes)
34         binding.bookYear.setText(yearRes)
35         binding.bookAbout.setText(aboutRes)
36         binding.bookImage.setImageResource(imageResId)
37     }
38
39     override fun onDestroyView() {
40         super.onDestroyView()
41         _binding = null
42     }
43 }

```

MyItemRecyclerViewAdapter.kt

Tabel 13. Source Code MyItemRecyclerViewAdapter.kt Soal 1 Modul 3 XML

```

1 package com.example.booklist
2
3 import android.content.Context
4 import android.content.Intent
5 import android.net.Uri
6 import android.view.LayoutInflater
7 import android.view.ViewGroup
8 import androidx.recyclerview.widget.RecyclerView
9 import com.example.booklist.databinding.ItemBookBinding
10
11 class BookAdapter(
12     private val context: Context,
13     private val books: List<Book>,
14     private val onAboutClick: (Book) -> Unit

```



```

15 ) : RecyclerView.Adapter<BookAdapter.BookViewHolder>() {
16
17     inner class BookViewHolder(val binding:
ItemBookBinding) : RecyclerView.ViewHolder(binding.root)
18
19     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): BookViewHolder {
20         val binding =
ItemBookBinding.inflate(LayoutInflater.from(context),
parent, false)
21         return BookViewHolder(binding)
22     }
23
24     override fun onBindViewHolder(holder: BookViewHolder,
position: Int) {
25         val book = books[position]
26         with(holder.binding) {
27             bookTitle.text =
context.getString(book.titleResourceId)
28             bookYear.text =
context.getString(book.yearResourceId)
29             bookAbout.text =
context.getString(book.aboutResourceId)
30
31             bookImage.setImageResource(book.imageResourceId)
32
33             btnBuy.setOnClickListener {
                val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(context.getString(book.webUrlResourceId)))
34                 context.startActivity(intent)
35             }
36
37             btnAbout.setOnClickListener {
38                 onAboutClick(book)
39             }
40         }
41     }
42
43     override fun getItemCount() = books.size
44 }

```

activity_main.xml

Tabel 14. Source Code activity_main.xml Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/andro id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<FrameLayout
11	android:id="@+id/frame_layout"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintHorizontal_bias="1.0"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintVertical_bias="0.0">
20	
21	<androidx.fragment.app.FragmentContainerView
22	android:id="@+id/nav_host_fragment"
23	android:name="androidx.navigation.fragment.NavHostFragm ent"
24	android:layout_width="match_parent"
25	android:layout_height="match_parent"
26	app:navGraph="@navigation/nav_graph"
27	app:defaultNavHost="true" />
28	
29	</FrameLayout>
30	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_book.xml

Tabel 15. Source Code *fragment_book.xml* Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/andro id"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".BookFragment">
9	
10	<androidx.recyclerview.widget.RecyclerView
11	android:id="@+id/recyclerView"
12	android:layout_width="0dp"
13	android:layout_height="0dp"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintBottom_toBottomOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintEnd_toEndOf="parent" />
18	
19	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_detail.xml

Tabel 16. Source Code fragment_detail.xml Soal 1 Modul 3 XML

1	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
2	xmlns:app="http://schemas.android.com/apk/res-auto"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:padding="16dp"
8	android:background="#1C1C1C">
9	
10	<androidx.constraintlayout.widget.ConstraintLayout
11	android:layout_width="match_parent"
12	android:layout_height="match_parent">
13	
14	<androidx.cardview.widget.CardView
15	android:id="@+id/card_image"
16	android:layout_width="wrap_content"
17	android:layout_height="0dp"
18	app:cardCornerRadius="8dp"
19	app:layout_constraintEnd_toEndOf="parent"
20	app:layout_constraintStart_toStartOf="parent"
21	app:layout_constraintTop_toTopOf="parent">
22	
23	<ImageView
24	android:id="@+id/bookImage"
25	android:layout_width="350dp"
26	android:layout_height="520dp"
27	android:scaleType="centerCrop"
28	android:src="@drawable/image1" />
29	</androidx.cardview.widget.CardView>
30	
31	<TextView
32	android:id="@+id/bookTitle"
33	android:layout_width="266dp"
34	android:layout_height="0dp"
35	android:layout_marginStart="8dp"
36	android:layout_marginTop="23dp"
37	android:layout_marginEnd="8dp"
38	android:text="@string/book_title1"
39	android:textColor="@color/white"
40	android:textSize="24sp"
41	android:textStyle="bold"
42	app:layout_constraintEnd_toStartOf="@+id/bookYear"
43	app:layout_constraintStart_toStartOf="parent"
44	app:layout_constraintTop_toBottomOf="@+id/card_image"
	/>
45	

```

46         <TextView
47             android:id="@+id/bookYear"
48             android:layout_width="63dp"
49             android:layout_height="33dp"
50             android:layout_marginTop="24dp"
51             android:layout_marginEnd="8dp"
52             android:layout_marginBottom="38dp"
53             android:text="@string/book_year1"
54             android:textColor="@color/white"
55             android:textSize="24sp"
56             app:layout_constraintBottom_toTopOf="@+id/bookAbout"
57             app:layout_constraintEnd_toEndOf="parent"
58 app:layout_constraintStart_toEndOf="@+id/bookTitle"
59             app:layout_constraintTop_toBottomOf="@+id/card_image"
60         />
61
62         <TextView
63             android:id="@+id/textTentang"
64             android:layout_width="61dp"
65             android:layout_height="23dp"
66             android:layout_marginStart="17dp"
67             android:layout_marginTop="8dp"
68             android:layout_marginEnd="302dp"
69             android:text="@string/about"
70             android:textColor="@color/white"
71             android:textSize="14sp"
72
73 app:layout_constraintBottom_toTopOf="@+id/bookAbout"
74             app:layout_constraintEnd_toEndOf="parent"
75             app:layout_constraintStart_toStartOf="parent"
76
77 app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
78
79         <TextView
80             android:id="@+id/bookAbout"
81             android:layout_width="346dp"
82             android:layout_height="0dp"
83             android:layout_marginStart="17dp"
84             android:layout_marginEnd="16dp"
85             android:text="@string/book_about1"
86             android:textColor="@color/white"
87             android:textSize="14sp"
88 app:layout_constraintBottom_toBottomOf="parent"
89             app:layout_constraintEnd_toEndOf="parent"
90             app:layout_constraintHorizontal_bias="0.0"
91             app:layout_constraintStart_toStartOf="parent"
92
93 app:layout_constraintTop_toBottomOf="@+id/textTentang"
94         />

```

90	
91	</androidx.constraintlayout.widget.ConstraintLayout>
92	</LinearLayout>

item_book.xml

Tabel 17. Source Code item_book.xml Soal 1 Modul 3 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/andro id"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content">
7	
8	<androidx.cardview.widget.CardView
9	android:layout_width="wrap_content"
10	android:layout_height="wrap_content"
11	android:layout_margin="8dp"
12	app:cardBackgroundColor="@color/dark_gray"
13	app:cardCornerRadius="16dp"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintTop_toTopOf="parent">
18	
19	<androidx.constraintlayout.widget.ConstraintLayout
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content">
22	
23	<androidx.cardview.widget.CardView
24	android:id="@+id/card_image"
25	android:layout_width="wrap_content"
26	android:layout_height="wrap_content"
27	android:layout_marginStart="16dp"
28	android:layout_marginTop="16dp"
29	android:layout_marginBottom="16dp"
30	app:cardCornerRadius="8dp"
31	app:layout_constraintBottom_toBottomOf="parent"
32	app:layout_constraintStart_toStartOf="parent"
33	app:layout_constraintTop_toTopOf="parent">
34	
35	<ImageView
36	android:id="@+id/bookImage"
37	android:layout_width="123dp"
38	android:layout_height="200dp"
39	android:scaleType="centerCrop"
40	app:srcCompat="@drawable/image1" />
41	</androidx.cardview.widget.CardView>
42	
43	<TextView
44	android:id="@+id/bookTitle"

45	android:layout_width="193dp"
46	android:layout_height="wrap_content"
47	android:layout_marginTop="8dp"
48	android:layout_marginEnd="5dp"
49	android:text="@string/book_title1"
50	android:textColor="@color/white"
51	android:textSize="24sp"
52	android:textStyle="bold"
53	
54	app:layout_constraintEnd_toStartOf="@+id/bookYear"
55	app:layout_constraintStart_toEndOf="@+id/card_image"
56	app:layout_constraintTop_toTopOf="parent" />
57	<TextView
58	android:id="@+id/bookYear"
59	android:layout_width="wrap_content"
60	android:layout_height="wrap_content"
61	android:layout_marginTop="16dp"
62	android:layout_marginEnd="16dp"
63	android:text="@string/book_year1"
64	android:textColor="@color/white"
65	app:layout_constraintEnd_toEndOf="parent"
66	app:layout_constraintTop_toTopOf="parent"
	/>
67	
68	<TextView
69	android:id="@+id/textTentang"
70	android:layout_width="wrap_content"
71	android:layout_height="0dp"
72	android:layout_marginStart="8dp"
73	android:layout_marginTop="24dp"
74	android:text="@string/about"
75	android:textColor="@color/white"
76	android:textSize="14sp"
77	app:layout_constraintEnd_toStartOf="@+id/bookAbout"
78	app:layout_constraintStart_toEndOf="@+id/card_image"
79	app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
80	
81	<TextView
82	android:id="@+id/bookAbout"
83	android:layout_width="184dp"
84	android:layout_height="wrap_content"
85	android:layout_marginStart="6dp"
86	android:layout_marginTop="24dp"

```

87         android:layout_marginEnd="9dp"
88         android:text="@string/book_about1"
89         android:textColor="@color/white"
90         android:textSize="14sp"
91         app:layout_constraintEnd_toEndOf="parent"
92     app:layout_constraintStart_toEndOf="@+id/textTentang"
93     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
94
95     <Button
96         android:id="@+id/btnBuy"
97         android:layout_width="wrap_content"
98         android:layout_height="wrap_content"
99         android:layout_marginStart="55dp"
100        android:layout_marginTop="27dp"
101        android:layout_marginBottom="8dp"
102        android:text="@string/beli_button"
103        android:textSize="14sp"
104    app:layout_constraintBottom_toBottomOf="parent"
105    app:layout_constraintEnd_toStartOf="@+id/btnAbout"
106    app:layout_constraintStart_toEndOf="@+id/card_image"
107    app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
108
109    <Button
110        android:id="@+id/btnAbout"
111        android:layout_width="wrap_content"
112        android:layout_height="wrap_content"
113        android:layout_marginStart="8dp"
114        android:layout_marginTop="27dp"
115        android:layout_marginEnd="16dp"
116        android:layout_marginBottom="8dp"
117        android:text="@string/detail_button"
118        android:textSize="14sp"
119    app:layout_constraintBottom_toBottomOf="parent"
120    app:layout_constraintEnd_toEndOf="parent"
121    app:layout_constraintStart_toEndOf="@+id/btnBuy"
122    app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
123 </androidx.constraintlayout.widget.ConstraintLayout>
124 </androidx.cardview.widget.CardView>
125 </androidx.constraintlayout.widget.ConstraintLayout>

```

Compose:

MainActivity.kt

Tabel 18. Source Code MainActivity.kt Soal 1 Modul 3 Compose

```
1 package com.example.booklist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.WindowInsets
8 import androidx.compose.foundation.layout.asPaddingValues
9 import
10 androidx.compose.foundation.layout.calculateEndPadding
11 import
12 androidx.compose.foundation.layout.calculateStartPadding
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.safeDrawing
16 import
17 androidx.compose.foundation.layout.statusBarsPadding
18 import androidx.compose.material3.Surface
19 import androidx.compose.runtime.Composable
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.platform.LocalLayoutDirection
22 import androidx.compose.ui.tooling.preview.Preview
23 import androidx.compose.ui.unit.dp
24 import androidx.navigation.compose.NavHost
25 import androidx.navigation.compose.composable
26 import androidx.navigation.compose.rememberNavController
27 import com.example.booklist.data.Datasource
28 import com.example.booklist.model.Book
29 import com.example.booklist.screens.BookDetailScreen
30 import com.example.booklist.screens.BookListScreen
31 import com.example.booklist.ui.theme.BookListTheme
32
33 class MainActivity : ComponentActivity() {
34     override fun onCreate(savedInstanceState: Bundle?) {
35         super.onCreate(savedInstanceState)
36         enableEdgeToEdge()
37         setContent {
38             BookListTheme {
39                 BooksApp()
40             }
41         }
42     }
43 }
```

```

42 @Composable
43 fun BooksApp() {
44     val layoutDirection = LocalLayoutDirection.current
45     val navController = rememberNavController()
46     val books = Datasource().loadBooks()
47     Surface(
48         modifier = Modifier
49             .fillMaxSize()
50             .statusBarsPadding()
51             .padding(
52                 start
53                 = WindowInsets.safeDrawing.asPaddingValues().calculateStartPadding(layoutDirection),
54                 end
55                 = WindowInsets.safeDrawing.asPaddingValues().calculateEndPadding(layoutDirection),
56             ) {
57         NavHost(navController = navController,
58             startDestination = "ListScreen") {
59             composable(route = "ListScreen") {
60                 BookListScreen(bookList = books,
61                     onDetailClick = {
62                         index
63                         -> navController.navigate("detail/$index")
64                     })
65             }
66             composable("detail/{index}") { backStackEntry
67                 ->
68                 val indexString
69                 = backStackEntry.arguments?.getString("index")
70                 val index = indexString?.toIntOrNull()
71                 val book
72                 = index?.let {
73                     books.getOrNull(it) }
74                 if (book != null) {
75                     BookDetailScreen(book = book)
76                 } else {
77                     androidx.compose.material3.Text("Buku
78                     tidak ditemukan", modifier = Modifier.padding(16.dp))
79                 }
80             }
81         }
82     }
83 }

```

Datasource.kt

Tabel 19. Source Code Datasource.kt Soal 1 Modul 3 Compose

1	package com.example.booklist.data
2	
3	import com.example.booklist.R
4	import com.example.booklist.model.Book
5	
6	class Datasource {
7	fun loadBooks(): List<Book> {
8	return listOf<Book> (
9	Book(R.string.book_title1,
	R.string.book_year1, R.string.book_about1,
	R.string.book_webUrl1, R.drawable.image1) ,
10	Book(R.string.book_title2,
	R.string.book_year2, R.string.book_about2,
	R.string.book_webUrl2, R.drawable.image2),
11	Book(R.string.book_title3,
	R.string.book_year3, R.string.book_about3,
	R.string.book_webUrl3, R.drawable.image3),
12	Book(R.string.book_title4,
	R.string.book_year4, R.string.book_about4,
	R.string.book_webUrl4, R.drawable.image4),
13	Book(R.string.book_title5,
	R.string.book_year5, R.string.book_about5,
	R.string.book_webUrl5, R.drawable.image5),
14	Book(R.string.book_title6,
	R.string.book_year6, R.string.book_about6,
	R.string.book_webUrl6, R.drawable.image6),
15	Book(R.string.book_title7,
	R.string.book_year7, R.string.book_about7,
	R.string.book_webUrl7, R.drawable.image7),
16	Book(R.string.book_title8,
	R.string.book_year8, R.string.book_about8,
	R.string.book_webUrl8, R.drawable.image8)
17	)
18	}
19	}

Book.kt

Tabel 20. Source Code Book.kt Soal 1 Modul 3 Compose

1	package com.example.booklist.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,

37	painter	=
38	painterResource(book.imageResourceId),	
39	contentDescription = null,	
40	modifier = Modifier	
	.size(width = 220.dp, height =	
	350.dp)	
41	.clip(RoundedCornerShape(16.dp)),	
42	contentScale = ContentScale.Crop	
43	)	
44		
45	Spacer(modifier = Modifier.width(16.dp))	
46		
47	Column(	
48	modifier = Modifier.weight(1f),	
49	horizontalAlignment = Alignment.Start	
50	) {	
51	Text(	
52	text	=
	LocalContext.current.getString(book.titleResourceId),	
53	style	=
	MaterialTheme.typography.labelLarge,	
54	fontSize = 26.sp	
55	)	
56	Text(	
57	text	=
	LocalContext.current.getString(book.yearResourceId),	
58	style	=
	MaterialTheme.typography.bodyMedium,	
59	fontSize = 20.sp	
60	)	
61	Spacer(modifier = Modifier.height(8.dp))	
62	Text("Tentang:", style	=
	MaterialTheme.typography.bodySmall)	
63	Text(	
64	text	=
	LocalContext.current.getString(book.aboutResourceId),	
65	style	=
	MaterialTheme.typography.bodySmall,	
66	fontSize = 14.sp	
67	)	
68	}	
69	}	
70	} else {	
71	Column(modifier = Modifier.padding(16.dp)) {	
72	Image(	
73	painter	=
	painterResource(book.imageResourceId),	
74	contentDescription = null,	
75	modifier = Modifier	

76	.fillMaxWidth()	
77	.height(500.dp)	
78	.clip(RoundedCornerShape(16.dp)),	
79	contentScale = ContentScale.Crop	
80	)	
81	Spacer(modifier = Modifier.height(8.dp))	
82	Row {	
83	Text(	
84	text	=
85	LocalContext.current.getString(book.titleResourceId),	
86	modifier = Modifier.weight(1f),	
87	style	=
88	MaterialTheme.typography.labelLarge,	
89	fontSize = 26.sp	
90	)	
91	Text(	
92	text	=
93	LocalContext.current.getString(book.yearResourceId),	
94	style	=
95	MaterialTheme.typography.bodyMedium,	
96	fontSize = 20.sp	
97	)	
98	}	
99	Spacer(modifier = Modifier.height(8.dp))	
100	Text("Tentang:", style	=
101	MaterialTheme.typography.bodySmall)	
102	Text(	
103	text	=
104	LocalContext.current.getString(book.aboutResourceId),	
	style	=
	MaterialTheme.typography.bodySmall,	
	fontSize = 14.sp	
	)	
	}	
	}	
	}	

BookListScreen.kt

Tabel 22. Source Code BookListScreen.kt Soal 1 Modul 3 Compose

1	package com.example.booklist.screens
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row

```

9      import androidx.compose.foundation.layout.Spacer
10     import androidx.compose.foundation.layout.fillMaxWidth
11     import androidx.compose.foundation.layout.height
12     import androidx.compose.foundation.layout.padding
13     import androidx.compose.foundation.layout.size
14     import androidx.compose.foundation.layout.width
15     import androidx.compose.foundation.lazy.LazyColumn
16     import androidx.compose.foundation.lazy.items
17     import androidx.compose.foundation.shape.RoundedCornerShape
18     import androidx.compose.material3.Button
19     import androidx.compose.material3.Card
20     import androidx.compose.material3.MaterialTheme
21     import androidx.compose.material3.Text
22     import androidx.compose.runtime.Composable
23     import androidx.compose.ui.Alignment
24     import androidx.compose.ui.Modifier
25     import androidx.compose.ui.draw.clip
26     import androidx.compose.ui.layout.ContentScale
27     import androidx.compose.ui.platform.LocalContext
28     import androidx.compose.ui.res.painterResource
29     import androidx.compose.ui.res.stringResource
30     import androidx.compose.ui.text.style.TextOverflow
31     import androidx.compose.ui.tooling.preview.Preview
32     import androidx.compose.ui.unit.dp
33     import androidx.compose.ui.unit.sp
34     import com.example.booklist.BooksApp
35     import com.example.booklist.model.Book
36
37     @Composable
38     fun BookListScreen(bookList: List<Book>, onDetailClick:
39       (Int) -> Unit, modifier: Modifier = Modifier) {
40         LazyColumn(modifier = modifier) {
41             items(bookList.indices.toList()) {
42                 index -> BookCard(
43                     book = bookList[index],
44                     onDetailClick = { onDetailClick(index) }
45                 )
46             }
47         }
48
49         @Composable
50         fun BookCard(book: Book, onDetailClick: () -> Unit, modifier:
51           Modifier = Modifier) {
52             Card(
53                 modifier = modifier
54                 .fillMaxWidth()
55                 .padding(8.dp),
56                 shape = RoundedCornerShape(16.dp)

```

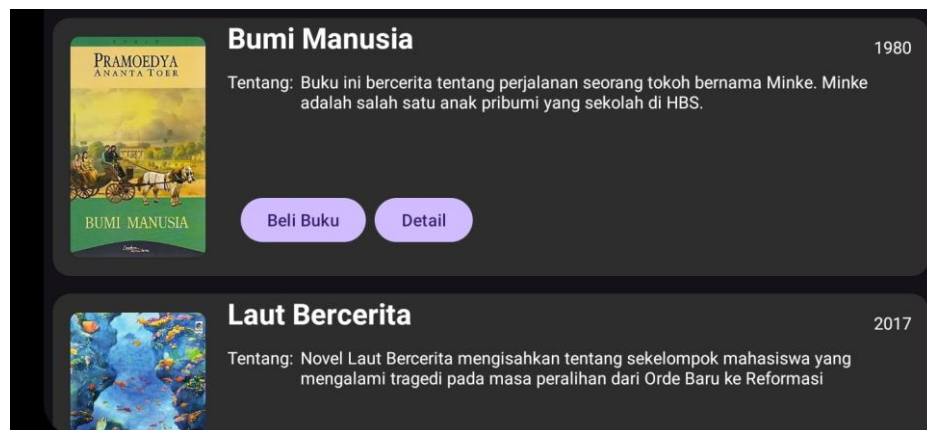
56	) {	
57	val context = LocalContext.current	
58	Row(	
59	modifier = Modifier	
60	.padding(16.dp)	
61	.fillMaxWidth()	
62	) {	
63	Image(	
64	painter	=
65	painterResource(book.imageResourceId),	
66	contentDescription	=
67	stringResource(book.aboutResourceId),	
68	modifier = Modifier	
69	.size(width = 120.dp, height = 200.dp)	
70	.clip(RoundedCornerShape(12.dp))	
71	.align(Alignment.CenterVertically),	
72	contentScale = ContentScale.Crop	
73	) {	
74	Spacer(modifier = Modifier.width(12.dp))	
75	Column(	
76	modifier = Modifier	
77	.weight(1f)	
78	.align(Alignment.CenterVertically)	
79	) {	
80	Row(	
81	modifier = Modifier.fillMaxWidth(),	
82	horizontalArrangement	=
83	Arrangement.SpaceBetween	
84	) {	
85	Text(	
86	text	=
87	context.getString(book.titleResourceId),	
88	modifier = Modifier.weight(1f),	
89	style	=
90	MaterialTheme.typography.labelLarge,	
91	overflow = TextOverflow.Ellipsis,	
92	fontSize = 26.sp	
93	) {	
94	Text(	
95	text	=
96	context.getString(book.yearResourceId),	
97	style	=
	MaterialTheme.typography.bodyMedium,	
	fontSize = 14.sp	
	) {	
	Spacer(modifier = Modifier.height(20.dp))	
	Row {	

98	Text(	
99	text = "Tentang: ",	
100	style	=
	MaterialTheme.typography.bodySmall,	
101	fontSize = 14.sp	
102	)	
103	Text(	
104	text	=
	context.getString(book.aboutResourceId),	
105	style	=
	MaterialTheme.typography.bodySmall,	
106	fontSize = 14.sp	
107	)	
108	}	
109	Spacer(modifier = Modifier.height(16.dp))	
110	Row(modifier = Modifier.fillMaxWidth(),	
111	horizontalArrangement = Arrangement.End)	
	{	
112	Button(onClick = {	
113	val webUrl	=
	context.getString(book.webUrlResourceId)	
114	val intent	=
	Intent(Intent.ACTION_VIEW, Uri.parse(webUrl))	
115	context.startActivity(intent)	
116	}) {	
117	Text("Beli Buku")	
118	}	
119	Spacer(modifier = Modifier.width(8.dp))	
120	Button(onClick = {	
121	onDetailClick()	
122	}) {	
123	Text("Detail")	
124	}	
125	}	
126		
127	}	
128	}	
129		
130	}	
131	}	

B. Output Program



Gambar 21. Tampilan Awal UI Soal 1 Modul 3 XML



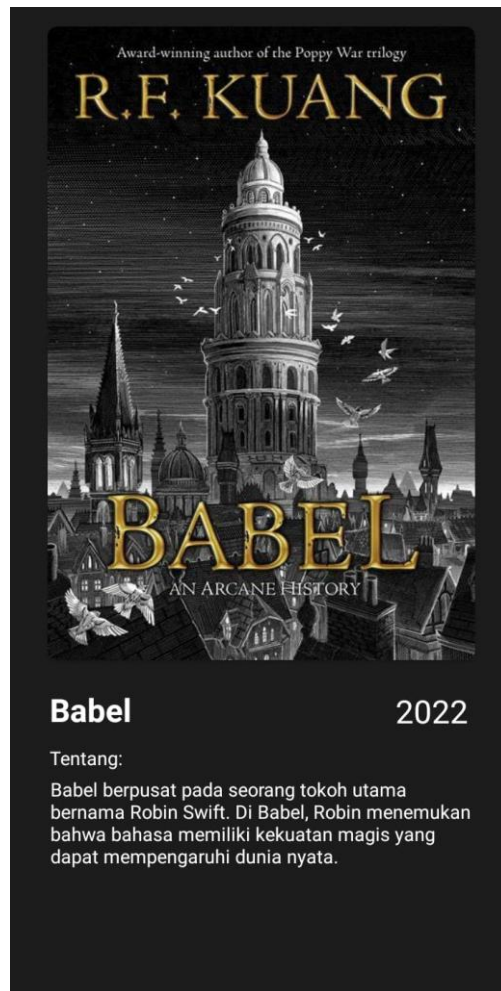
Gambar 22. Tampilan Awal UI Landscape Soal 1 Modul 3 XML



Gambar 23. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 3 XML



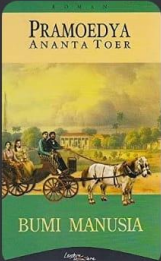
Gambar 24. Tampilan Ketika Discroll Soal 1 Modul 3 XML



Gambar 25. Tampilan Tombol Detail Ditekan Soal 1 Modul 3 XML



Gambar 26. Tampilan Tombol Detail Ditekan Landscape Soal 1 Modul 3 XML



Bumi Manusia

1980

Tentang: Buku ini bercerita tentang perjalanan seorang tokoh bernama Minke. Minke adalah salah satu anak pribumi yang sekolah di HBS.

[Beli Buku](#)
[Detail](#)

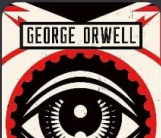


Laut Bercerita

2017

Tentang: Novel Laut Bercerita mengisahkan tentang sekelompok mahasiswa yang mengalami tragedi pada masa peralihan dari Orde Baru ke Reformasi

[Beli Buku](#)
[Detail](#)



1984

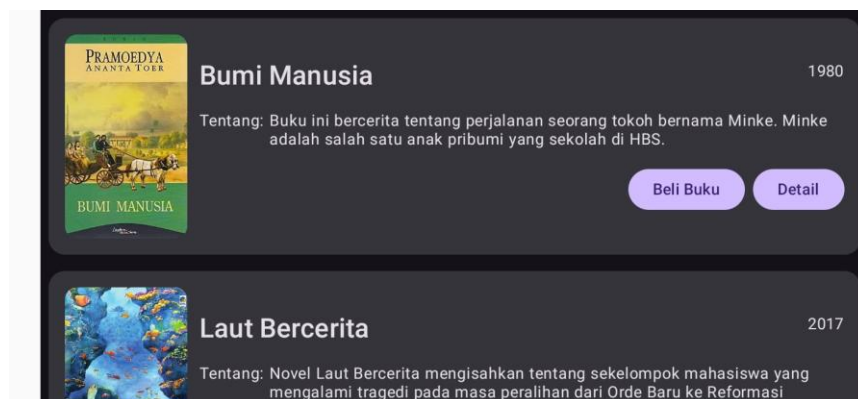
1949

Tentang: Dalam dunia yang penuh pengawasan dan propaganda ini, Partai mengendalikan semua aspek kehidupan warganya. Kehidupan pribadi dan pemikiran

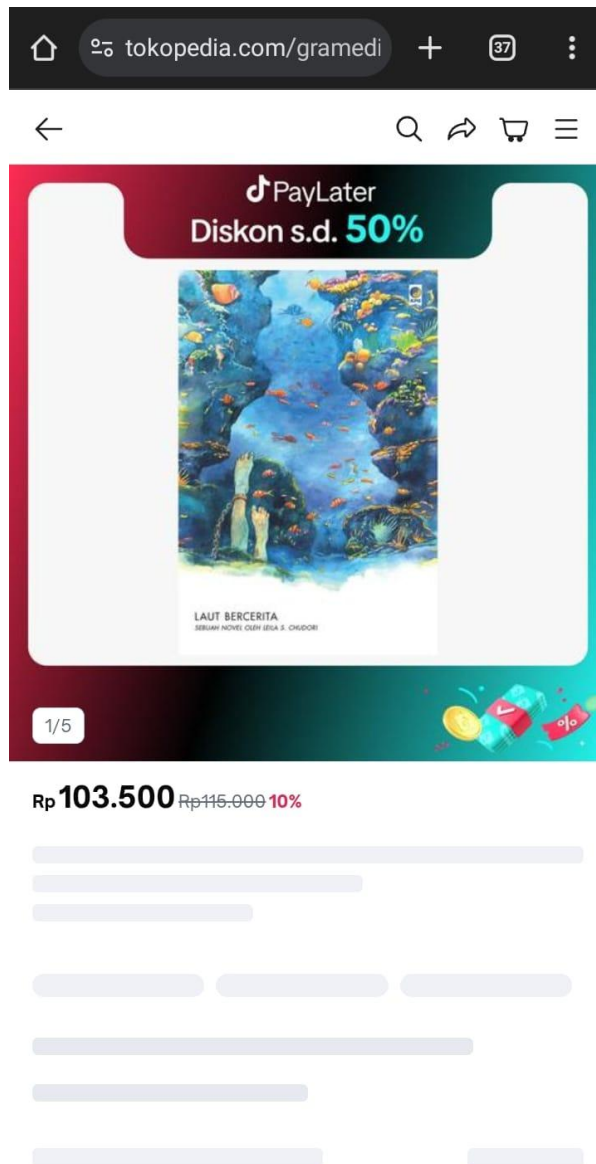
Gambar 27. Tampilan Awal UI Soal 1 Modul 3 Compose



Gambar 28. Tampilan Ketika Discroll Soal 1 Modul 3 Compose



Gambar 29. Tampilan Awal UI Landscape Soal 1 Modul 3 Compose



Gambar 30. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 3 Compose



Gambar 31. Tampilan Detail Soal 1 Modul 3 Compose



Gambar 32. Tampilan Detail Landscape Soal 1 Modul 3 Compose

C. Pembahasan

XML:

MainActivity.kt

Pada baris [1], dideklarasikan nama package file Kotlin

Pada baris [3], diimport bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada baris [4], diimport enableEdgeToEdge agar konten bisa tampil sampai ke pinggir layar.

Pada baris [5], diimport AppCompatActivity superclass untuk activity yang menggunakan fitur-fitur dari AndroidX.

Pada baris [6], diimport ViewCompat digunakan untuk manipulasi property View.

Pada baris [7], diimport WindowInsetsCompat untuk mengatur tata letak yang mempertimbangkan status bar dan navigation bar.

Pada baris [8], diimport ActivityMainBinding class binding otomatis yang dihasilkan dari file XML activity_main.xml.

Pada baris [10], mendeklarasikan kelas MainActivity yang merupakan turunan dari AppCompatActivity sebagai titik awal saat aplikasi dijalankan.

Pada baris [11], mendefinisikan method onCreate, yaitu fungsi lifecycle yang dipanggil saat activity pertama kali dibuat.

Pada baris [12], memanggil implementasi onCreate di superclass untuk memastikan sistem Android tetap menjalankan proses inisialisasi standar activity.

Pada baris [13], membuat objek binding dengan cara menginflate layout XML yaitu activity_main.xml agar bisa diakses lewat kode Kotlin.

Pada baris [14], menetapkan root dari layout yang diinflate sebagai tampilan utama activity.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport DrawableRes untuk anotasi @DrawableRes yang menandakan bahwa parameter bertipe Int tersebut adalah resource drawable atau gambar.

Pada baris [4], diimport StringRes untuk anotasi @StringRes yang menandakan bahwa parameter Int tersebut adalah resource string dari folder res/values/strings.xml.

Pada baris [6-11], mendefinisikan kelas data Book untuk menyimpan data buku.

@StringRes val titleResourceId: Int adalah properti untuk menyimpan ID dari resource judul buku. Anotasi @StringRes memberi tahu Android Studio bahwa nilai ini harus berasal dari resource string (R.string.nama_judul). @StringRes val yearResourceId: Int adalah ID resource string untuk tahun terbit buku (misal "2022"). @StringRes val aboutResourceId: Int menyimpan ID resource string yang menjelaskan isi buku atau deskripsinya. @StringRes val webUrlResourceId: Int menyimpan ID resource string berupa URL website (untuk beli buku). @DrawableRes val imageResourceId: Int menyimpan ID resource gambar (drawable)

untuk sampul buku. Anotasi `@DrawableRes` menjamin bahwa nilai ini berasal dari folder `res/drawable`.

BookFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-10], diimport berbagai komponen penting yang dibutuhkan untuk menjalankan fungsi fragment ini. `Bundle` digunakan untuk mengirim data antar fragment. `LayoutInflater`, `View`, dan `ViewGroup` digunakan untuk mengatur tampilan layout. `Fragment` adalah kelas dasar untuk membuat fragment di Android.

`findNavController()` adalah fungsi untuk melakukan navigasi antar fragment dengan `Jetpack Navigation`. `LinearLayoutManager` digunakan untuk menampilkan data list secara vertikal di dalam `RecyclerView`. Terakhir, `FragmentBookBinding` merupakan class binding otomatis yang dihasilkan dari file XML `fragment_book.xml`, sehingga kita bisa mengakses komponen XML-nya langsung dari Kotlin.

Pada baris [12], mendeklarasikan class fragment.

Pada baris [14], menyimpan binding dengan tipe nullable untuk menghindari memory leak.

Pada baris [15], memberi akses non null ke binding saat view aktif.

Pada baris [17-26] mendefinisikan list berisi 8 buku, setiap buku merupakan objek dari class `Book` yang mengambil data dari `strings.xml` dan gambar dari `drawable`.

Pada baris [28-31], method `onCreateView()` digunakan untuk menginflate layout `fragment_book.xml` menjadi view, lalu `FragmentBookBinding.inflate()` menghubungkan dengan `ViewBinding`. Return `binding.root` untuk mengembalikan view utama fragment.

Pada baris [33-45], method `onViewCreated()` dipanggil setelah layout fragment berhasil ditampilkan di layar. Di dalamnya, pertama-tama diambil referensi ke

RecyclerView menggunakan `binding.recyclerview`. Kemudian, RecyclerView dikonfigurasi untuk menampilkan daftar item secara vertikal dengan menggunakan `LinearLayoutManager`. Setelah itu, disetel adapter `BookAdapter`, yang akan menampilkan daftar buku dari `bookList`. Adapter juga menerima lambda (callback) yang akan dipanggil saat pengguna mengklik salah satu buku. Ketika buku diklik, dibuat Bundle berisi informasi judul, tahun, deskripsi, dan gambar buku. Bundle ini dikirimkan ke `DetailFragment` menggunakan perintah navigasi `findNavController().navigate()`.

Pada baris [47-51], method `onDestroyView()` dipanggil saat fragment akan dihancurkan (misalnya ketika berpindah ke fragment lain). Di sini, `_binding` diset ke null untuk menghapus referensi ke view yang sudah tidak aktif, sehingga membantu mencegah terjadinya memory leak.

DetailFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–8], diimport komponen-komponen penting yang dibutuhkan dalam fragment ini. Bundle digunakan untuk menerima data yang dikirim dari fragment sebelumnya. Fragment adalah superclass yang digunakan untuk membuat fragment. `LayoutInflater`, `View`, dan `ViewGroup` digunakan untuk mengelola dan menampilkan tampilan (layout) fragment. Terakhir, `FragmentDetailBinding` adalah class `ViewBinding` yang secara otomatis dihasilkan dari file XML `fragment_detail.xml`.

Pada baris [10], dideklarasikan kelas `DetailFragment` yang merupakan turunan dari `Fragment`. Kelas ini bertugas menampilkan detail buku yang dipilih oleh pengguna dari daftar buku.

Pada baris [12], dibuat properti `_binding` yang bertipe nullable dari `FragmentDetailBinding`. Properti ini akan menyimpan objek binding yang

menghubungkan komponen layout dengan kode Kotlin, dan dibuat nullable agar bisa dihapus saat view dihancurkan untuk mencegah memory leak.

Pada baris [13], dibuat properti binding yang bersifat non-nullable dengan cara memaksa akses ke `_binding` menggunakan `!!` untuk mempermudah akses ke komponen layout selama fragment masih dalam keadaan aktif.

Pada baris [15–22], method `onCreateView()` digunakan untuk menginflate layout `fragment_detail.xml` menggunakan `FragmentManager.inflate()`. Hasil inflate disimpan ke `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama dari fragment ini.

Pada baris [24-35], method `onViewCreated()` dijalankan ketika layout fragment sudah selesai ditampilkan. Di sini, kita mengambil data yang dikirim dari `BookFragment` menggunakan objek `arguments`. Kemudian, masing-masing data diambil dari `Bundle` dengan kunci `"titleRes"`, `"yearRes"`, `"aboutRes"`, dan `"imageResId"`. Jika datanya tidak tersedia, maka akan digantikan dengan nilai default 0. Setelah itu, data tersebut ditampilkan ke layout. judul buku di-set ke `bookTitle`, tahun terbit ke `bookYear`, deskripsi ke `bookAbout`, dan gambar sampul ke `bookImage`.

Pada baris [37-41], method `onDestroyView()` dipanggil saat view fragment akan dihancurkan. Di sini, `_binding` diset ke `null` untuk membersihkan referensi terhadap view dan mencegah terjadinya memory leak.

MyItemRecyclerViewAdapter.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–9], diimport berbagai komponen yang dibutuhkan dalam pembuatan adapter `RecyclerView`. `Context` dibutuhkan untuk akses ke resource aplikasi. `Intent` dan `Uri` digunakan untuk membuka link eksternal (seperti membuka halaman web). `LayoutInflater` dipakai untuk meng-inflate layout XML ke bentuk `View`. `ViewGroup` digunakan sebagai induk tampilan item. `RecyclerView` adalah komponen utama

untuk membuat daftar yang bisa digulir. Terakhir, `ItemBookBinding` adalah class binding otomatis yang dihasilkan dari file `item_book.xml`.

Pada baris [11–15], dideklarasikan kelas `BookAdapter` sebagai turunan dari `RecyclerView.Adapter`. Kelas ini bertugas menyediakan data dan tampilan untuk setiap item buku di dalam `RecyclerView`. Adapter ini memiliki tiga parameter: `context` untuk akses ke resource dan layout inflater, `books` yaitu list data buku yang akan ditampilkan, dan `onAboutClick` yaitu fungsi callback yang dijalankan ketika tombol "About" pada suatu item diklik.

Pada baris [17], dibuat inner class `BookViewHolder` yang bertugas menyimpan binding ke tampilan setiap item buku. Kelas ini merupakan turunan dari `RecyclerView.ViewHolder`, dan menggunakan `binding.root` sebagai tampilan utama untuk setiap elemen daftar.

Pada baris [19-22], diimplementasikan method `onCreateViewHolder()` yang bertugas membuat `ViewHolder` baru. Method ini meng-inflate layout `item_book.xml` menggunakan `ItemBookBinding`, lalu membungkusnya dalam objek `BookViewHolder` yang dikembalikan.

Pada baris [24-41], diimplementasikan method `onBindViewHolder()` yang bertugas menghubungkan data buku dengan tampilan. Data buku diambil berdasarkan posisi (index) dalam list. Kemudian, nilai dari setiap properti buku ditampilkan pada komponen UI: judul, tahun, deskripsi, dan gambar buku. Untuk tombol "Buy", ketika diklik, akan membuat Intent dengan `ACTION_VIEW` dan membuka URL website buku di browser. Untuk tombol "About", ketika diklik, akan memanggil fungsi `onAboutClick()` dengan parameter objek `Book` yang sesuai.

Pada baris [43], diimplementasikan method `getItemCount()` yang mengembalikan jumlah total item dalam list buku. Method ini digunakan oleh `RecyclerView` untuk mengetahui berapa banyak data yang harus ditampilkan.

activity_main.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–30], dideklarasikan root layout yaitu `ConstraintLayout`, yang berasal dari library `androidx.constraintlayout.widget.ConstraintLayout`. Layout ini memungkinkan pengaturan posisi elemen berdasarkan batas (constraint) antar komponen atau terhadap parent. Di dalamnya juga terdapat deklarasi `xmlns` (XML namespace) untuk `android`, `app`, dan `tools`, yang masing-masing digunakan untuk atribut standar Android, atribut khusus dari `AndroidX`, dan preview di editor. `tools:context=".MainActivity"` digunakan hanya di `Android Studio` untuk menunjukkan bahwa layout ini digunakan oleh `MainActivity`.

Pada baris [10–29], didefinisikan `FrameLayout` dengan ID `@+id/frame_layout` sebagai wadah utama yang mengisi seluruh layar (`match_parent` untuk lebar dan tinggi). Komponen ini diberi constraint agar menempel ke semua sisi parent `ConstraintLayout`, yaitu `Top`, `Bottom`, `Start`, dan `End`. Ini berarti layout akan memenuhi seluruh layar aplikasi. Bias horizontal dan vertikal juga diset (meskipun tidak wajib) untuk mengatur keseimbangan layout di tengah.

Pada baris [21–27], di dalam `FrameLayout`, terdapat komponen `FragmentContainerView` yang merupakan tempat untuk menampilkan fragment. Komponen ini diberi ID `@+id/nav_host_fragment`, dan diberi atribut `android:name="androidx.navigation.fragment.NavHostFragment"` yang menunjukkan bahwa fragment ini digunakan untuk sistem navigasi. Atribut `app:navGraph="@navigation/nav_graph"` menghubungkan fragment ini dengan file navigasi `nav_graph.xml`, yang berisi pengaturan alur navigasi antar fragment. Atribut `app:defaultNavHost="true"` memastikan bahwa `NavHostFragment` ini akan menangani navigasi kembali (Back) secara otomatis dari sistem.

fragment_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–19], dideklarasikan root layout menggunakan `ConstraintLayout` dari `AndroidX` (`androidx.constraintlayout.widget.ConstraintLayout`). Layout ini memungkinkan pengaturan posisi elemen UI secara fleksibel dengan menggunakan `constraint` (batas) terhadap parent atau elemen lainnya. Tiga atribut `xmlns` didefinisikan: `android` untuk atribut dasar Android, `app` untuk atribut khusus dari library `AndroidX`, dan `tools` untuk keperluan preview di `Android Studio`. Atribut `tools:context=".BookFragment"` menunjukkan bahwa layout ini digunakan oleh `BookFragment`, sehingga `Android Studio` bisa menampilkan preview sesuai konteks fragment tersebut.

Pada baris [10–17], didefinisikan sebuah `RecyclerView` dengan ID `@+id/recyclerView`. Komponen ini digunakan untuk menampilkan daftar item buku secara efisien dalam bentuk scrollable list. Lebar (`layout_width`) dan tinggi (`layout_height`) diset ke `0dp` agar ukuran akan ditentukan oleh `constraint`. Komponen ini diposisikan menggunakan `constraint`: bagian atas dan bawah ditautkan ke `Top` dan `Bottom` parent, sedangkan bagian kiri (`Start`) dan kanan (`End`) juga ditautkan ke parent, sehingga `RecyclerView` akan memenuhi seluruh ruang dalam `ConstraintLayout`.

fragment_detail.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [1–92], didefinisikan `LinearLayout` sebagai root layout dengan orientasi vertikal. Ini berarti elemen-elemen di dalamnya akan disusun dari atas ke bawah. Layout ini mengambil seluruh lebar dan tinggi layar (`match_parent`), memberi padding sebesar `16dp`, dan diberi latar belakang warna gelap `#1C1C1C`. Selain itu, terdapat deklarasi `xmlns` standar untuk namespace Android, atribut `app` khusus `AndroidX`, dan `tools` untuk keperluan preview.

Pada baris [10–01], terdapat `ConstraintLayout` di dalam `LinearLayout`. Layout ini digunakan agar setiap elemen UI bisa diposisikan secara fleksibel berdasarkan constraint ke elemen lain. `ConstraintLayout` ini juga mengambil seluruh ruang yang tersedia di dalam parent-nya (`LinearLayout`).

Pada baris [14–29], terdapat `CardView` dengan ID `card_image`, yang digunakan sebagai wadah untuk menampilkan gambar buku dengan sudut melengkung (`cardCornerRadius="8dp"`). Tinggi layout diset `0dp` karena akan ditentukan berdasarkan constraint. Di dalamnya terdapat `ImageView` berukuran `350dp x 520dp` yang menampilkan gambar buku (`@drawable/image1`) dan menggunakan `scaleType="centerCrop"` agar gambar mengisi ruang dengan proporsi yang tepat.

Pada baris [31–44], didefinisikan `TextView` dengan ID `bookTitle` yang menampilkan judul buku. `TextView` ini menggunakan `textSize="24sp"` dan `textStyle="bold"`, dengan warna teks putih. Letaknya dikunci menggunakan constraint ke bawah dari `card_image`, ke kiri parent, dan ke kanan `bookYear`, sehingga posisinya akan diatur antara gambar dan tahun buku.

Pada baris [46–59], terdapat `TextView` lain dengan ID `bookYear`, yang menampilkan tahun terbit buku. Ukuran teks sama seperti judul buku (`24sp`) dan berwarna putih. Posisi `bookYear` dikunci di samping kanan dari `bookTitle`, dan berada di bawah `card_image`, serta di atas `bookAbout`, menyesuaikan tata letak dengan fleksibel.

Pada baris [61–74], `TextView` dengan ID `textTentang` digunakan sebagai label kecil “Tentang” untuk bagian deskripsi buku. Ukurannya kecil (`14sp`) dan diletakkan tepat di bawah `bookTitle`. Atribut `layout_marginEnd="302dp"` membuat teks ini lebih ke kiri, menjaga proporsi dari sisi kanan.

Pada baris [76–89], `TextView` dengan ID `bookAbout` menampilkan deskripsi atau sinopsis buku. Ukuran teks kecil (`14sp`) dengan lebar `346dp`. Komponen ini terletak di bagian paling bawah layout dan dikunci posisinya di bawah `textTentang`, serta dijaga menyentuh batas bawah layar (`constraintBottom_toBottomOf="parent"`), sehingga bisa menyesuaikan tinggi layar.

Item_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–125], layout utama menggunakan ConstraintLayout yang fleksibel dalam menempatkan elemen berdasarkan constraint. Lebarnya diset `match_parent`, artinya akan mengisi lebar parent-nya, dan tingginya `wrap_content`, jadi hanya sebesar konten di dalamnya.

Pada baris [8–24], terdapat CardView sebagai kontainer utama yang membungkus semua isi item buku. Ini memberikan efek melengkung pada sudutnya (`cardCornerRadius="16dp"`) dan latar belakang abu gelap (`@color/dark_gray`). CardView ini diberi margin 8dp agar tidak mepet ke item lain di RecyclerView. Constraint-nya mengatur agar tetap di tengah-tengah parent layout.

Pada baris [19–123], di dalam CardView luar terdapat ConstraintLayout kedua yang mengatur elemen-elemen detail buku.

Pada baris [23–33], terdapat CardView kecil (ID: `card_image`) yang berisi gambar buku (ImageView). CardView ini memiliki `cardCornerRadius="8dp"` agar gambar memiliki sudut melengkung. Di dalamnya, ImageView berukuran 123dp x 200dp menampilkan gambar buku dengan `scaleType="centerCrop"` agar gambar memenuhi ruang secara proporsional.

Pada baris [43–55], TextView dengan ID `bookTitle` menampilkan judul buku. Ukurannya besar (24sp), bold, dan berwarna putih. Constraint-nya menempatkan teks ini di kanan gambar, dan tidak sampai ke tepi kanan karena akan ada elemen lain di sana.

Pada baris [57–66], TextView dengan ID `bookYear` menampilkan tahun terbit buku, berada di kanan `bookTitle`, dan dekat tepi kanan parent.

Pada baris [68–79], TextView dengan ID textTentang menampilkan label “Tentang”. Letaknya tepat di bawah bookTitle, dan berada di sebelah kiri bookAbout.

Pada baris [81–93], TextView dengan ID bookAbout berisi sinopsis atau deskripsi buku. Letaknya di sebelah kanan textTentang, dan posisinya di bawah bookTitle. Lebar ditentukan 184dp, agar cukup panjang untuk menampilkan kalimat pendek atau potongan sinopsis.

Pada baris [95–107], terdapat Button dengan ID btnBuy untuk aksi “Beli”. Tombol ini posisinya di bawah bookAbout dan di sebelah kiri tombol detail. Ukurannya fleksibel (wrap_content) dengan teks dari string resource beli_button.

Pada baris [109–122], tombol btnAbout untuk membuka detail buku. Letaknya di sebelah kanan tombol beli, dan juga di bawah bookAbout. Tombol ini akan memicu perpindahan ke halaman detail jika diklik.

Compose:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–28], diimpor berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. ComponentActivity diimpor untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. setContent digunakan untuk menetapkan tampilan UI aplikasi, sementara enableEdgeToEdge memungkinkan tampilan aplikasi untuk menggunakan seluruh area layar, mengabaikan area tepi yang biasanya terblokir oleh status bar atau navigation bar.

Pada baris [30], MainActivity dideklarasikan sebagai turunan dari ComponentActivity. Kelas ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai aktivitas utama, MainActivity bertanggung jawab untuk

mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [31-39], dalam method `onCreate()`, kita memanggil `super.onCreate(savedInstanceState)` untuk memastikan siklus hidup activity dijalankan sesuai dengan standar. Kemudian, `enableEdgeToEdge()` dipanggil untuk memungkinkan konten tampilan memenuhi seluruh layar, tanpa ada pembatasan dari status bar. Selanjutnya, `setContent` digunakan untuk menetapkan tampilan UI aplikasi yang menggunakan tema `BookListTheme`, yang didefinisikan dalam file tema dan memastikan konsistensi gaya visual aplikasi. Di dalamnya, kita memanggil fungsi `BooksApp()` yang akan menampilkan tampilan utama aplikasi.

Pada baris [42-76], terdapat deklarasi fungsi `BooksApp()`, yang bertanggung jawab untuk mengelola struktur UI utama aplikasi dengan menggunakan Jetpack Compose. Fungsi ini diatur agar dapat bekerja dengan sistem navigasi dan memberikan tampilan daftar buku dan detail buku yang dapat dipilih oleh pengguna.

Pada baris [44], variabel `layoutDirection` dideklarasikan untuk mendapatkan informasi tentang arah tata letak UI. Variabel ini berguna untuk mengelola arah tata letak komponen, terutama ketika mendukung bahasa RTL (Right-to-Left), yang mempengaruhi cara elemen-elemen UI diposisikan.

Pada baris [45], variabel `navController` dideklarasikan dan diinisialisasi dengan `rememberNavController()`, yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. `navController` digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar buku ke tampilan detail buku.

Pada baris [46], variabel `books` diinisialisasi dengan memanggil `Datasource().loadBooks()`. Di sini, kita memanggil method `loadBooks()` dari class `Datasource` untuk mengambil data buku yang akan ditampilkan dalam aplikasi. Data buku ini berupa list objek `Book` yang mencakup informasi seperti judul, tahun terbit, deskripsi, dan gambar.

Pada baris [47–74], Surface dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI, dengan menggunakan modifier untuk mengatur ukuran dan padding. `fillMaxSize()` membuat Surface mengambil seluruh ruang yang tersedia di layar, sementara `statusBarsPadding()` memberi ruang di bagian atas layar agar konten tidak tertutup oleh status bar. Kemudian, `padding()` digunakan untuk menambahkan padding di sisi kiri dan kanan berdasarkan tata letak yang aman (safe drawing area), yang memastikan konten tidak terhalang oleh elemen UI seperti status bar atau navigation bar.

Pada baris [56–73], `NavHost` dideklarasikan untuk mengelola navigasi antar tampilan. `startDestination` ditetapkan ke `"ListScreen"`, yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar buku. Di dalam `NavHost`, ada dua route yang didefinisikan. Route `"ListScreen"`, `BookListScreen` ditampilkan, dengan daftar buku yang diambil dari `books`. Ketika sebuah item buku diklik, fungsi `onDetailClick` akan menavigasi ke tampilan detail dengan mengirimkan parameter `index` buku yang dipilih. Route `"detail/{index}"`, tampilan `BookDetailScreen` ditampilkan berdasarkan `index` yang diterima dari route. Jika `index` valid, maka detail buku yang sesuai akan ditampilkan. Jika tidak, sebuah pesan "Buku tidak ditemukan" akan ditampilkan menggunakan `Text`.

Pada baris [62–72], `composable("detail/{index}")` digunakan untuk mendefinisikan tampilan detail buku berdasarkan parameter `index`. Parameter ini diterima melalui `backStackEntry`, yang berisi informasi navigasi seperti argumen yang diteruskan. `indexString` adalah nilai yang diterima dari route sebagai string, dan kemudian dikonversi menjadi integer menggunakan `toIntOrNull()`. Jika konversi berhasil, buku yang sesuai akan ditampilkan, jika tidak, pesan error "Buku tidak ditemukan" akan ditampilkan.

Datasource.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-4], diimport resource R yang merujuk pada file resource aplikasi, seperti strings.xml dan drawable. Import ini memungkinkan kita untuk mengakses berbagai resource yang ada dalam aplikasi, seperti string untuk teks, gambar untuk drawable, dan lainnya. Selain itu, diimport juga Book dari model `com.example.booklist.model.Book`. Book adalah class model yang menyimpan informasi tentang buku, seperti judul, tahun terbit, deskripsi, URL, dan gambar sampul yang akan digunakan dalam aplikasi untuk menampilkan data buku.

Pada baris [6], dideklarasikan class `Datasource`, yang bertugas untuk menyediakan data buku. Class ini merupakan bagian dari arsitektur aplikasi yang bertanggung jawab untuk mengelola dan menyediakan data yang diperlukan. `Datasource` biasanya digunakan untuk memisahkan logika pengambilan data dari logika tampilan atau UI, sehingga kode aplikasi lebih modular dan mudah dikelola. Dalam hal ini, class `Datasource` menyediakan data buku yang akan digunakan oleh fragment atau activity dalam aplikasi.

Pada baris [7-18], terdapat method `loadBooks()` yang berfungsi untuk mengembalikan sebuah list yang berisi objek `Book`. Dalam method ini, kita menggunakan `listOf<Book>()` untuk membuat sebuah list baru yang berisi objek-objek `Book`. Setiap objek `Book` dibentuk dengan memasukkan berbagai ID resource dari file strings.xml dan drawable. Misalnya, untuk judul buku, kita menggunakan `R.string.book_title1` yang merujuk pada string dengan nama `book_title1` di strings.xml. Begitu juga dengan tahun terbit, deskripsi, URL, dan gambar sampul buku yang diambil dari file strings.xml dan folder drawable. Dengan cara ini, aplikasi kita menjadi lebih fleksibel karena kita bisa mengelola data di file resource dan tidak perlu mengubah kode sumber setiap kali ada perubahan pada data buku. List yang berisi data buku ini kemudian dapat digunakan oleh komponen lain dalam aplikasi, seperti adapter di `RecyclerView`, untuk menampilkan daftar buku ke pengguna.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimpor dua anotasi penting dari AndroidX: `@DrawableRes` dan `@StringRes`. Kedua anotasi ini digunakan untuk menunjukkan bahwa nilai parameter merupakan ID dari resource, bukan data literal.

Pada baris [6–12], didefinisikan sebuah data class bernama `Book`. Class ini digunakan untuk merepresentasikan data sebuah buku dalam bentuk struktur objek yang sederhana dan otomatis menyediakan fungsi-fungsi dasar seperti `toString()`, `equals()`, dan `hashCode()` karena merupakan data class. `titleResourceId`, `yearResourceId`, `aboutResourceId`, dan `webViewResourceId` bertipe `Int` dan diberi anotasi `@StringRes`, menandakan bahwa nilainya merujuk pada string resource. Sedangkan `imageResourceId` bertipe `Int` dan diberi anotasi `@DrawableRes`, menandakan bahwa ini adalah ID dari resource gambar.

BookDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-27], diimport komponen penting dari Android dan Jetpack Compose. `Intent` dan `Uri` dari `android.content` dan `android.net` digunakan untuk membuka URL eksternal. Komponen tata letak seperti `Row`, `Column`, `Spacer`, `Image`, dan `LazyColumn` diambil dari `androidx.compose.foundation.layout` dan `foundation.lazy`. Untuk tampilan material modern, digunakan `Card`, `Button`, `Text`, dan `MaterialTheme` dari `androidx.compose.material3`. Elemen tambahan seperti `painterResource`, `stringResource`, `LocalContext`, serta atribut tampilan (`clip`, `ContentScale`, dan `Alignment`) mendukung pengambilan resource dan pengaturan gambar. `Preview`, `Modifier`, dan unit ukuran seperti `dp` dan `sp` juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data `Book` diimpor dari package model dan class `BooksApp` disiapkan dari package utama aplikasi.

Pada baris [30-104], dideklarasikan fungsi `BookDetailScreen()` sebagai composable yang menerima satu parameter `book` bertipe `Book`. Fungsi ini bertugas menampilkan halaman detail dari sebuah buku tertentu, termasuk gambar, judul, tahun, dan deskripsi buku.

Pada baris [31-32], memanggil `LocalConfiguration.current.orientation` untuk mengetahui orientasi perangkat. Nilai ini kemudian disimpan dalam variabel `orientation`. Selanjutnya dibuat variabel `isLandscape`, yang bernilai `true` jika orientasi adalah `Configuration.ORIENTATION_LANDSCAPE`. Jika orientasi layar adalah `landscape`, maka blok kode di dalam kondisi `if (isLandscape)` akan dijalankan, dimulai dari baris [34].

Pada baris [35–43], digunakan layout `Row`, yang menata elemen secara horizontal. Di dalamnya terdapat `Image` yang menampilkan gambar buku berdasarkan resource ID yang disimpan dalam objek `book`. Fungsi `painterResource(book.imageResourceId)` digunakan untuk memuat gambar `drawable`. Modifier seperti `size(width = 220.dp, height = 350.dp)` mengatur ukuran gambar, dan `clip(RoundedCornerShape(16.dp))` memberikan sudut yang membulat. `contentScale = ContentScale.Crop` memastikan gambar memenuhi ukuran yang ditentukan tanpa distorsi.

Pada baris [45], `Spacer(modifier = Modifier.width(16.dp))` digunakan untuk memberi jarak horizontal antara gambar dan kolom teks di sebelahnya.

Pada baris [47–69], digunakan `Column` untuk menampilkan teks dalam bentuk vertikal. `Modifier.weight(1f)` pada kolom ini mengatur agar kolom mengisi ruang kosong yang tersedia. Properti `horizontalAlignment = Alignment.Start` mengatur agar semua elemen dalam kolom diratakan ke kiri.

Pada baris [51–55], teks judul buku ditampilkan menggunakan `Text`. Nilai teks diambil menggunakan `LocalContext.current.getString(book.titleResourceId)` yang menerjemahkan ID string dari resource menjadi teks nyata. Gaya teks diatur menggunakan `MaterialTheme.typography.labelLarge`, dan ukuran huruf ditetapkan sebesar 26.sp.

Pada baris [56–60], tahun terbit buku ditampilkan dengan pendekatan yang serupa, menggunakan Text dan mengambil nilai dari book.yearResourceId, lalu ditampilkan dengan gaya bodyMedium dan ukuran 20.sp.

Pada baris [61], Spacer menambahkan jarak vertikal sebesar 8.dp sebelum bagian deskripsi.

Pada baris [62], teks “Tentang:” ditampilkan sebagai label sebelum deskripsi buku, menggunakan gaya bodySmall dari tema Material Design.

Pada baris [63–66], deskripsi buku ditampilkan dalam Text, dengan teks yang diambil dari book.aboutResourceId. Ukuran teks ditetapkan 14.sp, dan gaya teks tetap menggunakan bodySmall. Jika orientasi perangkat adalah portrait (bukan landscape), maka blok else mulai dari baris [70] akan dieksekusi.

Pada baris [71–80], digunakan Column sebagai layout utama. Gambar buku ditampilkan dengan lebar penuh (fillMaxWidth()) dan tinggi 500.dp. Modifier clip(RoundedCornerShape(16.dp)) tetap digunakan untuk menghasilkan tampilan gambar yang membulat, serta ContentScale.Crop memastikan gambar tetap proporsional.

Pada baris [81], Spacer kembali ditambahkan untuk memberi jarak antara gambar dan teks.

Pada baris [82–94], Row digunakan untuk menampilkan judul buku dan tahun terbit secara horizontal. Text pertama menampilkan judul, diberi modifier weight(1f) agar mengisi ruang tersisa, dengan ukuran huruf 26.sp. Di sisi kanan, tahun terbit ditampilkan menggunakan gaya teks bodyMedium dengan ukuran 20.sp.

Pada baris [95], jarak vertikal ditambahkan dengan Spacer sebelum teks deskripsi.

Pada baris [96], label teks “Tentang:” ditampilkan menggunakan gaya teks kecil (bodySmall).

Pada baris [97–101], ditampilkan deskripsi lengkap buku, diambil dari resource string melalui `LocalContext.current.getString(book.aboutResourceId)`, dengan ukuran font 14.sp dan gaya teks `bodySmall`.

BookListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-35], diimport komponen penting dari Android dan Jetpack Compose. `Intent` dan `Uri` dari `android.content` dan `android.net` digunakan untuk membuka URL eksternal. Komponen tata letak seperti `Row`, `Column`, `Spacer`, `Image`, dan `LazyColumn` diambil dari `androidx.compose.foundation.layout` dan `foundation.lazy`. Untuk tampilan material modern, digunakan `Card`, `Button`, `Text`, dan `MaterialTheme` dari `androidx.compose.material3`. Elemen tambahan seperti `painterResource`, `stringResource`, `LocalContext`, serta atribut tampilan (`clip`, `ContentScale`, dan `Alignment`) mendukung pengambilan resource dan pengaturan gambar. `Preview`, `Modifier`, dan unit ukuran seperti `dp` dan `sp` juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data `Book` diimpor dari package model dan class `BooksApp` disiapkan dari package utama aplikasi.

Pada baris [37-47], didefinisikan fungsi composable `BookListScreen()` yang menerima tiga parameter: `bookList` berupa daftar buku (`List<Book>`), `onDetailClick` yaitu lambda untuk menangani klik tombol detail, dan modifier opsional untuk menyesuaikan tampilan dari luar.

Pada baris [39], digunakan `LazyColumn` sebagai layout vertikal yang mendukung pemrosesan daftar besar secara efisien. Modifier disisipkan jika diperlukan dari luar composable ini.

Pada baris [40–45], digunakan fungsi `items()` untuk mengiterasi daftar indeks dari `bookList`. Untuk setiap index, dipanggil composable `BookCard`, di mana objek

bookList[index] dikirim sebagai argumen. Parameter onDetailClick juga diteruskan, membungkus nilai index agar dapat dikenali saat tombol diklik.

Pada baris [50-131], dideklarasikan composable BookCard(), yang bertugas menampilkan kartu berisi data singkat dari sebuah buku. Fungsi ini menerima book sebagai parameter utama, onDetailClick untuk navigasi ke detail, dan modifier opsional.

Pada baris [51–130], digunakan Card dari Material 3 sebagai kontainer utama yang memiliki padding sebesar 8.dp dan bentuk sudut membulat (RoundedCornerShape 16.dp). Modifier fillMaxWidth() memastikan kartu mengisi lebar layar.

Pada baris [57], variabel context diset sebagai LocalContext.current agar dapat digunakan untuk mengambil string resource atau memulai intent dari dalam fungsi composable.

Pada baris [58–127], layout Row digunakan untuk menyusun elemen secara horizontal. Padding ditetapkan 16.dp, dan fillMaxWidth() membuat baris mengisi lebar penuh.

Pada baris [63–71], ditampilkan gambar buku menggunakan Image. Resource gambar diakses dari ID dalam book.imageResourceId melalui painterResource. contentDescription diberikan melalui stringResource(book.aboutResourceId) agar sesuai dengan prinsip aksesibilitas. Ukuran gambar diatur dengan size(width = 120.dp, height = 200.dp) dan bentuknya dibulatkan menggunakan clip(RoundedCornerShape(12.dp)). Gambar disejajarkan secara vertikal di tengah baris (align(Alignment.CenterVertically)) dan ContentScale.Crop menjaga rasio aspek gambar.

Pada baris [72], Spacer memberikan jarak horizontal 12.dp antara gambar dan kolom teks di sebelah kanan.

Pada baris [73-127], digunakan Column untuk menampilkan isi teks dan tombol dalam tata letak vertikal. Modifier weight(1f) membuat kolom ini memenuhi ruang

horizontal yang tersisa, dan `align(Alignment.CenterVertically)` memastikan kolom berada di tengah baris secara vertikal.

Pada baris [78–95], `Row` di dalam kolom digunakan untuk menampilkan judul buku di sebelah kiri dan tahun terbit di kanan. `Arrangement.SpaceBetween` memastikan keduanya memiliki jarak yang merata di antara mereka. Judul buku ditampilkan dalam `Text` menggunakan resource string, dengan gaya `labelLarge`, ukuran huruf 26.sp, dan `overflow = TextOverflow.Ellipsis` agar teks panjang tidak meluber keluar. Tahun terbit ditampilkan di kanan dengan ukuran lebih kecil, 14.sp, dan gaya `bodyMedium`.

Pada baris [96], `Spacer` menambahkan jarak vertikal sebesar 20.dp sebelum deskripsi buku.

Pada baris [97–108], terdapat `Row` untuk memuat deskripsi buku ditampilkan dalam bentuk dua `Text`. Pertama teks label “Tentang:”, lalu diikuti oleh isi deskripsi dari resource string `book.aboutResourceId`. Keduanya menggunakan gaya `bodySmall` dan ukuran huruf 14.sp.

Pada baris [109], `Spacer` memberikan jarak sebesar 16.dp sebelum tombol aksi.

Pada baris [110–125], digunakan `Row` dengan `horizontalArrangement = Arrangement.End` agar tombol-tombol sejajar di sebelah kanan layar. Dua buah tombol ditampilkan: “Beli Buku” dan “Detail”.

Pada baris [112–118], tombol “Beli Buku” diberi aksi ketika diklik, yaitu mengambil URL dari `book.webUrlResourceId`, lalu membuat `Intent` dengan `ACTION_VIEW` dan membuka URL tersebut dengan `context.startActivity()` untuk membuka halaman pembelian buku melalui browser.

Pada baris [119], `Spacer` memberi jarak horizontal antara dua tombol sebesar 8.dp.

Pada baris [120–124], tombol “Detail” memanggil `onDetailClick()`, yaitu fungsi yang diteruskan dari luar composable dan biasanya mengarahkan pengguna ke halaman detail buku.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

Alasan kenapa RecyclerView masih digunakan walaupun LazyColumn lebih sederhana, karena RecyclerView support banyak fitur dan jauh lebih fleksibel karena imperative. Selain itu, masih kompatibel dengan project lama (masih bisa dipakai untuk pengembang yang tidak mau bermigrasi ke Jetpack Compose). Sehingga, RecyclerView lebih cocok untuk aplikasi yang kompleks dan dinamis karena termasuk paradigma imperative (mengatur logikanya sendiri), sedangkan LazyColumn cocok untuk list simple karena termasuk pendekatan declarative (cukup sebutkan apa yang diinginkan tanpa membuat logikanya sendiri sehingga sulit diotak atik)

MODUL 4 : VIEWMODEL AND DEBUGGING

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

XML:

MainActivity.kt

Tabel 23. Source Code MainActivity.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge

5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import
	com.example.booklist.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	val binding =
	ActivityMainBinding.inflate(layoutInflater)
14	setContentView(binding.root)
15	}
16	}

Book.kt

Tabel 24. Source Code Book.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,
8	@StringRes val yearResourceId: Int,
9	@StringRes val aboutResourceId: Int,
10	@StringRes val webUrlResourceId: Int,
11	@DrawableRes val imageResourceId: Int
12	)

BookFragment.kt

Tabel 25. Source Code BookFragment.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.lifecycle.LifecycleScope
10	import androidx.navigation.fragment.findNavController
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import
	com.example.booklist.databinding.FragmentBookBinding

```

13 import kotlinx.coroutines.flow.collectLatest
14 import kotlinx.coroutines.launch
15 import timber.log.Timber
16
17
18 class BookFragment : Fragment() {
19     private val viewModel: BookViewModel by viewModels {
20         BookViewModelFactory("MyCategory")
21     }
22
23     private var _binding: FragmentBookBinding? = null
24     private val binding get() = _binding!!
25
26     override fun onCreateView(inflater: LayoutInflater,
27 container: ViewGroup?, savedInstanceState: Bundle?):
28 View {
29         _binding = FragmentBookBinding.inflate(inflater,
30 container, false)
31         return binding.root
32     }
33
34     override fun onViewCreated(view: View,
35 savedInstanceState: Bundle?) {
36         val recyclerView = binding.recyclerView
37         recyclerView.layoutManager =
38 LinearLayoutManager(requireContext())
39
40         lifecycleScope.launch {
41             viewModel.books.collectLatest { bookList ->
42                 recyclerView.adapter =
43 BookAdapter(requireContext(), bookList) { selectedBook -
44 >
45                     Timber.d("Tombol 'Tentang' ditekan
46 untuk buku dengan judul ID:
47 ${selectedBook.titleResourceId}")
48                     val bundle = Bundle().apply {
49                         putInt("titleRes",
50 selectedBook.titleResourceId)
51                         putInt("yearRes",
52 selectedBook.yearResourceId)
53                         putInt("aboutRes",
54 selectedBook.aboutResourceId)
55                         putInt("imageResId",
56 selectedBook.imageResourceId)
57                     }
58
59                     findNavController().navigate(R.id.action_bookFragment_to
60 _detailFragment, bundle)
61                 }
62             }
63         }

```

48	}
49	}
50	}
51	
52	override fun onDestroyView() {
53	super.onDestroyView()
54	_binding = null
55	}

DetailFragment.kt

Tabel 26. Source Code DetailFragment.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import
	com.example.booklist.databinding.FragmentDetailBinding
9	
10	class DetailFragment : Fragment() {
11	
12	private var _binding: FragmentDetailBinding? = null
13	private val binding get() = _binding!!
14	
15	override fun onCreateView(
16	inflater: LayoutInflater,
17	container: ViewGroup?,
18	savedInstanceState: Bundle?
19	): View? {
20	_binding =
	FragmentDetailBinding.inflate(inflater, container, false)
21	return binding.root
22	}
23	
24	override fun onViewCreated(view: View,
	savedInstanceState: Bundle?) {
25	val args = arguments
26	val titleRes = args?.getInt("titleRes") ?: 0
27	val yearRes = args?.getInt("yearRes") ?: 0
28	val aboutRes = args?.getInt("aboutRes") ?: 0
29	val imageResId = args?.getInt("imageResId") ?: 0
30	
31	binding.bookTitle.setText(titleRes)
32	binding.bookYear.setText(yearRes)
33	binding.bookAbout.setText(aboutRes)

34	binding.bookImage.setImageResource(imageResId)
35	}
36	
37	override fun onDestroyView() {
38	super.onDestroyView()
39	_binding = null
40	}
41	}

BookViewModel.kt

Tabel 27. Source Code BookViewModel.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import timber.log.Timber
8	
9	class BookViewModel(category: String) : ViewModel() {
10	
11	private val _books =
12	MutableStateFlow<List<Book>>(emptyList())
13	val books: StateFlow<List<Book>> = _books
14	init {
15	loadBooks()
16	}
17	
18	
19	private fun loadBooks() {
20	_books.value = listOf(
21	Book(R.string.book_title1,
	R.string.book_year1, R.string.book_about1,
	R.string.book_webUrl1, R.drawable.image1),
22	Book(R.string.book_title2,
	R.string.book_year2, R.string.book_about2,
	R.string.book_webUrl2, R.drawable.image2),
23	Book(R.string.book_title3,
	R.string.book_year3, R.string.book_about3,
	R.string.book_webUrl3, R.drawable.image3),
24	Book(R.string.book_title4,
	R.string.book_year4, R.string.book_about4,
	R.string.book_webUrl4, R.drawable.image4),
25	Book(R.string.book_title5,
	R.string.book_year5, R.string.book_about5,
	R.string.book_webUrl5, R.drawable.image5),

26	Book(R.string.book_title6,
	R.string.book_year6, R.string.book_about6,
	R.string.book_webUrl6, R.drawable.image6),
27	Book(R.string.book_title7,
	R.string.book_year7, R.string.book_about7,
	R.string.book_webUrl7, R.drawable.image7),
28	Book(R.string.book_title8,
	R.string.book_year8, R.string.book_about8,
	R.string.book_webUrl8, R.drawable.image8)
29	)
30	Timber.i("Data buku berhasil dimuat:
	\${_books.value.size}")
31	}
32	}
33	
34	class BookViewModelFactory(private val category: String)
35	: ViewModelProvider.Factory {
36	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
37	if
	(modelClass.isAssignableFrom(BookViewModel::class.java))
	{
38	@Suppress("UNCHECKED_CAST")
39	return BookViewModel(category) as T
40	}
41	throw IllegalArgumentException("Unknown
42	ViewModel class")
43	}
44	
45	}

MyItemRecyclerViewAdapter.kt

Tabel 28. Source Code MyItemRecyclerViewAdapter.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import android.content.Context
4	import android.content.Intent
5	import android.net.Uri
6	import android.view.LayoutInflater
7	import android.view.ViewGroup
8	import androidx.recyclerview.widget.RecyclerView
9	import com.example.booklist.databinding.ItemBookBinding
10	import timber.log.Timber
11	
12	class BookAdapter(
13	private val context: Context,
14	private val books: List<Book>,

```

15     private val onAboutClick: (Book) -> Unit
16 ) : RecyclerView.Adapter<BookAdapter.BookViewHolder>() {
17
18     inner class BookViewHolder(val binding:
ItemBookBinding) : RecyclerView.ViewHolder(binding.root)
19
20     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): BookViewHolder {
21         val binding =
ItemBookBinding.inflate(LayoutInflater.from(context),
parent, false)
22         return BookViewHolder(binding)
23     }
24
25     override fun onBindViewHolder(holder:
BookViewHolder, position: Int) {
26         val book = books[position]
27         with(holder.binding) {
28             bookTitle.text =
context.getString(book.titleResourceId)
29             bookYear.text =
context.getString(book.yearResourceId)
30             bookAbout.text =
context.getString(book.aboutResourceId)
31             bookImage.setImageResource(book.imageResourceId)
32
33             btnBuy.setOnClickListener {
34                 Timber.d("Tombol 'Beli' ditekan untuk
URL ID: ${book.webUrlResourceId}")
35                 val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(context.getString(book.webUrlResourceId)))
36                 context.startActivity(intent)
37             }
38
39             btnAbout.setOnClickListener {
40                 Timber.d("Tombol 'Tentang' ditekan untuk
buku: ${context.getString(book.titleResourceId)}")
41                 onAboutClick(book)
42             }
43         }
44     }
45
46     override fun getItemCount() = books.size
47 }

```

MyApp.kt

Tabel 29. Source Code MyApp.kt Soal 1 Modul 4 XML

1	package com.example.booklist
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApp : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

activity_main.xml

Tabel 30. Source Code activity_main.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/andro id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<FrameLayout
11	android:id="@+id/frame_layout"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintHorizontal_bias="1.0"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintVertical_bias="0.0">
20	
21	<androidx.fragment.app.FragmentContainerView
22	android:id="@+id/nav_host_fragment"
23	android:name="androidx.navigation.fragment.NavHostFragm ent"
24	android:layout_width="match_parent"
25	android:layout_height="match_parent"
26	app:navGraph="@navigation/nav_graph"
27	app:defaultNavHost="true" />
28	
29	</FrameLayout>
30	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_book.xml

Tabel 31. Source Code fragment_book.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro id"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	xmlns:tools="http://schemas.android.com/tools"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	tools:context=".BookFragment">
10	
11	<androidx.recyclerview.widget.RecyclerView
12	android:id="@+id/recyclerView"
13	android:layout_width="0dp"
14	android:layout_height="0dp"
15	app:layout_constraintTop_toTopOf="parent"
16	app:layout_constraintBottom_toBottomOf="parent"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintEnd_toEndOf="parent" />
19	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_detail.xml

Tabel 32. Source Code fragment_detail.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/andro id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:background="#1C1C1C"
7	android:padding="16dp">
8	
9	<LinearLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:orientation="vertical">
13	
14	<androidx.constraintlayout.widget.ConstraintLayout
15	android:layout_width="match_parent"
16	android:layout_height="wrap_content">
17	
18	<androidx.cardview.widget.CardView
19	android:id="@+id/card_image"
20	android:layout_width="wrap_content"

```

21         android:layout_height="wrap_content"
22         app:cardCornerRadius="8dp"
23         app:layout_constraintTop_toTopOf="parent"
24         app:layout_constraintStart_toStartOf="parent"
25 app:layout_constraintEnd_toEndOf="parent">
26
27         <ImageView
28             android:id="@+id/bookImage"
29             android:layout_width="350dp"
30             android:layout_height="520dp"
31             android:scaleType="centerCrop"
32             android:src="@drawable/image1" />
33     </androidx.cardview.widget.CardView>
34
35     <TextView
36         android:id="@+id/bookTitle"
37         android:layout_width="0dp"
38         android:layout_height="wrap_content"
39         android:layout_marginTop="23dp"
40         android:text="@string/book_title1"
41         android:textColor="@color/white"
42         android:textSize="24sp"
43         android:textStyle="bold"
44         app:layout_constraintTop_toBottomOf="@id/card_image"
45 app:layout_constraintStart_toStartOf="parent"
46 app:layout_constraintEnd_toStartOf="@id/bookYear"
47 app:layout_constraintHorizontal_weight="1" />
48
49     <TextView
50         android:id="@+id/bookYear"
51         android:layout_width="wrap_content"
52         android:layout_height="wrap_content"
53         android:layout_marginTop="24dp"
54         android:text="@string/book_year1"
55         android:textColor="@color/white"
56         android:textSize="24sp"
57 app:layout_constraintTop_toBottomOf="@id/card_image"
58         app:layout_constraintEnd_toEndOf="parent"
59 app:layout_constraintStart_toEndOf="@id/bookTitle" />
60
61     <TextView
62         android:id="@+id/textTentang"
63         android:layout_width="wrap_content"
64         android:layout_height="wrap_content"
65         android:layout_marginTop="8dp"
66         android:text="@string/about"
67         android:textColor="@color/white"
68         android:textSize="14sp"
69 app:layout_constraintTop_toBottomOf="@id/bookTitle"

```

70	<code>app:layout_constraintStart_toStartOf="parent" /></code>
71	
72	<code><TextView</code>
73	<code>android:id="@+id/bookAbout"</code>
74	<code>android:layout_width="0dp"</code>
75	<code>android:layout_height="wrap_content"</code>
76	<code>android:layout_marginTop="8dp"</code>
77	<code>android:layout_marginBottom="8dp"</code>
78	<code>android:text="@string/book_about1"</code>
79	<code>android:textColor="@color/white"</code>
80	<code>android:textSize="14sp"</code>
81	<code>app:layout_constraintTop_toBottomOf="@id/textTentang"</code>
82	<code>app:layout_constraintStart_toStartOf="parent"</code>
83	<code>app:layout_constraintEnd_toEndOf="parent"</code>
	<code>/></code>
84	
85	<code></androidx.constraintlayout.widget.ConstraintLayout></code>
86	
87	<code></LinearLayout></code>
88	<code></ScrollView></code>

item_book.xml

Tabel 33. Source Code item_book.xml Soal 1 Modul 4 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/andro id"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content">
7	
8	<androidx.cardview.widget.CardView
9	android:layout_width="wrap_content"
10	android:layout_height="wrap_content"
11	android:layout_margin="8dp"
12	app:cardBackgroundColor="@color/dark_gray"
13	app:cardCornerRadius="16dp"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintTop_toTopOf="parent">
18	
19	<androidx.constraintlayout.widget.ConstraintLayout
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content">
22	
23	<androidx.cardview.widget.CardView
24	android:id="@+id/card_image"
25	android:layout_width="wrap_content"
26	android:layout_height="wrap_content"
27	android:layout_marginStart="16dp"
28	android:layout_marginTop="16dp"
29	android:layout_marginBottom="16dp"
30	app:cardCornerRadius="8dp"
31	app:layout_constraintBottom_toBottomOf="parent"
32	app:layout_constraintStart_toStartOf="parent"
33	app:layout_constraintTop_toTopOf="parent">
34	
35	<ImageView
36	android:id="@+id/bookImage"
37	android:layout_width="123dp"
38	android:layout_height="200dp"
39	android:scaleType="centerCrop"
40	app:srcCompat="@drawable/image1" />
41	</androidx.cardview.widget.CardView>
42	
43	<TextView
44	android:id="@+id/bookTitle"
45	android:layout_width="193dp"

46	android:layout_height="wrap_content"
47	android:layout_marginTop="8dp"
48	android:layout_marginEnd="5dp"
49	android:text="@string/book_title1"
50	android:textColor="@color/white"
51	android:textSize="24sp"
52	android:textStyle="bold"
53	app:layout_constraintEnd_toStartOf="@+id/bookYear"
54	app:layout_constraintStart_toEndOf="@+id/card_image"
55	app:layout_constraintTop_toTopOf="parent" />
56	
57	<TextView
58	android:id="@+id/bookYear"
59	android:layout_width="wrap_content"
60	android:layout_height="wrap_content"
61	android:layout_marginTop="16dp"
62	android:layout_marginEnd="16dp"
63	android:text="@string/book_year1"
64	android:textColor="@color/white"
65	app:layout_constraintEnd_toEndOf="parent"
66	app:layout_constraintTop_toTopOf="parent" />
67	
68	<TextView
69	android:id="@+id/textTentang"
70	android:layout_width="wrap_content"
71	android:layout_height="0dp"
72	android:layout_marginStart="8dp"
73	android:layout_marginTop="24dp"
74	android:text="@string/about"
75	android:textColor="@color/white"
76	android:textSize="14sp"
77	app:layout_constraintEnd_toStartOf="@+id/bookAbout"
78	app:layout_constraintStart_toEndOf="@+id/card_image"
79	app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
80	
81	<TextView
82	android:id="@+id/bookAbout"
83	android:layout_width="184dp"
84	android:layout_height="wrap_content"
85	android:layout_marginStart="6dp"
86	android:layout_marginTop="24dp"
87	android:layout_marginEnd="9dp"
88	android:text="@string/book_about1"
89	android:textColor="@color/white"

90	android:textSize="14sp"
91	app:layout_constraintEnd_toEndOf="parent"
92	app:layout_constraintStart_toEndOf="@+id/textTentang"
93	app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
94	
95	<Button
96	android:id="@+id/btnBuy"
97	android:layout_width="wrap_content"
98	android:layout_height="wrap_content"
99	android:layout_marginStart="55dp"
100	android:layout_marginTop="27dp"
101	android:layout_marginBottom="8dp"
102	android:text="@string/beli_button"
103	android:textSize="14sp"
104	app:layout_constraintBottom_toBottomOf="parent"
105	app:layout_constraintEnd_toStartOf="@+id/btnAbout"
106	app:layout_constraintStart_toEndOf="@+id/card_image"
107	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
108	
109	<Button
110	android:id="@+id/btnAbout"
111	android:layout_width="wrap_content"
112	android:layout_height="wrap_content"
113	android:layout_marginStart="8dp"
114	android:layout_marginTop="27dp"
115	android:layout_marginEnd="16dp"
116	android:layout_marginBottom="8dp"
117	android:text="@string/detail_button"
118	android:textSize="14sp"
119	app:layout_constraintBottom_toBottomOf="parent"
120	app:layout_constraintEnd_toEndOf="parent"
121	app:layout_constraintStart_toEndOf="@+id/btnBuy"
122	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
123	</androidx.constraintlayout.widget.ConstraintLayout>
124	</androidx.cardview.widget.CardView>
125	</androidx.constraintlayout.widget.ConstraintLayout>

Compose:

MainActivity.kt

Tabel 34. Source Code MainActivity.kt Soal 1 Modul 4 Compose

```
1 package com.example.booklist
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.WindowInsets
8 import
9 androidx.compose.foundation.layout.asPaddingValues
10 import
11 androidx.compose.foundation.layout.calculateEndPadding
12 import
13 androidx.compose.foundation.layout.calculateStartPadding
14 import androidx.compose.foundation.layout.fillMaxSize
15 import androidx.compose.foundation.layout.padding
16 import androidx.compose.foundation.layout.safeDrawing
17 import
18 androidx.compose.foundation.layout.statusBarsPadding
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.runtime.Composable
22 import androidx.compose.runtime.collectAsState
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.platform.LocalLayoutDirection
25 import androidx.compose.runtime.getValue
26 import androidx.compose.ui.unit.dp
27 import androidx.lifecycle.viewmodel.compose.viewModel
28 import androidx.navigation.compose.NavHost
29 import androidx.navigation.compose.composable
30 import androidx.navigation.compose.rememberNavController
31
32 import com.example.booklist.screens.BookDetailScreen
33 import com.example.booklist.screens.BookListScreen
34 import com.example.booklist.screens.BookListViewModel
35 import com.example.booklist.ui.theme.BookListTheme
36
37 class MainActivity : ComponentActivity() {
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         enableEdgeToEdge()
41         setContent {
42             BookListTheme {
43                 BooksApp()
44             }
45         }
46     }
47 }
```

```

43
44 @Composable
45 fun BooksApp() {
46     val layoutDirection = LocalLayoutDirection.current
47     val navController = rememberNavController()
48     val viewModel: BookListViewModel = viewModel()
49     val bookList by viewModel.booksList.collectAsState()
50     val selectedBook by
51     viewModel.selectedBook.collectAsState()
52
53     Surface(
54         modifier = Modifier
55             .fillMaxSize()
56             .statusBarsPadding()
57             .padding(
58                 start =
59                 WindowInsets.safeDrawing.asPaddingValues().calculateStartPadding(layoutDirection),
60                 end =
61                 WindowInsets.safeDrawing.asPaddingValues().calculateEndPadding(layoutDirection),
62             )
63     ) {
64         NavHost(navController = navController,
65             startDestination = "list") {
66             composable("list") {
67                 BookListScreen(bookList = bookList,
68                     onDetailClick = { index ->
69                     viewModel.selectBook(bookList[index])
70                     navController.navigate("detail")
71                 })
72             }
73             composable("detail") { backStackEntry ->
74                 if (selectedBook != null) {
75                     BookDetailScreen(book =
76                     selectedBook!!)
77                 } else {
78                     Text("Buku tidak ditemukan",
79                         modifier = Modifier.padding(16.dp))
80                 }
81             }
82         }
83     }
84 }

```

Datasource.kt

Tabel 35. Source Code Datasource.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.data
2	
3	import com.example.booklist.R
4	import com.example.booklist.model.Book
5	
6	class Datasource {
7	fun loadBooks(): List<Book> {
8	return listOf<Book> (
9	Book(R.string.book_title1,
	R.string.book_year1, R.string.book_about1,
	R.string.book_webUrl1, R.drawable.image1) ,
10	Book(R.string.book_title2,
	R.string.book_year2, R.string.book_about2,
	R.string.book_webUrl2, R.drawable.image2),
11	Book(R.string.book_title3,
	R.string.book_year3, R.string.book_about3,
	R.string.book_webUrl3, R.drawable.image3),
12	Book(R.string.book_title4,
	R.string.book_year4, R.string.book_about4,
	R.string.book_webUrl4, R.drawable.image4),
13	Book(R.string.book_title5,
	R.string.book_year5, R.string.book_about5,
	R.string.book_webUrl5, R.drawable.image5),
14	Book(R.string.book_title6,
	R.string.book_year6, R.string.book_about6,
	R.string.book_webUrl6, R.drawable.image6),
15	Book(R.string.book_title7,
	R.string.book_year7, R.string.book_about7,
	R.string.book_webUrl7, R.drawable.image7),
16	Book(R.string.book_title8,
	R.string.book_year8, R.string.book_about8,
	R.string.book_webUrl8, R.drawable.image8)
17	)
18	}
19	}

Book.kt

Tabel 36. Source Code Book.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,

8	@StringRes val yearResourceId: Int,
9	@StringRes val aboutResourceId: Int,
10	@StringRes val webUrlResourceId: Int,
11	@DrawableRes val imageResourceId: Int
12	)

BookDetailScreen.kt

Tabel 37. Source Code BookDetailScreen.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.screens
2	
3	import android.content.res.Configuration
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Arrangement
6	import androidx.compose.foundation.layout.Column
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer
9	import androidx.compose.foundation.layout.fillMaxSize
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.height
12	import androidx.compose.foundation.layout.padding
13	import androidx.compose.foundation.layout.size
14	import androidx.compose.foundation.layout.width
15	import androidx.compose.foundation.rememberScrollState
16	import
	androidx.compose.foundation.shape.RoundedCornerShape
17	import androidx.compose.foundation.verticalScroll
18	import androidx.compose.material3.MaterialTheme
19	import androidx.compose.material3.Text
20	import androidx.compose.runtime.Composable
21	import androidx.compose.ui.Alignment
22	import androidx.compose.ui.Modifier
23	import androidx.compose.ui.draw.clip
24	import androidx.compose.ui.layout.ContentScale
25	import androidx.compose.ui.platform.LocalConfiguration
26	import androidx.compose.ui.platform.LocalContext
27	import androidx.compose.ui.res.painterResource
28	import androidx.compose.ui.text.style.TextOverflow
29	import androidx.compose.ui.unit.dp
30	import androidx.compose.ui.unit.sp
31	import com.example.booklist.model.Book
32	import timber.log.Timber
33	
34	@Composable
35	fun BookDetailScreen(book: Book) {
36	Timber.i("Detail buku ditampilkan:
	\${LocalContext.current.getString(book.titleResourceId)} ")

37	Column(
38	modifier = Modifier
39	.fillMaxSize()
40	.padding(16.dp)
41	.verticalScroll(rememberScrollState())
42	) {
43	Image(
44	painter =
45	painterResource(book.imageResourceId),
46	contentDescription = null,
47	modifier = Modifier
48	.size(width = 380.dp, height =
49	550.dp)
50	.clip(RoundedCornerShape(16.dp))
51	.align(Alignment.CenterHorizontally),
52	contentScale = ContentScale.Crop
53	) {
54	Spacer(modifier = Modifier.height(16.dp))
55	Column(
56	modifier = Modifier.fillMaxWidth(),
57	horizontalAlignment = Alignment.Start
58	) {
59	Row(horizontalArrangement =
60	Arrangement.SpaceBetween, modifier =
61	Modifier.fillMaxWidth()) {
62	Text(
63	text =
64	LocalContext.current.getString(book.titleResourceId),
65	style =
66	MaterialTheme.typography.labelLarge,
67	fontSize = 26.sp
68	) {
69	Text(
70	text =
71	LocalContext.current.getString(book.yearResourceId),
72	style =
73	MaterialTheme.typography.bodyMedium,
	fontSize = 20.sp
	) {
	Spacer(modifier =
	Modifier.height(8.dp))
	Text("Tentang:", style =
	MaterialTheme.typography.bodySmall)
	Text(

74	text =
75	LocalContext.current.getString(book.aboutResourceId),
76	style =
77	MaterialTheme.typography.bodySmall,
78	fontSize = 14.sp
79	)
80	}

BookListScreen.kt

Tabel 38. Source Code BookListScreen.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.screens
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.util.Log
6	import androidx.compose.foundation.Image
7	import androidx.compose.foundation.layout.Arrangement
8	import androidx.compose.foundation.layout.Column
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.layout.size
15	import androidx.compose.foundation.layout.width
16	import androidx.compose.foundation.lazy.LazyColumn
17	import androidx.compose.foundation.lazy.items
18	import androidx.compose.foundation.lazy.itemsIndexed
19	import
20	androidx.compose.foundation.shape.RoundedCornerShape
21	import androidx.compose.material3.Button
22	import androidx.compose.material3.Card
23	import androidx.compose.material3.MaterialTheme
24	import androidx.compose.material3.Text
25	import androidx.compose.runtime.Composable
26	import androidx.compose.ui.Alignment
27	import androidx.compose.ui.Modifier
28	import androidx.compose.ui.draw.clip
29	import androidx.compose.ui.layout.ContentScale
30	import androidx.compose.ui.platform.LocalContext
31	import androidx.compose.ui.res.painterResource
32	import androidx.compose.ui.res.stringResource
33	import androidx.compose.ui.text.style.TextOverflow
34	import androidx.compose.ui.tooling.preview.Preview

```

35 import androidx.compose.ui.unit.sp
36 import com.example.booklist.BooksApp
37 import com.example.booklist.model.Book
38 import timber.log.Timber
39
40 @Composable
41 fun BookListScreen(bookList: List<Book>, onDetailClick:
  (Int) -> Unit, modifier: Modifier = Modifier) {
42     LazyColumn(modifier = modifier) {
43         itemsIndexed(bookList) { index, book ->
44             Timber.d("Tombol 'Detail' ditekan untuk
menampilkan: ${book.titleResourceId}")
45             BookCard(book = book, onDetailClick = {
46                 onDetailClick(index)
47             })
48         }
49     }
50 }
51
52 @Composable
53 fun BookCard(book: Book, onDetailClick: () -> Unit,
modifier: Modifier = Modifier) {
54     Card(
55         modifier = modifier
56         .fillMaxWidth()
57         .padding(8.dp),
58         shape = RoundedCornerShape(16.dp)
59     ) {
60         val context = LocalContext.current
61         Row(
62             modifier = Modifier
63                 .padding(16.dp)
64                 .fillMaxWidth()
65         ) {
66             Image(
67                 painter =
painterResource(book.imageResourceId),
68                 contentDescription =
stringResource(book.aboutResourceId),
69                 modifier = Modifier
70                     .size(width = 120.dp, height =
200.dp)
71                     .clip(RoundedCornerShape(12.dp))
72                     .align(Alignment.CenterVertically),
73                 contentScale = ContentScale.Crop
74             )
75             Spacer(modifier = Modifier.width(12.dp))
76             Column(
77                 modifier = Modifier

```



```

78         .weight(1f)
79         .align(Alignment.CenterVertically)
80     ) {
81         Row(
82             modifier = Modifier.fillMaxWidth(),
83             horizontalArrangement =
Arrangement.SpaceBetween
84         ) {
85             Text(
86                 text =
context.getString(book.titleResourceId),
87                 modifier = Modifier.weight(1f),
88                 style =
MaterialTheme.typography.labelLarge,
89                 overflow =
TextOverflow.Ellipsis,
90                 fontSize = 26.sp
91             )
92             Text(
93                 text =
context.getString(book.yearResourceId),
94                 style =
MaterialTheme.typography.bodyMedium,
95                 fontSize = 14.sp
96             )
97         }
98         Spacer(modifier =
Modifier.height(20.dp))
99         Row {
100             Text(
101                 text = "Tentang: ",
102                 style =
MaterialTheme.typography.bodySmall,
103                 fontSize = 14.sp
104             )
105             Text(
106                 text =
context.getString(book.aboutResourceId),
107                 style =
MaterialTheme.typography.bodySmall,
108                 fontSize = 14.sp
109             )
110         }
111         Spacer(modifier =
Modifier.height(16.dp))
112         Row(modifier = Modifier.fillMaxWidth(),
113             horizontalArrangement =
Arrangement.End) {

```

115	Button(onClick = {
116	val webUrl =
117	context.getString(book.webUrlResourceId)
117	Timber.tag("BookCard").d("URL
118	yang akan dibuka: \$webUrl")
118	val intent =
119	Intent(Intent.ACTION_VIEW, Uri.parse(webUrl))
119	context.startActivity(intent)
120	}) {
121	Text("Beli Buku")
122	}
123	Spacer(modifier =
124	Modifier.width(8.dp))
124	Button(onClick = {
125	onDetailClick()
126	}) {
127	Text("Detail")
128	}
129	}
130	}
131	}
132	}
133	}
134	}
135	}

BookListViewModel.kt

Tabel 39. Source Code BookListViewModel.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.screens
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.booklist.data.Datasource
6	import com.example.booklist.model.Book
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.launch
10	import timber.log.Timber
11	
12	class BookListViewModel(private val source: String) :
12	ViewModel() {
13	private val _booksList =
13	MutableStateFlow<List<Book>>(emptyList())
14	val booksList: StateFlow<List<Book>> = _booksList
15	
16	private val _selectedBook =
16	MutableStateFlow<Book?>(null)

17	val selectedBook: StateFlow<Book?> = _selectedBook
18	
19	init {
20	Timber.d("Sumber data ViewModel: \$source")
21	loadBooks()
22	}
23	
24	private fun loadBooks() {
25	viewModelScope.launch {
26	val books = Datasource().loadBooks()
27	booksList.value = books
28	Timber.d("Data buku berhasil dimuat: \${books.size} item")
29	}
30	}
31	
32	fun selectBook(book: Book) {
33	_selectedBook.value = book
34	Timber.d("Buku yang dipilih: \${book.titleResourceId}")
35	}
36	}

BookListViewModelFactory.kt

Tabel 40. Source Code BookListViewModelFactory.kt Soal 1 Modul 4 Compose

1	package com.example.booklist.screens
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class BookListViewModelFactory(private val source: String) : ViewModelProvider.Factory {
7	override fun <T : ViewModel> create(modelClass: Class<T>): T {
8	if (modelClass.isAssignableFrom((BookListViewModel::class.java))) {
9	@Suppress("UNCHECKED_CAST")
10	return BookListViewModel(source) as T
11	}
12	throw IllegalArgumentException("Unknown ViewModel class")
13	}
14	}

MyApplication.kt

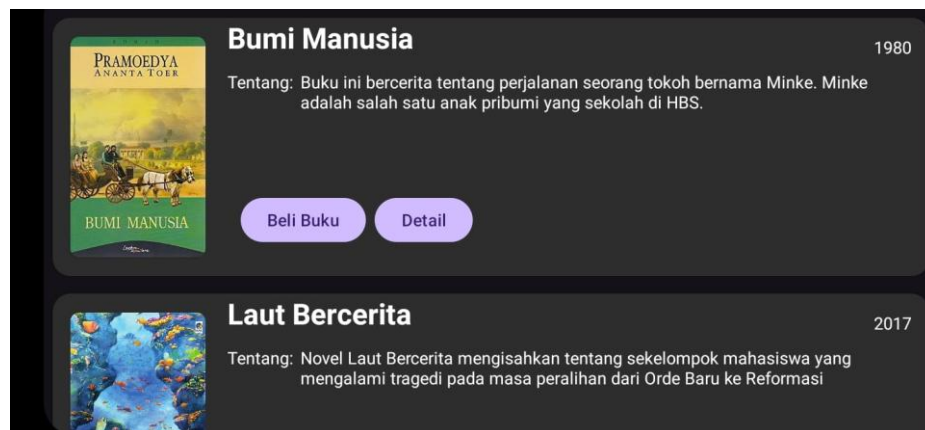
Tabel 41. Source CodeMyApplication.kt Soal 1 Modul 4 Compose

1	package com.example.booklist
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	
7	class MyApplication : Application() {
8	override fun onCreate() {
9	super.onCreate()
10	
11	if(BuildConfig.DEBUG) {
12	Timber.plant(Timber.DebugTree())
13	}
14	}
15	}

B. Output Program



Gambar 33. Tampilan Awal UI Soal 1 Modul 4 XML



Gambar 34. Tampilan Awal UI Landscape Soal 1 Modul 4 XML



Gambar 35. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 4 XML



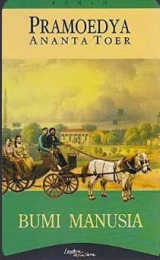
Gambar 36. Tampilan Ketika Discroll Soal 1 Modul 4 XML



Gambar 37. Tampilan Tombol Detail Ditekan Soal 1 Modul 4 XML



Gambar 38. Tampilan Tombol Detail Ditekan Landscape Soal 1 Modul 4 XML



Bumi Manusia

1980

Tentang: Buku ini bercerita tentang perjalanan seorang tokoh bernama Minke. Minke adalah salah satu anak pribumi yang sekolah di HBS.

[Beli Buku](#)
[Detail](#)

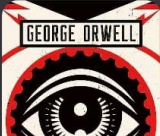


Laut Bercerita

2017

Tentang: Novel Laut Bercerita mengisahkan tentang sekelompok mahasiswa yang mengalami tragedi pada masa peralihan dari Orde Baru ke Reformasi

[Beli Buku](#)
[Detail](#)



1984

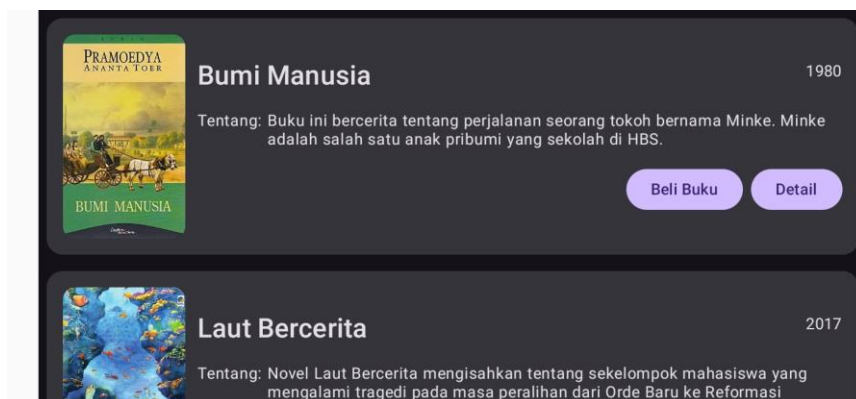
1949

Tentang: Dalam dunia yang penuh pengawasan dan propaganda ini, Partai mengendalikan semua aspek kehidupan warganya. Kehidupan pribadi dan pemikiran

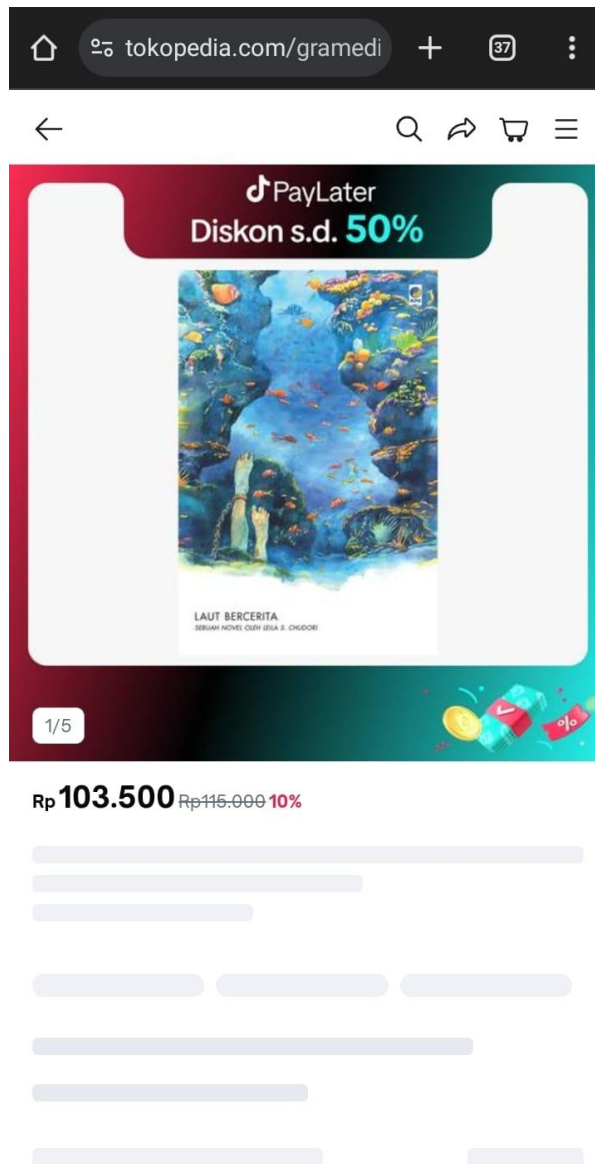
Gambar 39. Tampilan Awal UI Soal 1 Modul 4 Compose



Gambar 40. Tampilan Ketika Discroll Soal 1 Modul 4 Compose



Gambar 41. Tampilan Awal UI Landscape Soal 1 Modul 4 Compose



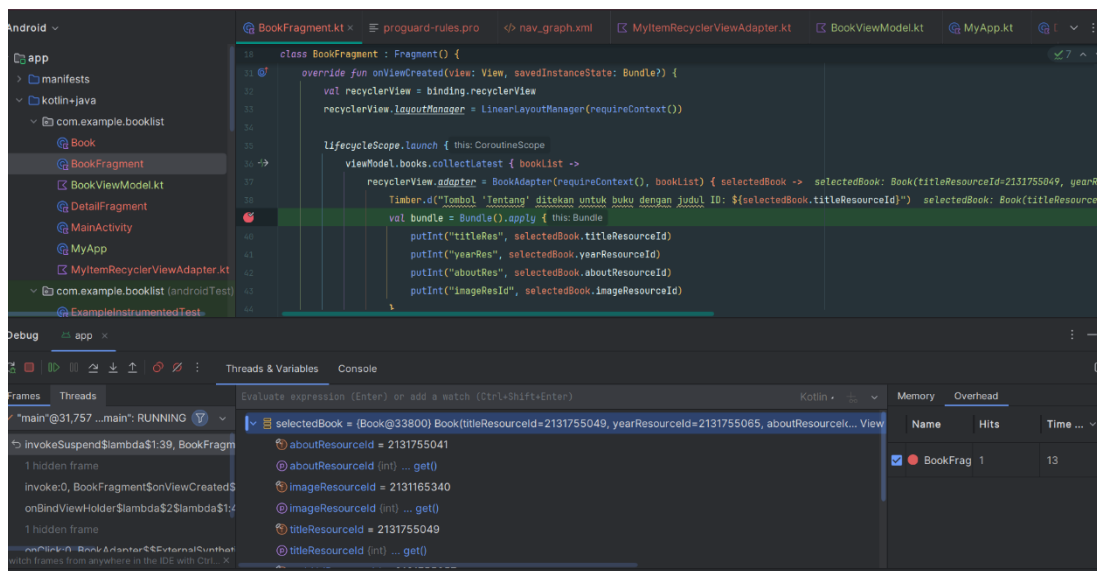
Gambar 42. Tampilan Tombol Beli Buku Ditekan Soal 1 Modul 4 Compose



Gambar 43. Tampilan Detail Soal 1 Modul 4 Compose



Gambar 44. Tampilan Detail Landscape Soal 1 Modul 4 Compose



Gambar 45. Penggunaan Debugger Soal 1 Modul 4

C. Pembahasan

XML:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin

Pada baris [3], diimport bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada baris [4], diimport `enableEdgeToEdge` agar konten bisa tampil sampai ke pinggir layar.

Pada baris [5], diimport `AppCompatActivity` superclass untuk activity yang menggunakan fitur-fitur dari AndroidX.

Pada baris [6], diimport `ViewCompat` digunakan untuk manipulasi property View.

Pada baris [7], diimport `WindowInsetsCompat` untuk mengatur tata letak yang mempertimbangkan status bar dan navigation bar.

Pada baris [8], diimport `ActivityMainBinding` class binding otomatis yang dihasilkan dari file XML `activity_main.xml`.

Pada baris [10], mendeklarasikan class `MainActivity` yang merupakan turunan dari `AppCompatActivity` sebagai titik awal saat aplikasi dijalankan.

Pada baris [11], mendefinisikan method `onCreate`, yaitu fungsi lifecycle yang dipanggil saat activity pertama kali dibuat.

Pada baris [12], memanggil implementasi `onCreate` di superclass untuk memastikan sistem Android tetap menjalankan proses inisialisasi standar activity.

Pada baris [13], membuat objek binding dengan cara menginflate layout XML yaitu `activity_main.xml` agar bisa diakses lewat kode Kotlin.

Pada baris [14], menetapkan root dari layout yang diinflate sebagai tampilan utama activity.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport `DrawableRes` untuk anotasi `@DrawableRes` yang menandakan bahwa parameter bertipe `Int` tersebut adalah resource drawable atau gambar.

Pada baris [4], diimport `StringRes` untuk anotasi `@StringRes` yang menandakan bahwa parameter `Int` tersebut adalah resource string dari folder `res/values/strings.xml`.

Pada baris [6-11], mendefinisikan class data `Book` untuk menyimpan data buku. `@StringRes val titleResourceId: Int` adalah properti untuk menyimpan ID dari resource judul buku. Anotasi `@StringRes` memberi tahu Android Studio bahwa nilai ini harus berasal dari resource string (`R.string.nama_judul`). `@StringRes val yearResourceId: Int`

adalah ID resource string untuk tahun terbit buku (misal "2022"). `@StringRes val aboutResourceId: Int` menyimpan ID resource string yang menjelaskan isi buku atau deskripsinya. `@StringRes val webUrlResourceId: Int` menyimpan ID resource string berupa URL website (untuk beli buku). `@DrawableRes val imageResourceId: Int` menyimpan ID resource gambar (drawable) untuk sampul buku. Anotasi `@DrawableRes` menjamin bahwa nilai ini berasal dari folder `res/drawable`.

BookFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–15], diimport berbagai komponen penting yang dibutuhkan untuk menjalankan fungsi fragment ini. `Bundle` digunakan untuk mengirim data antar fragment. `LayoutInflater`, `View`, dan `ViewGroup` digunakan untuk mengatur tampilan layout. `Fragment` adalah class dasar untuk membuat fragment di Android. `viewModels` digunakan untuk menginisialisasi `ViewModel` di dalam fragment. `lifecycleScope` digunakan untuk menjalankan coroutine yang mengikuti lifecycle fragment. `findNavController()` adalah fungsi untuk melakukan navigasi antar fragment dengan Jetpack Navigation. `LinearLayoutManager` digunakan untuk menampilkan data list secara vertikal di dalam `RecyclerView`. `FragmentBookBinding` merupakan class binding otomatis yang dihasilkan dari file XML `fragment_book.xml`, sehingga kita bisa mengakses komponen XML-nya langsung dari Kotlin. `collectLatest` digunakan untuk mengambil data terbaru dari `StateFlow`. `launch` digunakan untuk memulai coroutine. `Timber` adalah library logging yang digunakan untuk mencatat event dalam logcat.

Pada baris [18], mendeklarasikan class fragment.

Pada baris [19-21], mendeklarasikan `ViewModel` dengan nama `viewModel` bertipe `BookViewModel`. `ViewModel` ini diinisialisasi menggunakan delegasi `by viewModels` dan dibantu oleh `BookViewModelFactory` dengan parameter kategori “MyCategory”. Pada baris [23], menyimpan binding dengan tipe nullable untuk menghindari memory leak.

Pada baris [24], memberi akses non null ke binding saat view aktif.

Pada baris [26–29], method `onCreateView()` digunakan untuk menginflate layout `fragment_book.xml` menjadi view, lalu `FragmentBookBinding.inflate()` menghubungkan dengan `ViewBinding`. Return `binding.root` untuk mengembalikan view utama fragment.

Pada baris [31–49], method `onViewCreated()` dipanggil setelah layout fragment berhasil ditampilkan di layar. Di dalamnya, pertama-tama diambil referensi ke `RecyclerView` menggunakan `binding.recyclerView`. Kemudian, `RecyclerView` dikonfigurasi untuk menampilkan daftar item secara vertikal dengan menggunakan `LinearLayoutManager`. Setelah itu, diluncurkan coroutine untuk mengamati `StateFlow` dari `ViewModel`. Adapter `BookAdapter` diset dengan data dari `bookList` yang berasal dari `ViewModel`. Adapter juga menerima lambda (callback) yang akan dipanggil saat pengguna mengklik tombol tentang pada salah satu buku. Ketika tombol ditekan, dicetak log menggunakan `Timber.d` yang berisi ID judul buku. Setelah itu, dibuat `Bundle` berisi informasi judul, tahun, deskripsi, dan gambar buku. `Bundle` ini dikirimkan ke `DetailFragment` menggunakan perintah navigasi `findNavController().navigate()`.

Pada baris [51–55], method `onDestroyView()` dipanggil saat fragment akan dihancurkan (misalnya ketika berpindah ke fragment lain). Di sini, `_binding` diset ke null untuk menghapus referensi ke view yang sudah tidak aktif, sehingga membantu mencegah terjadinya memory leak.

DetailFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–8], diimport komponen-komponen penting yang dibutuhkan dalam fragment ini. `Bundle` digunakan untuk menerima data yang dikirim dari fragment sebelumnya. `Fragment` adalah superclass yang digunakan untuk membuat fragment. `LayoutInflater`, `View`, dan `ViewGroup` digunakan untuk mengelola dan menampilkan tampilan (layout) fragment. Terakhir, `FragmentDetailBinding` adalah class `ViewBinding` yang secara otomatis dihasilkan dari file XML `fragment_detail.xml`.

Pada baris [10], dideklarasikan class DetailFragment yang merupakan turunan dari Fragment. Class ini bertugas menampilkan detail buku yang dipilih oleh pengguna dari daftar buku.

Pada baris [12], dibuat properti `_binding` yang bertipe nullable dari `FragmentDetailBinding`. Properti ini akan menyimpan objek binding yang menghubungkan komponen layout dengan kode Kotlin, dan dibuat nullable agar bisa dihapus saat view dihancurkan untuk mencegah memory leak.

Pada baris [13], dibuat properti binding yang bersifat non-nullable dengan cara memaksa akses ke `_binding` menggunakan `!!` untuk mempermudah akses ke komponen layout selama fragment masih dalam keadaan aktif.

Pada baris [15–22], method `onCreateView()` digunakan untuk menginflate layout `fragment_detail.xml` menggunakan `FragmentDetailBinding.inflate()`. Hasil inflate disimpan ke `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama dari fragment ini.

Pada baris [24–35], method `onViewCreated()` dijalankan ketika layout fragment sudah selesai ditampilkan. Di sini, kita mengambil data yang dikirim dari `BookFragment` menggunakan objek arguments. Kemudian, masing-masing data diambil dari `Bundle` dengan kunci `"titleRes"`, `"yearRes"`, `"aboutRes"`, dan `"imageResId"`. Jika datanya tidak tersedia, maka akan digantikan dengan nilai default 0. Setelah itu, data tersebut ditampilkan ke layout. judul buku di-set ke `bookTitle`, tahun terbit ke `bookYear`, deskripsi ke `bookAbout`, dan gambar sampul ke `bookImage`.

Pada baris [37–41], method `onDestroyView()` dipanggil saat view fragment akan dihancurkan. Di sini, `_binding` diset ke null untuk membersihkan referensi terhadap view dan mencegah terjadinya memory leak.

BookViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–7], diimport berbagai komponen penting yang dibutuhkan untuk mendefinisikan `ViewModel`. `ViewModel` digunakan untuk menyimpan data yang terkait dengan UI dan bertahan saat konfigurasi berubah (seperti rotasi layar). `ViewModelProvider` digunakan untuk membuat `ViewModel` menggunakan factory.

MutableStateFlow dan StateFlow digunakan untuk menyimpan dan mengamati data secara reaktif dalam arsitektur berbasis coroutine. Timber adalah library logging yang digunakan untuk mencatat aktivitas dalam aplikasi.

Pada baris [9], dideklarasikan class BookViewModel yang merupakan subclass dari ViewModel. ViewModel ini digunakan untuk menyimpan dan mengelola daftar buku dalam aplikasi. Class ini memiliki satu parameter bernama category bertipe String yang akan diterima melalui factory.

Pada baris [11], dibuat variabel privat _books bertipe MutableStateFlow dengan tipe data List<Book>. Variabel ini menyimpan daftar buku yang nantinya akan diobservasi oleh UI.

Pada baris [12], dibuat properti books yang merupakan StateFlow<List<Book>> dan merepresentasikan data yang bisa diamati dari luar class, namun hanya bisa diubah dari dalam ViewModel.

Pada baris [14-16], dibuat blok init yang akan dijalankan secara otomatis saat ViewModel dibuat. Di dalam blok ini, dipanggil fungsi loadBooks() untuk memuat data awal ke dalam daftar buku.

Pada baris [19-32], didefinisikan fungsi privat loadBooks() yang akan menetapkan isi dari daftar buku _books. Di dalamnya, dimasukkan delapan objek Book yang setiap parameternya mengambil data dari R.string dan R.drawable. Semua data diambil dari file resources seperti strings.xml dan drawable. Setelah data dimuat, dicetak log menggunakan Timber.i() yang menampilkan jumlah buku yang berhasil dimuat.

Pada baris [34], dideklarasikan class BookViewModelFactory yang merupakan subclass dari ViewModelProvider.Factory. Factory ini digunakan untuk membuat instance BookViewModel yang memerlukan parameter tambahan, yaitu category.

Pada baris [35], fungsi create() dioverride dari ViewModelProvider.Factory. Fungsi ini akan dipanggil secara otomatis saat kita memanggil by viewModels { } di dalam fragment.

Pada baris [36-39], dilakukan pengecekan apakah modelClass merupakan subclass dari BookViewModel. Jika iya, maka dibuat instance baru BookViewModel dengan

parameter category, lalu dikembalikan sebagai tipe generic T dengan menggunakan `@SuppressWarnings("UNCHECKED_CAST")` untuk menghindari warning cast.

Pada baris [40], jika tipe ViewModel yang diminta tidak dikenali, maka akan dilemparkan exception `IllegalArgumentException`.

MyItemRecyclerViewAdapter.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport berbagai komponen yang dibutuhkan dalam pembuatan adapter RecyclerView. Context dibutuhkan untuk akses ke resource aplikasi. Intent dan Uri digunakan untuk membuka link eksternal (seperti membuka halaman web). LayoutInflater dipakai untuk meng-inflate layout XML ke bentuk View. ViewGroup digunakan sebagai induk tampilan item. RecyclerView adalah komponen utama untuk membuat daftar yang bisa digulir. Terakhir, ItemBookBinding adalah class binding otomatis yang dihasilkan dari file `item_book.xml`. Timber digunakan sebagai alat untuk mencatat aktivitas ke logcat.

Pada baris [12–16], dideklarasikan class BookAdapter sebagai turunan dari RecyclerView.Adapter. Class ini bertugas menyediakan data dan tampilan untuk setiap item buku di dalam RecyclerView. Adapter ini memiliki tiga parameter: context untuk akses ke resource dan layout inflater, books yaitu list data buku yang akan ditampilkan, dan onAboutClick yaitu fungsi callback yang dijalankan ketika tombol "About" pada suatu item diklik.

Pada baris [18], dibuat inner class BookViewHolder yang bertugas menyimpan binding ke tampilan setiap item buku. Class ini merupakan turunan dari RecyclerView.ViewHolder, dan menggunakan binding.root sebagai tampilan utama untuk setiap elemen daftar.

Pada baris [20–23], diimplementasikan method onCreateViewHolder() yang bertugas membuat ViewHolder baru. Method ini meng-inflate layout `item_book.xml` menggunakan ItemBookBinding, lalu membungkusnya dalam objek BookViewHolder yang dikembalikan.

Pada baris [25–44], diimplementasikan method `onBindViewHolder()` yang bertugas menghubungkan data buku dengan tampilan. Data buku diambil berdasarkan posisi (index) dalam list. Kemudian, nilai dari setiap properti buku ditampilkan pada komponen UI: judul, tahun, deskripsi, dan gambar buku. Untuk tombol "Buy", ketika diklik, akan dicetak log menggunakan Timber, lalu dibuat Intent dengan `ACTION_VIEW` dan membuka URL website buku di browser. Untuk tombol "About", ketika diklik, akan dicetak log berisi judul buku menggunakan Timber, lalu memanggil fungsi `onAboutClick()` dengan parameter objek `Book` yang sesuai.

Pada baris [46], diimplementasikan method `getItemCount()` yang mengembalikan jumlah total item dalam list buku. Method ini digunakan oleh `RecyclerView` untuk mengetahui berapa banyak data yang harus ditampilkan.

MyApp.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport class-class penting yang dibutuhkan untuk mendefinisikan class aplikasi. `Application` adalah superclass dari seluruh komponen aplikasi Android dan digunakan untuk mengelola konfigurasi global aplikasi. Timber adalah library logging yang digunakan untuk mencatat log ke Logcat selama pengembangan dan debugging.

Pada baris [6], dideklarasikan class `MyApp` sebagai turunan dari class `Application`. Class ini digunakan untuk menginisialisasi konfigurasi global saat aplikasi pertama kali dijalankan.

Pada baris [7–10], dioverride method `onCreate()` yang akan dipanggil secara otomatis saat aplikasi dimulai. Di dalam method ini, dipanggil `Timber.plant(Timber.DebugTree())` untuk menanam `DebugTree` yang memungkinkan kita menggunakan fungsi logging Timber seperti `Timber.d()` atau `Timber.i()` selama proses debugging aplikasi.

activity_main.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–30], dideklarasikan root layout yaitu `ConstraintLayout`, yang berasal dari `library androidx.constraintlayout.widget.ConstraintLayout`. Layout ini memungkinkan pengaturan posisi elemen berdasarkan batas (constraint) antar komponen atau terhadap parent. Di dalamnya juga terdapat deklarasi `xmlns` (XML namespace) untuk android, app, dan tools, yang masing-masing digunakan untuk atribut standar Android, atribut khusus dari `AndroidX`, dan preview di editor. `tools:context=".MainActivity"` digunakan hanya di `Android Studio` untuk menunjukkan bahwa layout ini digunakan oleh `MainActivity`.

Pada baris [10–29], didefinisikan `FrameLayout` dengan ID `@+id/frame_layout` sebagai wadah utama yang mengisi seluruh layar (`match_parent` untuk lebar dan tinggi). Komponen ini diberi constraint agar menempel ke semua sisi parent `ConstraintLayout`, yaitu Top, Bottom, Start, dan End. Ini berarti layout akan memenuhi seluruh layar aplikasi. Bias horizontal dan vertikal juga diset (meskipun tidak wajib) untuk mengatur keseimbangan layout di tengah.

Pada baris [21–27], di dalam `FrameLayout`, terdapat komponen `FragmentContainerView` yang merupakan tempat untuk menampilkan fragment. Komponen ini diberi ID `@+id/nav_host_fragment`, dan diberi atribut `android:name="androidx.navigation.fragment.NavHostFragment"` yang menunjukkan bahwa fragment ini digunakan untuk sistem navigasi. Atribut `app:navGraph="@navigation/nav_graph"` menghubungkan fragment ini dengan file navigasi `nav_graph.xml`, yang berisi pengaturan alur navigasi antar fragment. Atribut `app:defaultNavHost="true"` memastikan bahwa `NavHostFragment` ini akan menangani navigasi kembali (Back) secara otomatis dari sistem.

fragment_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–19], dideklarasikan root layout menggunakan `ConstraintLayout` dari `AndroidX` (`androidx.constraintlayout.widget.ConstraintLayout`). Layout ini

memungkinkan pengaturan posisi elemen UI secara fleksibel dengan menggunakan constraint (batas) terhadap parent atau elemen lainnya. Tiga atribut xmlns didefinisikan: android untuk atribut dasar Android, app untuk atribut khusus dari library AndroidX, dan tools untuk keperluan preview di Android Studio. Atribut `tools:context=".BookFragment"` menunjukkan bahwa layout ini digunakan oleh BookFragment, sehingga Android Studio bisa menampilkan preview sesuai konteks fragment tersebut.

Pada baris [10–17], didefinisikan sebuah RecyclerView dengan ID `@+id/recyclerView`. Komponen ini digunakan untuk menampilkan daftar item buku secara efisien dalam bentuk scrollable list. Lebar (`layout_width`) dan tinggi (`layout_height`) diset ke `0dp` agar ukuran akan ditentukan oleh constraint. Komponen ini diposisikan menggunakan constraint: bagian atas dan bawah ditautkan ke Top dan Bottom parent, sedangkan bagian kiri (Start) dan kanan (End) juga ditautkan ke parent, sehingga RecyclerView akan memenuhi seluruh ruang dalam ConstraintLayout.

fragment_detail.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–7], didefinisikan ScrollView sebagai root layout, yang memungkinkan konten di dalamnya dapat digulir secara vertikal. Layout ini menggunakan `match_parent` untuk lebar dan tinggi agar mengisi seluruh layar. ScrollView ini diberi latar belakang warna gelap `#1C1C1C` dan padding sebesar `16dp`. Selain itu, terdapat deklarasi xmlns standar untuk namespace Android, serta namespace app untuk atribut khusus AndroidX seperti `layout_constraint....`

Pada baris [9–12], terdapat LinearLayout sebagai child langsung dari ScrollView, dengan orientasi vertikal. Artinya, semua elemen UI di dalamnya akan disusun dari atas ke bawah. Lebar layout diatur agar mengisi seluruh layar, dan tingginya menyesuaikan konten (`wrap_content`).

Pada baris [14–84], terdapat ConstraintLayout di dalam LinearLayout. Layout ini digunakan agar setiap elemen UI bisa diposisikan secara fleksibel berdasarkan constraint ke elemen lain. ConstraintLayout ini menggunakan lebar `match_parent` dan

tinggi `wrap_content`, sehingga akan menyesuaikan tinggi total komponen yang ada di dalamnya.

Pada baris [18–25], terdapat `CardView` dengan ID `card_image`, yang digunakan sebagai wadah untuk menampilkan gambar buku dengan sudut melengkung. Atribut `cardCornerRadius="8dp"` membuat sudut dari `CardView` menjadi agak membulat. Posisi `CardView` ini dikunci (constraint) ke atas dan sisi kiri dan kanan parent. Di dalamnya terdapat `ImageView`.

Pada baris [27–32], terdapat `ImageView` dengan ID `bookImage` yang digunakan untuk menampilkan gambar buku. Ukuran gambar ditentukan sebesar 350dp x 520dp, dan menggunakan `scaleType="centerCrop"` agar gambar memenuhi ruang yang disediakan secara proporsional. Gambar default yang digunakan adalah `@drawable/image1`.

Pada baris [35–47], didefinisikan `TextView` dengan ID `bookTitle` yang menampilkan judul buku. `TextView` ini menggunakan ukuran teks 24sp dan style teks bold, dengan warna teks putih. Posisi komponen ini dikunci di bawah dari `card_image`, dimulai dari kiri parent, dan berakhir sebelum `bookYear`, sehingga judul akan berada di sebelah kiri tahun buku dengan proporsi otomatis.

Pada baris [49–59], terdapat `TextView` lain dengan ID `bookYear`, yang menampilkan tahun terbit buku. Ukuran teks sama seperti judul buku yaitu 24sp, dengan warna putih. Letaknya dikunci agar berada di kanan dari `bookTitle`, serta berada di bawah gambar. Penempatan ini membuat `bookYear` sejajar horizontal dengan `bookTitle`, tetapi di sisi kanan.

Pada baris [61–70], `TextView` dengan ID `textTentang` digunakan sebagai label kecil bertuliskan “Tentang”, sebagai penanda sebelum menampilkan deskripsi buku. Ukuran teks adalah 14sp dengan warna putih. Komponen ini diletakkan di bawah `bookTitle` dan berada di sisi kiri layar, mengikuti parent.

Pada baris [72–83], `TextView` dengan ID `bookAbout` digunakan untuk menampilkan deskripsi atau sinopsis buku. Ukuran teks kecil yaitu 14sp, dengan warna putih. Komponen ini dikunci posisinya tepat di bawah `textTentang`, dimulai dari kiri parent dan berakhir di sisi kanan parent. Karena tinggi layout `wrap_content`, maka isi deskripsi

akan menyesuaikan panjang teks. Margin atas dan bawah digunakan untuk memberi jarak dari elemen lain.

Item_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–125], layout utama menggunakan ConstraintLayout yang fleksibel dalam menempatkan elemen berdasarkan constraint. Lebarnya diset `match_parent`, artinya akan mengisi lebar parent-nya, dan tingginya `wrap_content`, jadi hanya sebesar konten di dalamnya.

Pada baris [8–24], terdapat CardView sebagai kontainer utama yang membungkus semua isi item buku. Ini memberikan efek melengkung pada sudutnya (`cardCornerRadius="16dp"`) dan latar belakang abu gelap (`@color/dark_gray`). CardView ini diberi margin 8dp agar tidak mepet ke item lain di RecyclerView. Constraint-nya mengatur agar tetap di tengah-tengah parent layout.

Pada baris [19–123], di dalam CardView luar terdapat ConstraintLayout kedua yang mengatur elemen-elemen detail buku.

Pada baris [23–33], terdapat CardView kecil (ID: `card_image`) yang berisi gambar buku (ImageView). CardView ini memiliki `cardCornerRadius="8dp"` agar gambar memiliki sudut melengkung. Di dalamnya, ImageView berukuran 123dp x 200dp menampilkan gambar buku dengan `scaleType="centerCrop"` agar gambar memenuhi ruang secara proporsional.

Pada baris [43–55], TextView dengan ID `bookTitle` menampilkan judul buku. Ukurannya besar (24sp), bold, dan berwarna putih. Constraint-nya menempatkan teks ini di kanan gambar, dan tidak sampai ke tepi kanan karena akan ada elemen lain di sana.

Pada baris [57–66], TextView dengan ID `bookYear` menampilkan tahun terbit buku, berada di kanan `bookTitle`, dan dekat tepi kanan parent.

Pada baris [68–79], TextView dengan ID `textTentang` menampilkan label “Tentang”. Letaknya tepat di bawah `bookTitle`, dan berada di sebelah kiri `bookAbout`.

Pada baris [81–93], TextView dengan ID bookAbout berisi sinopsis atau deskripsi buku. Letaknya di sebelah kanan textTentang, dan posisinya di bawah bookTitle. Lebar ditentukan 184dp, agar cukup panjang untuk menampilkan kalimat pendek atau potongan sinopsis.

Pada baris [95–107], terdapat Button dengan ID btnBuy untuk aksi “Beli”. Tombol ini posisinya di bawah bookAbout dan di sebelah kiri tombol detail. Ukurannya fleksibel (wrap_content) dengan teks dari string resource beli_button.

Pada baris [109–122], tombol btnAbout untuk membuka detail buku. Letaknya di sebelah kanan tombol beli, dan juga di bawah bookAbout. Tombol ini akan memicu perpindahan ke halaman detail jika diklik.

Compose:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–30], diimport berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. ComponentActivity diimport untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. setContent digunakan untuk menetapkan tampilan UI aplikasi, sementara enableEdgeToEdge memungkinkan tampilan aplikasi untuk menggunakan seluruh area layar, mengabaikan area tepi yang biasanya terblokir oleh status bar atau navigation bar. Selain itu, berbagai fungsi dan class seperti Surface, Text, Modifier, ViewModel, collectAsState, NavHost, composable, dan lainnya diimport untuk membangun dan mengelola UI serta navigasi menggunakan Jetpack Compose. BookDetailScreen dan BookListScreen digunakan sebagai composable utama yang menampilkan daftar dan detail buku. BookListTheme digunakan sebagai tema utama aplikasi.

Pada baris [32], MainActivity dideklarasikan sebagai turunan dari ComponentActivity. Class ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai

aktivitas utama, MainActivity bertanggung jawab untuk mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [33–41], dalam method onCreate(), kita memanggil super.onCreate(savedInstanceState) untuk memastikan siklus hidup activity dijalankan sesuai dengan standar. Kemudian, enableEdgeToEdge() dipanggil untuk memungkinkan konten tampilan memenuhi seluruh layar, tanpa ada pembatasan dari status bar. Selanjutnya, setContent digunakan untuk menetapkan tampilan UI aplikasi yang menggunakan tema BookListTheme, yang didefinisikan dalam file tema dan memastikan konsistensi style visual aplikasi. Di dalamnya, kita memanggil fungsi BooksApp() yang akan menampilkan tampilan utama aplikasi.

Pada baris [45–78], terdapat deklarasi fungsi BooksApp untuk mengelola struktur UI utama aplikasi dengan menggunakan Jetpack Compose. Fungsi ini diatur agar dapat bekerja dengan sistem navigasi dan memberikan tampilan daftar buku dan detail buku yang dapat dipilih oleh pengguna.

Pada baris [46], variabel layoutDirection dideklarasikan untuk mendapatkan informasi tentang arah tata letak UI. Variabel ini berguna untuk mengelola arah tata letak komponen, terutama ketika mendukung bahasa RTL (Right-to-Left), yang mempengaruhi cara elemen-elemen UI diposisikan.

Pada baris [47], variabel navController dideklarasikan dan diinisialisasi dengan rememberNavController(), yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. navController digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar buku ke tampilan detail buku.

Pada baris [48], variabel viewModel dideklarasikan dan diinisialisasi menggunakan fungsi viewModel(), yang mengambil instance dari BookListViewModel. ViewModel ini digunakan untuk menyimpan dan mengelola data yang diperlukan oleh UI.

Pada baris [49], variabel bookList dideklarasikan untuk mengobservasi StateFlow dari viewModel.booksList menggunakan fungsi collectAsState(). Variabel ini akan secara otomatis memperbarui UI saat data buku berubah.

Pada baris [50], variabel `selectedBook` dideklarasikan untuk mengobservasi `StateFlow` dari `viewModel.selectedBook` menggunakan `collectAsState()`. Data ini akan digunakan untuk menampilkan detail buku yang dipilih oleh pengguna.

Pada baris [52–76], `Surface` dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI, dengan menggunakan modifier untuk mengatur ukuran dan padding. `fillMaxSize()` membuat `Surface` mengambil seluruh ruang yang tersedia di layar, sementara `statusBarsPadding()` memberi ruang di bagian atas layar agar konten tidak tertutup oleh status bar. Kemudian, `padding()` digunakan untuk menambahkan padding di sisi kiri dan kanan berdasarkan tata letak yang aman (*safe drawing area*), yang memastikan konten tidak terhalang oleh elemen UI seperti status bar atau navigation bar.

Pada baris [61–75], `NavHost` dideklarasikan untuk mengelola navigasi antar tampilan. `startDestination` ditetapkan ke `"list"`, yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar buku. Di dalam `NavHost`, ada dua route yang didefinisikan. Route `"list"` akan memanggil `BookListScreen`, dengan daftar buku yang berasal dari variabel `bookList`. Ketika pengguna mengklik tombol detail, fungsi `onDetailClick` akan memanggil `viewModel.selectBook()` untuk memilih buku berdasarkan index, dan kemudian menavigasi ke tampilan `"detail"`. Route `"detail"` akan memanggil `BookDetailScreen` jika `selectedBook` tidak null. Jika data buku tidak ditemukan, maka akan ditampilkan pesan “Buku tidak ditemukan” menggunakan `Text`.

Datasource.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport resource `R` yang merujuk pada file resource aplikasi, seperti `strings.xml` dan `drawable`. Import ini memungkinkan kita untuk mengakses berbagai resource yang ada dalam aplikasi, seperti string untuk teks, gambar untuk `drawable`, dan lainnya. Selain itu, diimport juga `Book` dari model `com.example.booklist.model.Book`. `Book` adalah class model yang menyimpan informasi tentang buku, seperti judul, tahun terbit, deskripsi, URL, dan gambar sampul yang akan digunakan dalam aplikasi untuk menampilkan data buku.

Pada baris [6], dideklarasikan class `Datasource`, yang bertugas untuk menyediakan data buku. Class ini merupakan bagian dari arsitektur aplikasi yang bertanggung jawab untuk mengelola dan menyediakan data yang diperlukan. `Datasource` biasanya digunakan untuk memisahkan logika pengambilan data dari logika tampilan atau UI, sehingga kode aplikasi lebih modular dan mudah dikelola. Dalam hal ini, class `Datasource` menyediakan data buku yang akan digunakan oleh fragment atau activity dalam aplikasi.

Pada baris [7–18], terdapat method `loadBooks()` yang berfungsi untuk mengembalikan sebuah list yang berisi objek `Book`. Dalam method ini, kita menggunakan `listOf<Book>()` untuk membuat sebuah list baru yang berisi objek-objek `Book`. Setiap objek `Book` dibentuk dengan memasukkan berbagai ID resource dari file `strings.xml` dan `drawable`. Misalnya, untuk judul buku, kita menggunakan `R.string.book_title1` yang merujuk pada string dengan nama `book_title1` di `strings.xml`. Begitu juga dengan tahun terbit, deskripsi, URL, dan gambar sampul buku yang diambil dari file `strings.xml` dan folder `drawable`. Dengan cara ini, aplikasi kita menjadi lebih fleksibel karena kita bisa mengelola data di file resource dan tidak perlu mengubah kode sumber setiap kali ada perubahan pada data buku. List yang berisi data buku ini kemudian dapat digunakan oleh komponen lain dalam aplikasi, seperti adapter di `RecyclerView`, untuk menampilkan daftar buku ke pengguna.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport dua anotasi penting dari `AndroidX`: `@DrawableRes` dan `@StringRes`. Kedua anotasi ini digunakan untuk menunjukkan bahwa nilai parameter merupakan ID dari resource, bukan data literal.

Pada baris [6–12], didefinisikan sebuah data class bernama `Book`. Class ini digunakan untuk merepresentasikan data sebuah buku dalam bentuk struktur objek yang sederhana dan otomatis menyediakan fungsi-fungsi dasar seperti `toString()`, `equals()`, dan `hashCode()` karena merupakan data class. `titleResourceId`, `yearResourceId`, `aboutResourceId`, dan `webUrlResourceId` bertipe `Int` dan diberi anotasi `@StringRes`,

menandakan bahwa nilainya merujuk pada string resource. Sedangkan `imageResourceId` bertipe `Int` dan diberi anotasi `@DrawableRes`, menandakan bahwa ini adalah ID dari resource gambar.

BookDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–32], diimport komponen penting dari Android dan Jetpack Compose. `Configuration` digunakan untuk mendeteksi orientasi layar. Komponen tata letak seperti `Row`, `Column`, `Spacer`, `Image`, serta atribut ukuran seperti `size`, `width`, `height`, dan `fillMaxSize` diambil dari `androidx.compose.foundation.layout`. `RoundedCornerShape` digunakan untuk memberi sudut membulat pada gambar. `MaterialTheme` dan `Text` digunakan untuk styling teks sesuai Material Design 3. `Modifier`, `Alignment`, `ContentScale`, `clip`, dan `verticalScroll` digunakan untuk mengatur tampilan dan perilaku UI. `painterResource` dan `LocalContext` digunakan untuk mengambil gambar dan teks dari resource aplikasi. Satuan ukuran seperti `dp` dan `sp` digunakan untuk mengatur dimensi dan ukuran huruf. Objek `Book` diimport dari package `model` sebagai data yang ditampilkan. `Timber` digunakan untuk mencatat log aktivitas saat detail buku ditampilkan.

Pada baris [35–80], dideklarasikan fungsi `BookDetailScreen()` sebagai composable yang menerima satu parameter `book` bertipe `Book`. Fungsi ini bertugas menampilkan halaman detail dari sebuah buku tertentu, termasuk gambar, judul, tahun, dan deskripsi buku.

Pada baris [36], log dicetak menggunakan `Timber.i` untuk menandai bahwa detail dari buku yang dimaksud sedang ditampilkan. Teks log berisi string judul buku yang diambil dari resources dengan menggunakan `LocalContext.current.getString()`.

Pada baris [37–42], digunakan layout `Column`, yang menyusun elemen UI secara vertikal dari atas ke bawah. Modifier yang digunakan adalah `fillMaxSize()` agar komponen memenuhi seluruh layar, `padding 16.dp` untuk memberi ruang tepi, dan `verticalScroll` agar konten dapat digulir jika melebihi tinggi layar.

Pada baris [43–51], ditampilkan gambar buku menggunakan komponen Image. Gambar dimuat dari resources menggunakan `painterResource(book.imageResourceId)`. Ukuran gambar diatur sebesar lebar 380.dp dan tinggi 550.dp. Gambar dibulatkan menggunakan `clip(RoundedCornerShape(16.dp))` dan disejajarkan secara horizontal di tengah dengan `align(Alignment.CenterHorizontally)`. Properti `contentScale = ContentScale.Crop` digunakan agar gambar memenuhi ukuran yang ditentukan secara proporsional.

Pada baris [53], Spacer digunakan untuk memberikan jarak vertikal sebesar 16.dp antara gambar dan teks di bawahnya.

Pada baris [55–78], digunakan Column untuk menyusun teks informasi buku secara vertikal. `Modifier.fillMaxWidth()` digunakan agar kolom ini mengambil seluruh lebar yang tersedia. Atribut `horizontalAlignment = Alignment.Start` digunakan untuk menyelaraskan isi kolom ke sebelah kiri.

Pada baris [59–70], digunakan layout Row untuk menyusun judul buku dan tahun terbit secara horizontal di dalam satu baris. Atribut `horizontalArrangement = Arrangement.SpaceBetween` digunakan agar kedua komponen memiliki jarak antar ujung. Judul buku ditampilkan di sebelah kiri menggunakan Text dengan teks yang diambil dari `book.titleResourceId`, menggunakan style `MaterialTheme.typography.labelLarge` dan ukuran huruf 26.sp.

Pada baris [65–69], tahun terbit buku ditampilkan menggunakan Text di sisi kanan, dengan teks dari `book.yearResourceId`, menggunakan style `MaterialTheme.typography.bodyMedium` dan ukuran huruf 20.sp.

Pada baris [71], Spacer ditambahkan untuk memberi jarak vertikal sebesar 8.dp sebelum bagian deskripsi.

Pada baris [72], label teks “Tentang:” ditampilkan sebelum deskripsi buku, menggunakan style teks kecil `MaterialTheme.typography.bodySmall`.

Pada baris [73–77], deskripsi buku ditampilkan menggunakan Text dengan isi teks yang diambil dari `book.aboutResourceId`. Teks ini ditampilkan menggunakan style `MaterialTheme.typography.bodySmall` dan ukuran huruf 14.sp.

BookListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–38], diimport komponen penting dari Android dan Jetpack Compose. Intent dan Uri dari android.content dan android.net digunakan untuk membuka URL eksternal. Komponen tata letak seperti Row, Column, Spacer, Image, dan LazyColumn diambil dari androidx.compose.foundation.layout dan foundation.lazy. Untuk tampilan material modern, digunakan Card, Button, Text, dan MaterialTheme dari androidx.compose.material3. Elemen tambahan seperti painterResource, stringResource, LocalContext, serta atribut tampilan (clip, ContentScale, dan Alignment) mendukung pengambilan resource dan pengaturan gambar. Preview, Modifier, dan unit ukuran seperti dp dan sp juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data Book diimport dari package model dan class BooksApp disiapkan dari package utama aplikasi. Timber digunakan untuk mencatat log saat tombol ditekan.

Pada baris [41–50], didefinisikan fungsi composable BookListScreen() yang menerima tiga parameter: bookList berupa daftar buku (List<Book>), onDetailClick yaitu lambda untuk menangani klik tombol detail, dan modifier opsional untuk menyesuaikan tampilan dari luar.

Pada baris [42], digunakan LazyColumn sebagai layout vertikal yang mendukung pemrosesan daftar besar secara efisien. Modifier disisipkan jika diperlukan dari luar composable ini.

Pada baris [43–48], digunakan fungsi itemsIndexed() untuk mengiterasi daftar indeks dari bookList. Untuk setiap index, pertama dicetak log menggunakan Timber.d() untuk menandai bahwa tombol “Detail” ditekan untuk buku tertentu. Kemudian dipanggil composable BookCard, di mana objek bookList[index] dikirim sebagai argumen. Parameter onDetailClick juga diteruskan, membungkus nilai index agar dapat dikenali saat tombol diklik.

Pada baris [53–135], dideklarasikan composable BookCard(), yang bertugas menampilkan kartu berisi data singkat dari sebuah buku. Fungsi ini menerima book

sebagai parameter utama, `onDetailClick` untuk navigasi ke detail, dan modifier opsional.

Pada baris [54–58], digunakan `Card` dari `Material 3` sebagai kontainer utama yang memiliki `padding` sebesar `8.dp` dan bentuk sudut membulat (`RoundedCornerShape(16.dp)`). Modifier `fillMaxWidth()` memastikan kartu mengisi lebar layar.

Pada baris [60], variabel `context` diset sebagai `LocalContext.current` agar dapat digunakan untuk mengambil string resource atau memulai intent dari dalam fungsi `composable`.

Pada baris [65–132], `layout Row` digunakan untuk menyusun elemen secara horizontal. `Padding` ditetapkan `16.dp`, dan `fillMaxWidth()` membuat baris mengisi lebar penuh.

Pada baris [66–74], ditampilkan gambar buku menggunakan `Image.Resource` gambar diakses dari ID dalam `book.imageResourceId` melalui `painterResource()`. `contentDescription` diberikan melalui `stringResource(book.aboutResourceId)` agar sesuai dengan prinsip aksesibilitas. Ukuran gambar diatur dengan `size(width = 120.dp, height = 200.dp)` dan bentuknya dibulatkan menggunakan `clip(RoundedCornerShape(12.dp))`. Gambar disejajarkan secara vertikal di tengah baris (`align(Alignment.CenterVertically)`) dan `ContentScale.Crop` menjaga rasio aspek gambar.

Pada baris [75], `Spacer` memberikan jarak horizontal `12.dp` antara gambar dan kolom teks di sebelah kanan.

Pada baris [76–131], digunakan `Column` untuk menampilkan isi teks dan tombol dalam tata letak vertikal. Modifier `weight(1f)` membuat kolom ini memenuhi ruang horizontal yang tersisa, dan `align(Alignment.CenterVertically)` memastikan kolom berada di tengah baris secara vertikal.

Pada baris [81–98], `Row` di dalam kolom digunakan untuk menampilkan judul buku di sebelah kiri dan tahun terbit di kanan. `Arrangement.SpaceBetween` memastikan keduanya memiliki jarak yang merata di antara mereka. Judul buku ditampilkan dalam `Text` menggunakan resource string, dengan style `labelLarge`, ukuran huruf `26.sp`, dan

`overflow = TextOverflow.Ellipsis` agar teks panjang tidak meluber keluar. Tahun terbit ditampilkan di kanan dengan ukuran lebih kecil, 14.sp, dan style `bodyMedium`.

Pada baris [99], `Spacer` menambahkan jarak vertikal sebesar 20.dp sebelum deskripsi buku.

Pada baris [100–111], terdapat `Row` untuk memuat deskripsi buku ditampilkan dalam bentuk dua `Text`. Pertama teks label “Tentang:”, lalu diikuti oleh isi deskripsi dari resource string `book.aboutResourceId`. Keduanya menggunakan style `bodySmall` dan ukuran huruf 14.sp.

Pada baris [112], `Spacer` memberikan jarak sebesar 16.dp sebelum tombol aksi.

Pada baris [113–129], digunakan `Row` dengan `horizontalArrangement = Arrangement.End` agar tombol-tombol sejajar di sebelah kanan layar. Dua buah tombol ditampilkan: “Beli Buku” dan “Detail”.

Pada baris [115–122], tombol “Beli Buku” diberi aksi ketika diklik, yaitu mengambil URL dari `book.webUrlResourceId`, lalu membuat `Intent` dengan `ACTION_VIEW` dan membuka URL tersebut dengan `context.startActivity()` untuk membuka halaman pembelian buku melalui browser. Log aktivitas dicatat menggunakan `Timber.tag(...).d(...)`.

Pada baris [123], `Spacer` memberi jarak horizontal antara dua tombol sebesar 8.dp.

Pada baris [124–128], tombol “Detail” memanggil `onDetailClick()`, yaitu fungsi yang diteruskan dari luar `composable` dan biasanya mengarahkan pengguna ke halaman detail buku.

BookListViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport beberapa komponen penting untuk mendukung arsitektur MVVM (Model-View-ViewModel) dan pemrosesan asynchronous menggunakan `coroutine`. `ViewModel` dan `viewModelScope` dari `androidx.lifecycle` digunakan untuk mengelola data secara lifecycle-aware. Class `Datasource` dan model `Book` diambil dari package aplikasi yang menangani data dan struktur objek buku. `MutableStateFlow` dan

StateFlow dari `kotlinx.coroutines.flow` digunakan untuk mengelola dan mengamati state secara reaktif. Terakhir, Timber digunakan untuk mencatat log aktivitas.

Pada baris [12], dideklarasikan class `BookListViewModel` sebagai turunan dari `ViewModel`. Class ini memiliki satu parameter bernama `source` bertipe `String`, yang bisa digunakan untuk menentukan asal data atau untuk logging.

Pada baris [13–14], dibuat variabel privat `_booksList` bertipe `MutableStateFlow<List<Book>>`, yang menyimpan daftar buku dan diinisialisasi dengan list kosong. Versi publiknya, `booksList`, disediakan sebagai `StateFlow` yang hanya bisa dibaca, sehingga UI hanya dapat mengamati, tidak memodifikasi langsung nilainya.

Pada baris [16–17], dibuat variabel `_selectedBook` bertipe `MutableStateFlow<Book?>` untuk menyimpan satu buku yang dipilih. Nilainya bisa `null`. Sama seperti sebelumnya, variabel publik `selectedBook` disediakan dalam bentuk `StateFlow` untuk diamati UI tanpa bisa diubah secara langsung.

Pada baris [19–20], fungsi `init` dijalankan saat instance dari `ViewModel` ini pertama kali dibuat. Di dalamnya, dipanggil fungsi `loadBooks()` untuk memuat data buku.

Pada baris [20], dicatat log sumber data menggunakan `Timber.d()` dengan pesan yang menampilkan nilai parameter `source`. Ini berguna untuk debugging ketika ingin mengetahui dari mana data berasal.

Pada baris [24–30], didefinisikan fungsi privat `loadBooks()` yang digunakan untuk mengambil data dari sumber (`Datasource`). Pemanggilan dilakukan dalam coroutine menggunakan `viewModelScope.launch`, agar proses berjalan asynchronous tanpa membekukan UI. Dalam coroutine, dipanggil `Datasource().loadBooks()` untuk mengambil daftar buku. Hasilnya disimpan ke dalam `_booksList.value`. Setelah berhasil, dicatat log jumlah item buku yang dimuat menggunakan `Timber.d()`.

Pada baris [32–35], didefinisikan fungsi publik `selectBook(book: Book)` untuk menyimpan buku yang dipilih pengguna ke dalam `_selectedBook`. Fungsi ini juga mencatat log informasi ID judul buku yang dipilih menggunakan `Timber.d()`, yang berguna untuk debugging atau pelacakan interaksi pengguna.

BookListViewModelFactory.kt:

Pada baris [1], dideklarasikan package tempat file ini berada, yaitu `com.example.booklist.screens`.

Pada baris [3–4], diimport class `ViewModel` dan `ViewModelProvider` dari library `Jetpack Lifecycle`. Keduanya digunakan untuk mengelola siklus hidup dan penyediaan instance `ViewModel` secara aman dan sesuai konteks lifecycle Android.

Pada baris [6], dideklarasikan class `BookListViewModelFactory` yang merupakan turunan dari interface `ViewModelProvider.Factory`. Class ini berfungsi untuk membuat instance `ViewModel` secara manual, khususnya saat `ViewModel` memerlukan argumen dalam konstruktor. Parameter `source` disiapkan dalam konstruktor meskipun belum digunakan.

Pada baris [7–12], override dilakukan pada fungsi `create()`. Fungsi ini memeriksa apakah class yang diminta (`modelClass`) adalah subclass dari `BookListViewModel`. Jika ya, maka dibuat dan dikembalikan instance `BookListViewModel(source)`, yaitu dengan meneruskan parameter `source`. Karena fungsi mengembalikan tipe generik `T`, maka diperlukan casting `as T` dan anotasi `@SuppressWarnings("UNCHECKED_CAST")` untuk menghindari peringatan dari compiler. Jika `modelClass` tidak sesuai, dilemparkan `IllegalArgumentException` dengan pesan class tidak dikenali.

MyApplication.kt:

Pada baris [1], ditentukan package tempat file ini berada, yaitu `com.example.booklist`.

Pada baris [3], diimport class `Application` dari Android SDK. Class ini merepresentasikan entry point global aplikasi dan digunakan untuk menginisialisasi komponen global saat aplikasi pertama kali dibuat.

Pada baris [4], diimport library `Timber`, yaitu library logging populer di Android yang mempermudah debugging dan pencatatan log.

Pada baris [7], dideklarasikan class `MyApplication` yang merupakan turunan dari class `Application`. Ini berarti class ini akan dijalankan sekali saat aplikasi pertama kali dimulai, sebelum aktivitas manapun dibuat.

Pada baris [8], method `onCreate()` dioverride. Method ini dipanggil oleh sistem saat aplikasi pertama kali dijalankan.

Pada baris [9], dipanggil `super.onCreate()` untuk memastikan inisialisasi dari class `Application` tetap dilakukan oleh Android framework.

Pada baris [11], dilakukan pengecekan terhadap `BuildConfig.DEBUG`. Jika aplikasi dalam mode debug (misalnya saat pengembangan), maka blok di dalam if akan dieksekusi.

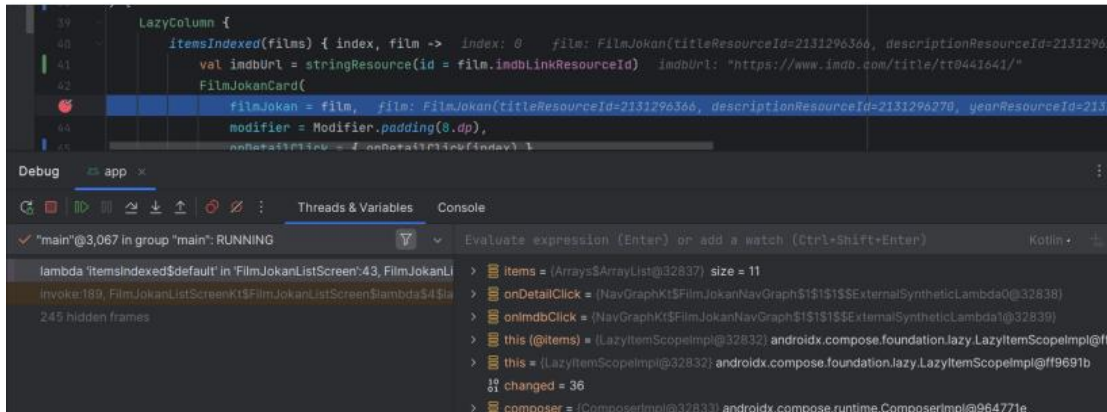
Pada baris [12], dipanggil `Timber.plant(Timber.DebugTree())`, yang artinya mengaktifkan logger `DebugTree` milik Timber. Logger ini menampilkan informasi log secara lengkap (termasuk nama file, baris, dan method) yang sangat membantu untuk debugging selama pengembangan aplikasi.

Debugger

Debugger di Android Studio adalah tool yang digunakan untuk memeriksa dan memperbaiki kesalahan dalam aplikasi. Dengan menetapkan breakpoint (titik henti), kita bisa menghentikan sementara jalannya program tepat di baris yang ingin dianalisis. Saat aplikasi berhenti di titik itu, kita dapat melihat nilai variabel, urutan eksekusi, dan kondisi aplikasi saat itu. Ini sangat membantu untuk menemukan bug yang tidak terlihat hanya dari membaca kode atau log biasa. Setelah aplikasi berhenti, kita bisa menggunakan fitur `Step Into` untuk masuk ke dalam method yang dipanggil, `Step Over` untuk menjalankan satu baris kode tanpa masuk ke method lain, dan `Step Out` untuk keluar dari method yang sedang dianalisis dan kembali ke bagian kode sebelumnya. Dengan ketiga fitur ini, kita bisa melacak alur eksekusi aplikasi secara detail dan memahami bagaimana data mengalir di dalam program.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



A. Pembahasan

Application class di Android berfungsi sebagai titik awal dari seluruh aplikasi dan hanya dijalankan satu kali selama aplikasi aktif. Class ini biasanya digunakan untuk mengatur konfigurasi global seperti inisialisasi library (misalnya Timber untuk logging atau Retrofit untuk jaringan), pengaturan database, atau menyimpan data yang dibutuhkan di banyak bagian aplikasi. Untuk menggunakannya, buat class baru yang mewarisi `Application`, lalu buat class tersebut di file `AndroidManifest.xml` melalui atribut `android:name`. Dengan cara ini, semua komponen penting bisa disiapkan di satu tempat dan tidak perlu diulang di setiap Activity atau Fragment agar membantu menjaga struktur aplikasi tetap rapi dan efisien.

MODUL 5 : CONNECT TO THE INTERNET

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

presentation/MainActivity.kt

Tabel 42. Source Code presentation/MainActivity.kt Soal 1 Modul 5

1	package com.example.movielist.presentation
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.isSystemInDarkTheme
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.material3.MaterialTheme
9	import androidx.compose.material3.Surface
10	import androidx.compose.material3.Text
11	import androidx.compose.runtime.Composable
12	import androidx.compose.runtime.collectAsState
13	import androidx.compose.runtime.getValue
14	import androidx.compose.ui.Modifier
15	import androidx.lifecycle.viewmodel.compose.viewModel
16	import androidx.navigation.compose.NavHost
17	import androidx.navigation.compose.composable
18	import androidx.navigation.compose.rememberNavController
19	import androidx.navigation.navArgument
20	import androidx.navigation.NavType
21	import com.example.movielist.AppContainer
22	import com.example.movielist.domain.model.Movie
23	import
	com.example.movielist.presentation.ui.screens.detail.MovieDetailScreen
24	import
	com.example.movielist.presentation.ui.screens.list.MovieListScreen
25	import
	com.example.movielist.presentation.ui.screens.list.MovieListViewModel
26	import com.example.movielist.ui.theme.MovieListTheme
27	import com.google.gson.Gson
28	class MainActivity : ComponentActivity() {
29	override fun onCreate(savedInstanceState: Bundle?) {
30	super.onCreate(savedInstanceState)
31	
32	val appContainer =
	AppContainer(applicationContext)
33	
34	setContent {
35	val settingsManager =
	appContainer.settingsManager
36	val isSystemDark = isSystemInDarkTheme()
37	// Mengamati isDarkMode dari
	SettingsManager, dengan initial value dari tema sistem
38	val isDarkMode by
	settingsManager.isDarkMode.collectAsState(initial =

```

39 isSystemDark)
40         MovieListTheme(darkTheme = isDarkMode) {
41             Surface(
42                 modifier = Modifier.fillMaxSize(),
43                 color =
44                 MaterialTheme.colorScheme.background
45             ) {
46                 val navController =
47                 rememberNavController()
48                 val movieListViewModel:
49                 MovieListViewModel = viewModel(
50                     factory =
51                     appContainer.movieListViewModelFactory
52                 )
53                 NavHost(navController =
54                 navController, startDestination = "movie_list") {
55                     composable("movie_list") {
56                         MovieListScreen(
57                             viewModel =
58                             movieListViewModel,
59                             settingsManager =
60                             settingsManager,
61                             navController =
62                             navController
63                         )
64                     }
65                     composable(
66                         route =
67                         "movie_detail/{movieJson}",
68                         arguments =
69                         listOf(navArgument("movieJson") { type =
70                         NavType.StringType })
71                     ) { backStackEntry ->
72                         val movieJson =
73                         backStackEntry.arguments?.getString("movieJson")
74                         val movie =
75                         Gson().fromJson(movieJson, Movie::class.java)
76                         movie?.let {
77                             MovieDetailScreen(movie
78                             = it)
79                         } ?: run {
80                             Text("Error: Movie
81                             details not available.", color =
82                             MaterialTheme.colorScheme.error)
83                         }
84                     }
85                 }
86             }
87         }

```


73	}
74	}
75	}
76	}
77	}

AppContainer.kt

Tabel 43. Source Code AppContainer.kt Soal 1 Modul 5

1	package com.example.movielist
2	
3	import android.content.Context
4	import com.example.movielist.data.local.AppDatabase
5	import
	com.example.movielist.data.remote.api.KtorApiClient
6	import
	com.example.movielist.data.repository.MovieRepositoryImp
7	import
	com.example.movielist.domain.repository.MovieRepository
8	import
	com.example.movielist.domain.usecase.GetMoviesUseCase
9	import
	com.example.movielist.presentation.ui.screens.list.Movie
	ListViewModelFactory
10	import com.example.movielist.util.SettingsManager
11	
12	class AppContainer(private val context: Context) {
13	
14	private val appDatabase: AppDatabase by lazy {
15	AppDatabase.create(context)
16	}
17	
18	private val movieDao by lazy {
19	appDatabase.movieDao()
20	}
21	
22	private val ktorApiClient: KtorApiClient by lazy {
23	KtorApiClient
24	}
25	
26	private val movieRepository: MovieRepository by lazy
27	{
28	MovieRepositoryImpl(ktorApiClient, movieDao)
29	}
30	
31	private val getMoviesUseCase: GetMoviesUseCase by
32	lazy {

33	GetMoviesUseCase(movieRepository)
34	}
35	
36	val movieListViewModelFactory:
	MovieListViewModelFactory by lazy {
37	MovieListViewModelFactory(getMoviesUseCase)
38	}
39	
40	val settingsManager: SettingsManager by lazy {
41	SettingsManager(context)
42	}
43	}

MyApplication.kt

Tabel 44. Source Code MyApplication.kt Soal 1 Modul 5

1	package com.example.movielist
2	
3	import android.app.Application
4	
5	class MyApplication : Application() {
6	lateinit var container: AppContainer
7	
8	override fun onCreate() {
9	super.onCreate()
10	container = AppContainer(this)
11	}
12	}

core/common/NetworkResult.kt

Tabel 45. Source Code core/common/NetworkResult Soal 1 Modul 5

1	package	com.example.movielist.core.common
2		
3	sealed	class NetworkResult<T> (
4	val	data: T? = null,
5	val	message: String? = null
6	)	{
7	class	Success<T>(data: T) : NetworkResult<T>(data)
8	class	Error<T>(message: String, data: T? = null) :
	NetworkResult<T>(data,	message)
9	class	Loading<T> : NetworkResult<T>()
10	}	

data/local/dao/MovieDao.kt

Tabel 46. Source Code data/local/dao/MovieDao.kt Soal 1 Modul 5

1	package	com.example.movielist.data.local.dao
2		
3	import	androidx.room.Dao
4	import	androidx.room.Insert
5	import	androidx.room.OnConflictStrategy
6	import	androidx.room.Query
7	import	androidx.room.Transaction
8	import	com.example.movielist.data.local.model.MovieEntity
9	import	kotlinx.coroutines.flow.Flow
10		
11	@Dao	
12	interface	MovieDao {
13		
14	@Query("SELECT * FROM movies")	
15	fun getMovies():	Flow<List<MovieEntity>>
16		
17	@Insert(onConflict = OnConflictStrategy.REPLACE)	
18	suspend fun insertMovies(movies: List<MovieEntity>)	
19		
20	@Query("DELETE FROM movies")	
21	suspend fun	clearMovies()
22		
23	@Transaction	
24	suspend fun	clearAndInsertMovies(movies:
25	List<MovieEntity>)	{
26	clearMovies()	
27	insertMovies(movies)	
28	}	}

data/local/model/MovieEntity.kt

Tabel 47. Source Code data/local/model/MovieEntity.kt Soal 1 Modul 5

1	package	com.example.movielist.data.local.model
2		
3	import	androidx.room.Entity
4	import	androidx.room.PrimaryKey
5		
6	@Entity(tableName = "movies")	
7	data class MovieEntity(	
8	@PrimaryKey val id: Int,	
9	val title: String,	
10	val overview: String,	
11	val posterPath: String,	
12	val backdropPath: String,	
13	val releaseDate: String,	

14	val voteAverage: Double,
15	val popularity: Double
16	)

Data/local/AppDatabase.kt

Tabel 48. Source Code data/local/AppDatabase.kt Soal 1 Modul 5

1	package com.example.movielist.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	import com.example.movielist.data.local.dao.MovieDao
8	import
	com.example.movielist.data.local.model.MovieEntity
9	
10	@Database(entities = [MovieEntity::class], version = 1,
	exportSchema = false)
11	abstract class AppDatabase : RoomDatabase() {
12	abstract fun movieDao(): MovieDao
13	
14	companion object {
15	@Volatile
16	private var INSTANCE: AppDatabase? = null
17	
18	fun create(context: Context): AppDatabase {
19	return INSTANCE ?: synchronized(this) {
20	val instance = Room.databaseBuilder(
21	context.applicationContext,
22	AppDatabase::class.java,
23	"movie_database"
24	)
25	.fallbackToDestructiveMigration()
26	.build()
27	INSTANCE = instance
28	instance
29	}
30	}
31	}
32	}

Data/mapper/MovieMapper.kt

Tabel 49. Source Code Data/mapper/MovieMapper.kt Soal 1 Modul 5

1	package com.example.movielist.data.mapper
2	

3	import
	com.example.movielist.data.local.model.MovieEntity
4	import
	com.example.movielist.data.remote.model.MovieResultDto
5	import com.example.movielist.domain.model.Movie
6	
7	fun MovieResultDto.toEntity(): MovieEntity {
8	return MovieEntity(
9	id = id ?: 0,
10	title = title.orEmpty(),
11	overview = overview.orEmpty(),
12	posterPath = posterPath.orEmpty(),
13	backdropPath = backdropPath.orEmpty(),
14	releaseDate = releaseDate.orEmpty(),
15	voteAverage = voteAverage ?: 0.0,
16	popularity = popularity ?: 0.0
17	)
18	}
19	
20	fun MovieEntity.toDomain(): Movie {
21	return Movie(
22	id = this.id,
23	title = this.title,
24	overview = this.overview,
25	posterPath = this.posterPath,
26	backdropPath = this.backdropPath,
27	releaseDate = this.releaseDate,
28	voteAverage = this.voteAverage
29	)
30	}

Data/remote/api/ApiConstants.kt

Tabel 50. Source Code Data/remote/api/ApiConstants.kt Soal 1 Modul 5

1	package com.example.movielist.data.remote.api
2	
3	object ApiConstants {
4	const val BASE_URL =
5	"https://api.themoviedb.org/3/"
	const val API_KEY = "api key saya, disamarkan di
	laprak"
6	}

Data/remote/api/KtorApiClient.kt

Tabel 51. Source Code data/remote/api/KtroApiClient.kt Soal 1 Modul 5

1	package com.example.movielist.data.remote.api
2	
3	import com.example.movielist.data.remote.model.MovieResponseDt o
4	import io.ktor.client.*
5	import io.ktor.client.call.*
6	import io.ktor.client.engine.android.*
7	import io.ktor.client.plugins.contentnegotiation.*
8	import io.ktor.client.plugins.logging.*
9	import io.ktor.client.request.*
10	import io.ktor.serialization.kotlinx.json.json
11	import kotlinx.serialization.json.Json
12	
13	object KtorApiClient {
14	val client = HttpClient(Android) {
15	install(Logging) {
16	level = LogLevel.ALL
17	}
18	install(ContentNegotiation) {
19	json(Json {
20	ignoreUnknownKeys = true
21	isLenient = true
22	prettyPrint = true
23	})
24	}
25	}
26	
27	suspend fun getPopularMovies(): MovieResponseDto {
28	return client.get(ApiConstants.BASE_URL + "movie/popular") {
29	parameter("api_key", ApiConstants.API_KEY)
30	}.body()
31	}
32	}

Data/remote/model/MovieDto.kt

Tabel 52. Source Code data/remote/model/MovieDto.kt Soal 1 Modul 5

1	package com.example.movielist.data.remote.model
2	
3	import android.annotation.SuppressLint
4	import kotlinx.serialization.SerialName
5	import kotlinx.serialization.Serializable
6	
7	@SuppressLint("UnsafeOptInUsageError")

8	@Serializable
9	data class MovieResponseDto(
10	val page: Int?,
11	val results: List<MovieResultDto>?,
12	@SerializedName("total_pages")
13	val totalPages: Int?,
14	@SerializedName("total_results")
15	val totalResults: Int?
16	)
17	
18	@SuppressLint("UnsafeOptInUsageError")
19	@Serializable
20	data class MovieResultDto(
21	val id: Int?,
22	val title: String?,
23	val overview: String?,
24	@SerializedName("poster_path")
25	val posterPath: String?,
26	@SerializedName("release_date")
27	val releaseDate: String?,
28	@SerializedName("vote_average")
29	val voteAverage: Double?,
30	@SerializedName("backdrop_path")
31	val backdropPath: String?,
32	val popularity: Double?
33	)

Data/repository/MovieRepositoryImpl.kt

Tabel 53. Source Code data/repository/MovieRepositoryImpl.kt Soal 1 Modul 5

1	package com.example.movielist.data.repository
2	
3	import com.example.movielist.core.common.NetworkResult
4	import com.example.movielist.data.local.dao.MovieDao
5	import com.example.movielist.data.mapper.toDomain
6	import com.example.movielist.data.mapper.toEntity
7	import
	com.example.movielist.data.remote.api.KtorApiClient
8	import
	com.example.movielist.data.remote.model.MovieResponseDt
	o
9	import com.example.movielist.domain.model.Movie
10	import
	com.example.movielist.domain.repository.MovieRepository
11	import kotlinx.coroutines.flow.Flow
12	import kotlinx.coroutines.flow.map
13	import java.io.IOException
14	

15	class MovieRepositoryImpl(
16	private val apiClient: KtorApiClient,
17	private val movieDao: MovieDao
18	) : MovieRepository {
19	
20	override fun getMovies(): Flow<List<Movie>> {
21	return movieDao.getMovies().map { entities ->
22	entities.map { it .toDomain() }
23	}
24	}
25	
26	override suspend fun refreshMovies():
27	NetworkResult<Unit> {
28	return try {
29	val response: MovieResponseDto =
30	apiClient.getPopularMovies()
31	val movieEntities = response.results?.map {
32	it .toEntity() } ?: emptyList()
33	movieDao.clearAndInsertMovies(movieEntities)
34	NetworkResult.Success(Unit)
35	} catch (e: IOException) {
36	NetworkResult.Error("Gagal menyambung ke
37	server. Periksa koneksi internet Anda.")
38	} catch (e: Exception) {
39	NetworkResult.Error("Terjadi error:
	\${e.message}")
	}
	}
	}

Domain/model/Movie.kt

Tabel 54. Source Code domain/model/Movie.kt Soal 1 Modul 5

1	package com.example.movielist.domain.model
2	
3	data class Movie(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterPath: String?,
8	val backdropPath: String?,
9	val voteAverage: Double,
10	val releaseDate: String?
11	) {
12	fun getPosterUrl(): String {
13	return if (posterPath != null &&
	posterPath.isNotEmpty())

14	}
15	
16	fun getBackdropUrl(): String {
17	return if (backdropPath != null && backdropPath.isNotEmpty()) "https://image.tmdb.org/t/p/w780\$backdropPath" else ""
18	}
19	
20	fun getTmdbUrl(): String {
21	return "https://www.themoviedb.org/movie/\$id"
22	}
23	}

Domain/repository/MovieRepository.kt

Tabel 55. Source Code domain/repository/MovieRepository.kt Soal 1 Modul 5

1	package com.example.movielist.domain.repository
2	
3	import com.example.movielist.core.common.NetworkResult
4	import com.example.movielist.domain.model.Movie
5	import kotlinx.coroutines.flow.Flow
6	
7	interface MovieRepository {
8	fun getMovies(): Flow<List<Movie>>
9	suspend fun refreshMovies(): NetworkResult<Unit>
10	}

Domain/usecase/GetMoviesUseCase.kt

Tabel 56. Source Code Domain/usecase/GetMovieUseCase.kt Soal 1 Modul 5

1	package com.example.movielist.domain.usecase
2	
3	import com.example.movielist.core.common.NetworkResult
4	import com.example.movielist.domain.model.Movie
5	import com.example.movielist.domain.repository.MovieRepository
	import kotlinx.coroutines.flow.Flow
6	
7	class GetMoviesUseCase(private val movieRepository:
8	MovieRepository) {
9	fun getMoviesFromDb(): Flow<List<Movie>> = movieRepository.getMovies()
10	
11	suspend fun refreshMoviesFromApi(): NetworkResult<Unit> = movieRepository.refreshMovies()
12	}

Presentation/ui/component/MovieCard.kt

Tabel 57. Source Code presentation/ui/component/MovieCard.kt Soal 1 Modul 5

1	package com.example.movielist.presentation.ui.component
2	
3	import androidx.compose.foundation.clickable
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Alignment
9	import androidx.compose.ui.Modifier
10	import androidx.compose.ui.draw.clip
11	import androidx.compose.ui.layout.ContentScale
12	import androidx.compose.ui.platform.LocalContext
13	import androidx.compose.ui.text.font.FontWeight
14	import androidx.compose.ui.text.style.TextOverflow
15	import androidx.compose.ui.unit.dp
16	import coil.compose.AsyncImage
17	import coil.request.ImageRequest
18	import com.example.movielist.domain.model.Movie
19	
20	@OptIn(ExperimentalMaterial3Api::class)
21	@Composable
22	fun MovieCard(
23	movie: Movie,
24	modifier: Modifier = Modifier,
25	onMovieClick: (Movie) -> Unit,
26	onTmdbClick: (String) -> Unit
27	) {
28	val context = LocalContext.current
29	
30	Card(
31	modifier = modifier
32	.fillMaxWidth()
33	.clickable { onMovieClick(movie) }
34	.padding(8.dp),
35	shape = RoundedCornerShape(12.dp),
36	elevation =
	CardDefaults.cardElevation(defaultElevation = 4.dp)
37	) {
38	Row(
39	modifier = Modifier
40	.fillMaxWidth()
41	.padding(12.dp)
42	) {

```

43         AsyncImage(
44             model =
ImageRequest.Builder(LocalContext.current)
45                 .data(movie.getPosterUrl())
46                 .crossfade(true)
47                 .build(),
48             contentDescription = movie.title,
49             modifier = Modifier
50                 .size(width = 100.dp, height =
150.dp)
51                 .clip(RoundedCornerShape(8.dp)),
52             contentScale = ContentScale.Crop
53         )
54
55         Spacer(modifier = Modifier.width(12.dp))
56
57         Column(
58             modifier = Modifier
59                 .weight(1f)
60                 .align(Alignment.CenterVertically)
61         ) {
62             Text(
63                 text = movie.title,
64                 style =
MaterialTheme.typography.titleMedium,
65                 fontWeight = FontWeight.Bold,
66                 maxLines = 2,
67                 overflow = TextOverflow.Ellipsis
68             )
69             Spacer(modifier =
Modifier.height(4.dp))
70
71             Text(
72                 text = "Rating:
${String.format("%.1f", movie.voteAverage)}/10",
73                 style =
MaterialTheme.typography.bodySmall
74             )
75             Spacer(modifier =
Modifier.height(8.dp))
76
77             Row(
78                 modifier = Modifier.fillMaxWidth(),
79                 horizontalArrangement =
Arrangement.End
80             ) {
81                 Button(
82                     onClick = {
onTmdbClick(movie.getTmdbUrl()) },

```

83	<code>modifier = Modifier.padding(end</code>
	<code>= 8.dp)</code>
84	<code>) {</code>
85	<code>Text("TMDB")</code>
86	<code>}</code>
87	
88	<code>Button(</code>
89	<code>onClick = { onMovieClick(movie)</code>
90	<code>}</code>
91	<code>) {</code>
92	<code>Text("Detail")</code>
93	<code>}</code>
94	<code>}</code>
95	<code>}</code>
96	<code>}</code>
97	<code>}</code>
98	<code>}</code>

Presentation/ui/navigation/Screen.kt

Tabel 58. Source Code presentation/ui/navigation/Screen.kt Soal 1 Modul 5

1	<code>package</code>
	<code>com.example.movielist.presentation.ui.navigation</code>
2	
3	<code>sealed class Screen(val route: String) {</code>
4	<code>data object MovieList : Screen("movie_list")</code>
5	<code>data object MovieDetail : Screen("movie_detail")</code>
6	<code>}</code>

Presentation/ui/screens/detail/MovieDetailScreen.kt

Tabel 59. Source Code presentation/ui/screens/detail/MovieDetailScreen.kt Soal 1 Modul 5

1	<code>package</code>
2	<code>com.example.movielist.presentation.ui.screens.detail</code>
3	<code>import androidx.compose.foundation.layout.*</code>
4	<code>import androidx.compose.foundation.rememberScrollState</code>
5	<code>import androidx.compose.foundation.verticalScroll</code>
6	<code>import androidx.compose.material.icons.Icons</code>
7	<code>import androidx.compose.material.icons.filled.Star</code>
8	<code>import androidx.compose.material3.*</code>
9	<code>import androidx.compose.runtime.Composable</code>
10	<code>import androidx.compose.ui.Alignment</code>
11	<code>import androidx.compose.ui.Modifier</code>
12	<code>import androidx.compose.ui.draw.clip</code>
13	<code>import androidx.compose.ui.graphics.Color</code>
14	<code>import androidx.compose.ui.layout.ContentScale</code>

```

15 import androidx.compose.ui.platform.LocalContext
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import coil.compose.AsyncImage
19 import coil.request.ImageRequest
20 import com.example.movielist.domain.model.Movie
21
22 @Composable
23 fun MovieDetailScreen(movie: Movie) {
24     Column(
25         modifier = Modifier
26             .fillMaxSize()
27             .padding(16.dp)
28             .verticalScroll(rememberScrollState())
29     ) {
30         AsyncImage(
31             model =
32             ImageRequest.Builder(LocalContext.current)
33                 .data(movie.getBackdropUrl())
34                 .crossfade(true)
35                 .build(),
36             contentDescription = movie.title,
37             modifier = Modifier
38                 .fillMaxWidth()
39                 .height(220.dp)
40                 .clip(MaterialTheme.shapes.medium),
41             contentScale = ContentScale.Crop
42         )
43         Spacer(modifier = Modifier.height(16.dp))
44         Text(
45             text = movie.title,
46             style =
47             MaterialTheme.typography.headlineMedium,
48             fontWeight = FontWeight.Bold
49         )
50         Spacer(modifier = Modifier.height(8.dp))
51         Row(verticalAlignment =
52             Alignment.CenterVertically) {
53             Icon(
54                 imageVector = Icons.Filled.Star,
55                 contentDescription = "Rating",
56                 tint = Color(0xFFFD835),
57                 modifier = Modifier.size(24.dp)
58             )
59             Spacer(modifier = Modifier.width(4.dp))
60             Text(
61                 text = "${String.format("%.1f",

```

62	movie.voteAverage)}/10", style = MaterialTheme.typography.titleMedium
63	)
64	}
65	Spacer(modifier = Modifier.height(8.dp))
66	
67	movie.releaseDate?.let {
68	Text(text = "Release Date: \$it", style = MaterialTheme.typography.bodyMedium)
69	Spacer(modifier = Modifier.height(8.dp))
70	}
71	Text(text = movie.overview, style = MaterialTheme.typography.bodyLarge
72	)
73	}
74	}
75	}
76	}

Presentation/ui/screens/list/MovieListScreen.kt

Tabel 60. Source Code presentation/ui/screens/list/MovieListScreen.kt Soal 1 Modul 5

1	package
2	com.example.movielist.presentation.ui.screens.list
3	
4	import android.content.Intent
5	import android.net.Uri
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.lazy.LazyColumn
8	import androidx.compose.foundation.lazy.grid.GridCells
9	import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
10	import androidx.compose.foundation.lazy.grid.items
11	import androidx.compose.foundation.lazy.items
12	import androidx.compose.material.icons.Icons
13	import androidx.compose.runtime.rememberCoroutineScope
14	import androidx.compose.material.icons.automirrored.filled.Lis t
15	import androidx.compose.material.icons.filled.CheckCircle
16	import androidx.compose.material.icons.filled.Menu
17	import androidx.compose.material3.*
18	import androidx.compose.runtime.Composable
19	import androidx.compose.runtime.collectAsState
20	import androidx.compose.runtime.getValue
21	import androidx.compose.ui.Alignment

```

22 import androidx.compose.ui.Modifier
23 import androidx.compose.ui.platform.LocalContext
24 import androidx.compose.ui.unit.dp
25 import androidx.navigation.NavController
26 import
  com.example.movielist.presentation.ui.component.MovieCa
  rd
27 import com.example.movielist.util.LayoutType
28 import com.example.movielist.util.SettingsManager
29 import com.google.gson.Gson
30 import kotlinx.coroutines.launch
31
32 @OptIn(ExperimentalMaterial3Api::class)
33 @Composable
34 fun MovieListScreen(
35     viewModel: MovieListViewModel,
36     settingsManager: SettingsManager,
37     navController: NavController
38 ) {
39     val uiState by viewModel.uiState.collectAsState()
40     val layoutType by
  settingsManager.layoutType.collectAsState(initial =
  LayoutType.LIST)
41     val isDarkMode by
  settingsManager.isDarkMode.collectAsState(initial =
  false)
42     val context = LocalContext.current
43     val scope = rememberCoroutineScope()
44
45     Scaffold(
46         topBar = {
47             TopAppBar(
48                 title = { Text("Popular Movies") },
49                 actions = {
50                     IconButton(onClick = {
51                         scope.launch {
52                             settingsManager.toggleDarkMode()
53                         }
54                     }) {
55                         Icon(
56                             imageVector = if
  (isDarkMode) Icons.Filled.CheckCircle else
  Icons.Filled.CheckCircle,
57                             contentDescription =
  "Toggle Dark/Light Mode"
58                         )
59                     }
60

```

```

61 IconButton(onClick = {
62     scope.launch {
63         val newLayout = if
        (layoutType == LayoutType.LIST) LayoutType.GRID else
        LayoutType.LIST
64
65 settingsManager.setLayoutType(newLayout)
66     }
67 }) {
68     Icon(
69         imageVector = if
        (layoutType == LayoutType.LIST) Icons.Filled.Menu else
        Icons.AutoMirrored.Filled.List,
70         contentDescription =
        "Change Layout"
71     )
72 }
73 }
74 )
75 }
76 ) { paddingValues ->
77     Box(
78         modifier = Modifier
79             .fillMaxSize()
80             .padding(paddingValues)
81     ) {
82         if (uiState.isLoading &&
83             uiState.movies.isEmpty()) {
84             CircularProgressIndicator(modifier =
85                 Modifier.align(Alignment.Center))
86         } else if (uiState.errorMessage != null &&
87             uiState.movies.isEmpty()) {
88             Text(
89                 text = uiState.errorMessage ?:
90                 "Unknown Error",
91                 color =
92                 MaterialTheme.colorScheme.error,
93                 modifier =
94                 Modifier.align(Alignment.Center)
95             )
96         } else {
97             if (layoutType == LayoutType.LIST) {
98                 LazyColumn(
99                     modifier =
100                     Modifier.fillMaxSize(),
101                     contentPadding =
102                     PaddingValues(16.dp),
103                     verticalArrangement =
104                     Arrangement.spacedBy(16.dp)

```



```

96         ) {
97             items(uiState.movies) { movie -
98                 >
99                     MovieCard(
100                         movie = movie,
101                         modifier =
102                         Modifier.fillMaxWidth(),
103                         onMovieClick = {
104                             selectedMovie ->
105                                 val movieJson =
106                                 Uri.encode(Gson().toJson(selectedMovie))
107                                 navController.navigate("movie_detail/$movieJson")
108                                 },
109                                 onTmdbClick = { tmdbUrl
110                                     ->
111                                         val intent =
112                                         Intent(Intent.ACTION_VIEW, Uri.parse(tmdbUrl))
113                                         context.startActivity(intent)
114                                     }
115                                 }
116                             }
117                         } else {
118                             LazyVerticalGrid(
119                                 columns = GridCells.Fixed(2),
120                                 modifier =
121                                 Modifier.fillMaxSize(),
122                                 contentPadding =
123                                 PaddingValues(16.dp),
124                                 verticalArrangement =
125                                 Arrangement.spacedBy(16.dp),
126                                 horizontalArrangement =
127                                 Arrangement.spacedBy(16.dp)
128                             ) {
129                                 items(uiState.movies) { movie -
130                                     >
131                                         MovieCard(
132                                             movie = movie,
133                                             onMovieClick = {
134                                                 selectedMovie ->
135                                                     val movieJson =
136                                                     Uri.encode(Gson().toJson(selectedMovie))
137                                                     navController.navigate("movie_detail/$movieJson")
138                                                     },
139                                                     onTmdbClick = { tmdbUrl

```

128	val intent =
129	Intent(Intent.ACTION_VIEW, Uri.parse(tmdbUrl))
130	context.startActivity(intent)
131	}
132	)
133	}
134	}
135	}
136	}
137	}
138	}
139	}

Presentation/ui/screens/list/MovieListViewModel.kt

Tabel 61. Source Code presentation/ui/screens/list/MovieListViewModel.kt Soal 1 Modul 5

1	package
2	com.example.movieslist.presentation.ui.screens.list
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.movieslist.core.common.NetworkResult
6	import com.example.movieslist.domain.model.Movie
7	import
8	com.example.movieslist.domain.usecase.GetMoviesUseCase
9	import kotlinx.coroutines.flow.MutableStateFlow
10	import kotlinx.coroutines.flow.StateFlow
11	import kotlinx.coroutines.flow.asStateFlow
12	import kotlinx.coroutines.flow.catch
13	import kotlinx.coroutines.flow.onStart
14	import kotlinx.coroutines.launch
15	data class MovieListUiState(
16	val movies: List<Movie> = emptyList(),
17	val isLoading: Boolean = false,
18	val errorMessage: String? = null
19	)
20	
21	class MovieListViewModel(
22	private val getMoviesUseCase: GetMoviesUseCase
23	) : ViewModel() {
24	
25	private val _uiState =
26	MutableStateFlow(MovieListUiState())
	val uiState: StateFlow<MovieListUiState> =
	_uiState.asStateFlow()

```

27
28     init {
29         observeMovies()
30         refreshData()
31     }
32
33     private fun observeMovies() {
34         viewModelScope.launch {
35             getMoviesUseCase.getMoviesFromDb()
36                 .onStart { _uiState.value =
37 _uiState.value.copy(isLoading = true) }
38                 .catch { e ->
39 _uiState.value.value =
40 _uiState.value.copy(
41                     isLoading = false,
42                     errorMessage = "Gagal memuat
43 data dari database: ${e.message}"
44                 )
45             }
46             .collect { movies ->
47 _uiState.value.value =
48 _uiState.value.copy(
49                     isLoading = false,
50                     movies = movies
51                 )
52             }
53         }
54     }
55
56     fun refreshData() {
57         viewModelScope.launch {
58             _uiState.value =
59 _uiState.value.copy(isLoading = true, errorMessage =
60 null)
61             when (val result =
62 getMoviesUseCase.refreshMoviesFromApi()) {
63                 is NetworkResult.Success -> {
64                     _uiState.value =
65 _uiState.value.copy(isLoading = false)
66                 }
67                 is NetworkResult.Error -> {
68                     _uiState.value =
69 _uiState.value.copy(
70                     isLoading = false,
71                     errorMessage = result.message
72                 )
73             }
74             else -> {}
75         }
76     }

```

	<pre> } } }</pre>
--	-------------------------------

Presentation/ui/screens/list/MovieListViewModelFactory.kt

Tabel 62. Source Code presentation/ui/screens/list/MovieListViewModelFactory.kt Soal 1 Modul 5

1	package
2	com.example.movielist.presentation.ui.screens.list
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
6	com.example.movielist.domain.usecase.GetMoviesUseCase
7	
8	class MovieListViewModelFactory(
9	private val getMoviesUseCase: GetMoviesUseCase
10	) : ViewModelProvider.Factory {
11	
12	override fun <T : ViewModel> create(modelClass:
13	Class<T>): T {
14	if
15	(modelClass.isAssignableFrom(MovieListViewModel::class.
16	java)) {
17	@Suppress("UNCHECKED_CAST")
18	return MovieListViewModel(getMoviesUseCase)
19	as T
20	}
21	throw IllegalArgumentException("Unknown
	ViewModel class")
	}
	}

B. Output Program



Warrior Queen

★ 3.4/10

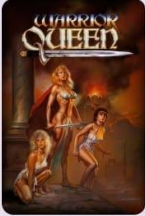
Release Date: 1987-01-26

In ancient Pompeii, slaves are bought and sold for household chores and sex. A mysterious queen moves among the elite, meantime secretly helping the slaves to escape. Eventually her life is also in jeopardy, and as the volcano erupts she and the slaves attempt to escape while being chased by the military.

Gambar 46. Tampilan Layar Detail Soal 1 Modul 5

Popular Movies





Warrior Queen

Rating: 3.4/10

[TMDB](#)[Detail](#)



Thor: Ragnarok

Rating: 7.6/10

[TMDB](#)[Detail](#)



The Accountant

Rating: 7.1/10

[TMDB](#)[Detail](#)

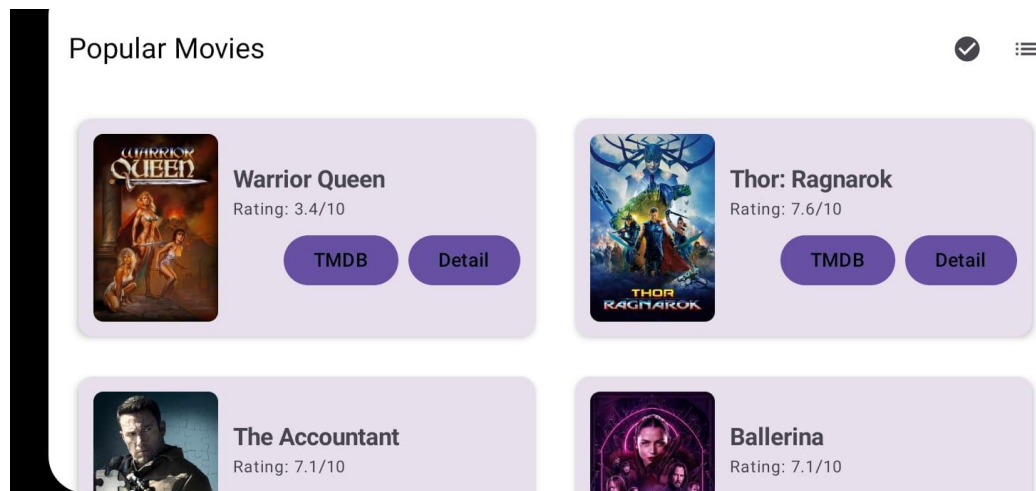


Ballerina

Rating: 7.1/10

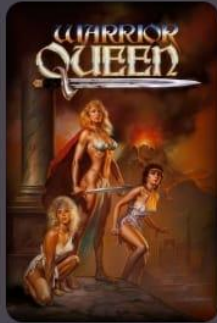
[TMDB](#)[Detail](#)

Gambar 47. Tampilan Awal Soal 1 Modul 5



Gambar 48. Tampilan Awal Landscape Soal 1 Modul 5

Popular Movies



Warrior Queen

Rating: 3.4/10

[TMDB](#)

[Detail](#)

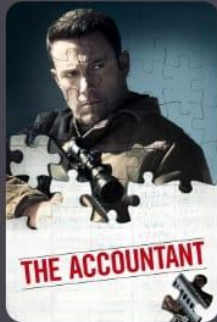


Thor: Ragnarok

Rating: 7.6/10

[TMDB](#)

[Detail](#)



The Accountant

Rating: 7.1/10

[TMDB](#)

[Detail](#)



Ballerina

Rating: 7.1/10

C. Pembahasan

Presentation/MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–27], diimport berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. `ComponentActivity` diimport untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. `setContent` digunakan untuk menetapkan tampilan UI aplikasi. `isSystemInDarkTheme` digunakan untuk mendeteksi apakah tema gelap sistem sedang aktif. Selain itu, berbagai fungsi dan class seperti `Surface`, `Text`, `MaterialTheme`, `Modifier`, `ViewModel`, `collectAsState`, `NavHost`, `composable`, `rememberNavController`, `navArgument`, dan `NavType` diimport untuk membangun dan mengelola UI serta navigasi menggunakan Jetpack Compose. `AppContainer` digunakan untuk mengelola dependensi aplikasi secara global. `Movie`, `MovieDetailScreen`, `MovieListScreen`, `MovieListViewModel`, dan `MovieListTheme` diimport sebagai entitas dan composable utama aplikasi yang menampilkan daftar dan detail film, serta tema aplikasi. `Gson` diimport untuk keperluan serialisasi dan deserialisasi objek JSON. Pada baris [28], `MainActivity` dideklarasikan sebagai turunan dari `ComponentActivity`. Class ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai aktivitas utama, `MainActivity` bertanggung jawab untuk mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [29], dalam method `onCreate()`, kita memanggil `super.onCreate(savedInstanceState)` untuk memastikan siklus hidup activity dijalankan sesuai dengan standar.

Pada baris [32], variabel `appContainer` dideklarasikan dan diinisialisasi dengan membuat instance `AppContainer` yang memerlukan `applicationContext`. `AppContainer` ini akan berfungsi sebagai wadah untuk semua dependensi aplikasi, seperti repositori, use case, dan view model factory, yang kemudian akan diakses di seluruh bagian aplikasi.

Pada baris [34–72], `setContent` digunakan untuk menetapkan tampilan UI aplikasi menggunakan Jetpack Compose.

Pada baris [35], variabel `settingsManager` dideklarasikan dan diinisialisasi dengan mengakses `appContainer.settingsManager`, yang menyediakan fungsionalitas untuk mengelola pengaturan aplikasi, seperti tema dan jenis tata letak.

Pada baris [36], variabel `isSystemDark` dideklarasikan dan diinisialisasi dengan memanggil `isSystemInDarkTheme()`, yang mengembalikan `true` jika tema gelap aktif di sistem operasi perangkat.

Pada baris [37], variabel `isDarkMode` dideklarasikan dan diobservasi dari `settingsManager.isDarkMode.collectAsState()`, dengan nilai awal yang berasal dari `isSystemDark`. Variabel ini akan secara reaktif memperbarui UI ketika mode gelap diubah melalui pengaturan aplikasi atau sistem.

Pada baris [39], `MovieListTheme` digunakan sebagai tema utama aplikasi, dengan properti `darkTheme` diatur berdasarkan nilai `isDarkMode` yang diobservasi, memastikan konsistensi gaya visual aplikasi sesuai mode gelap atau terang.

Pada baris [40–67], `Surface` dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI. `modifier = Modifier.fillMaxSize()` membuat `Surface` mengambil seluruh ruang yang tersedia di layar, sementara `color = MaterialTheme.colorScheme.background` menetapkan warna latar belakang sesuai skema warna tema aplikasi.

Pada baris [44], variabel `navController` dideklarasikan dan diinisialisasi dengan `rememberNavController()`, yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. `navController` digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar film ke tampilan detail film.

Pada baris [45–47], variabel `movieListViewModel` dideklarasikan dan diinisialisasi menggunakan fungsi `viewModel()`, yang mengambil instance dari `MovieListViewModel` dengan factory yang disediakan oleh `appContainer.movieListViewModelFactory`. `ViewModel` ini digunakan untuk menyimpan dan mengelola data serta logika bisnis yang diperlukan oleh UI daftar film.

Pada baris [49–65], NavHost dideklarasikan untuk mengelola navigasi antar tampilan. startDestination ditetapkan ke "movie_list", yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar film. Di dalam NavHost, ada dua rute yang didefinisikan. Pada baris [50–56], rute "movie_list" akan memanggil MovieListScreen, dengan viewModel dan settingsManager yang relevan, serta navController untuk navigasi. Pada baris [57–65], rute "movie_detail/{movieJson}" didefinisikan untuk menampilkan detail film. Rute ini mengambil argumen movieJson dari URL. movieJson kemudian dide-serialisasi menjadi objek Movie menggunakan Gson. Jika objek Movie tidak null, MovieDetailScreen akan dipanggil untuk menampilkan detail film tersebut. Jika data film tidak ditemukan, maka akan ditampilkan pesan "Error: Movie details not available." menggunakan Text dengan warna error dari tema.

AppContainer.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–11], diimport berbagai komponen yang diperlukan untuk manajemen dependensi aplikasi. Context diimport untuk menyediakan akses ke sumber daya dan layanan sistem. AppDatabase diimport untuk berinteraksi dengan database lokal. KtorApiClient diimport untuk menangani komunikasi dengan API remote. MovieRepositoryImpl dan MovieRepository diimport untuk mengimplementasikan dan mendefinisikan antarmuka repositori data film. GetMoviesUseCase diimport sebagai use case untuk mendapatkan data film. MovieListViewModelFactory diimport sebagai factory untuk membuat instance MovieListViewModel. SettingsManager diimport untuk mengelola pengaturan aplikasi.

Pada baris [12], AppContainer dideklarasikan sebagai sebuah kelas yang akan bertanggung jawab untuk menyediakan semua dependensi yang dibutuhkan oleh aplikasi, mengimplementasikan pola Dependency Injection. Konstruktornya menerima sebuah Context sebagai parameter, yang dibutuhkan untuk inisialisasi beberapa dependensi.

Pada baris [14–16], variabel appDatabase dideklarasikan menggunakan by lazy untuk memastikan inisialisasi yang malas (lazy initialization). AppDatabase.create(context)

dipanggil untuk membuat instance database Room, yang digunakan untuk menyimpan data film secara lokal.

Pada baris [18–20], variabel `movieDao` dideklarasikan menggunakan `by lazy`. Ini mengakses Movie Data Access Object (DAO) dari `appDatabase`, yang menyediakan metode untuk berinteraksi dengan tabel film di database.

Pada baris [22–24], variabel `ktorApiClient` dideklarasikan menggunakan `by lazy`. Ini mengacu langsung pada objek `KtorApiClient`, yang bertanggung jawab untuk melakukan panggilan jaringan ke API TheMovieDB.

Pada baris [26–28], variabel `movieRepository` dideklarasikan menggunakan `by lazy`. Ini membuat instance `MovieRepositoryImpl`, yang merupakan implementasi dari `MovieRepository`. Repositori ini bertugas mengelola sumber data, baik dari API (`ktorApiClient`) maupun dari database lokal (`movieDao`).

Pada baris [30–32], variabel `getMoviesUseCase` dideklarasikan menggunakan `by lazy`. Ini membuat instance `GetMoviesUseCase`, yang menerima `movieRepository` sebagai dependensinya. Use case ini mendefinisikan logika bisnis untuk mendapatkan daftar film.

Pada baris [34–36], variabel `movieListViewModelFactory` dideklarasikan menggunakan `by lazy`. Ini membuat instance `MovieListViewModelFactory`, yang menerima `getMoviesUseCase` sebagai dependensinya. Factory ini digunakan oleh Jetpack Compose untuk membuat instance `MovieListViewModel` dengan dependensi yang diperlukan.

Pada baris [38–40], variabel `settingsManager` dideklarasikan menggunakan `by lazy`. Ini membuat instance `SettingsManager`, yang menerima `context` sebagai dependensinya. `SettingsManager` bertanggung jawab untuk mengelola preferensi pengguna seperti tema aplikasi dan jenis tata letak (`grid/list`).

MyApplication.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport `Application`, sebuah kelas dasar yang menyediakan fungsionalitas inti untuk mengelola keadaan aplikasi secara global dan siklus hidupnya.

Pada baris [5], `MyApplication` dideklarasikan sebagai sebuah kelas yang merupakan turunan dari `Application`. Class ini berfungsi sebagai titik masuk khusus aplikasi saat pertama kali diluncurkan, memungkinkan inisialisasi komponen yang perlu ada sepanjang siklus hidup aplikasi.

Pada baris [6], variabel `container` dideklarasikan dengan kata kunci `lateinit`. Ini berarti `container` adalah sebuah instance dari `AppContainer` yang akan diinisialisasi nanti, memastikan bahwa objek `AppContainer` akan tersedia dan dapat diakses di seluruh bagian aplikasi.

Pada baris [8–11], dalam method `onCreate()`, `super.onCreate()` dipanggil untuk menjalankan inisialisasi standar dari kelas `Application` induk. Kemudian, pada baris [10], `container` diinisialisasi dengan membuat instance baru dari `AppContainer`, yang menerima `this` (instance `MyApplication` itu sendiri sebagai `Context`) sebagai argumen. Ini memastikan bahwa `AppContainer` dibuat satu kali saat aplikasi dimulai dan dapat diakses sebagai singleton oleh komponen lain di seluruh aplikasi.

Core/common/NetworkResult.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan kelas `NetworkResult` sebagai sealed class. Sealed class digunakan untuk merepresentasikan hierarki kelas terbatas, di mana semua subclass diketahui pada waktu kompilasi. Ini sangat berguna untuk merepresentasikan status yang berbeda (misalnya, sukses, error, loading) dari suatu operasi jaringan. Kelas ini memiliki dua properti opsional: `data` (bertipe `T`, yang bisa null) untuk menyimpan hasil sukses, dan `message` (bertipe `String`, yang bisa null) untuk menyimpan pesan error.

Pada baris [7], dideklarasikan `Success` sebagai subclass dari `NetworkResult`. Kelas `Success` merepresentasikan keadaan ketika operasi jaringan berhasil. Ia memiliki satu parameter `data` bertipe `T`, yang harus berisi hasil operasi yang sukses.

Pada baris [8], dideklarasikan `Error` sebagai subclass dari `NetworkResult`. Kelas `Error` merepresentasikan keadaan ketika operasi jaringan gagal. Ia memiliki dua parameter: `message` bertipe `String` yang wajib untuk menjelaskan error, dan `data` opsional bertipe

T yang bisa null (berguna jika ada data parsial atau cache yang bisa ditampilkan meskipun ada error).

Pada baris [9], dideklarasikan Loading sebagai subclass dari NetworkResult. Kelas Loading merepresentasikan keadaan ketika operasi jaringan sedang berlangsung. Kelas ini tidak memiliki properti karena pada tahap ini, data atau pesan error belum tersedia.

Data/local/dao/MovieDao.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–9], diimport berbagai komponen yang diperlukan dari pustaka Room dan Kotlin Coroutines. @Dao diimport untuk menandai antarmuka sebagai Data Access Object Room, yang bertanggung jawab untuk mendefinisikan metode interaksi database. @Insert, @OnConflictStrategy, @Query, dan @Transaction adalah anotasi Room yang digunakan untuk menentukan jenis operasi database. MovieEntity diimport sebagai representasi objek film di database. Flow diimport dari kotlinox.coroutines.flow untuk mendukung pengumpulan data asinkron.

Pada baris [11], antarmuka MovieDao dideklarasikan dan dianotasi dengan @Dao, menandakan bahwa ini adalah Data Access Object yang digunakan oleh Room untuk berinteraksi dengan database.

Pada baris [14–15], fungsi getMovies() dideklarasikan. Dianotasi dengan @Query("SELECT * FROM movies"), fungsi ini akan mengeksekusi kueri SQL untuk mengambil semua baris dari tabel bernama "movies". Fungsi ini mengembalikan Flow<List<MovieEntity>>, yang berarti ia akan secara asinkron memancarkan daftar MovieEntity setiap kali ada perubahan pada data film di database.

Pada baris [17–18], fungsi insertMovies() dideklarasikan sebagai suspend fungsi. Dianotasi dengan @Insert(onConflict = OnConflictStrategy.REPLACE), fungsi ini akan menyisipkan daftar objek MovieEntity ke dalam tabel "movies". Jika ada konflik (misalnya, entitas dengan primary key yang sama sudah ada), strategi OnConflictStrategy.REPLACE akan mengganti entitas yang sudah ada dengan yang baru.

Pada baris [20–21], fungsi `clearMovies()` dideklarasikan sebagai suspend fungsi. Dianotasi dengan `@Query("DELETE FROM movies")`, fungsi ini akan mengeksekusi kueri SQL untuk menghapus semua baris dari tabel "movies", secara efektif mengosongkan tabel.

Pada baris [23–27], fungsi `clearAndInsertMovies()` dideklarasikan sebagai suspend fungsi. Dianotasi dengan `@Transaction`, fungsi ini akan menjalankan operasi `clearMovies()` dan `insertMovies()` dalam satu transaksi database tunggal. Ini memastikan bahwa kedua operasi berhasil atau keduanya gagal, menjaga integritas data. Secara spesifik, ia akan menghapus semua film yang ada, lalu menyisipkan daftar film yang baru.

Data/local/model/MovieEntity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimport komponen yang diperlukan dari pustaka Room. `@Entity` diimport untuk menandai kelas sebagai entitas database Room, yang berarti kelas ini akan dipetakan ke sebuah tabel dalam database. `@PrimaryKey` diimport untuk menandai sebuah properti sebagai kunci utama tabel.

Pada baris [7], kelas data `MovieEntity` dideklarasikan. Kelas ini dianotasi dengan `@Entity(tableName = "movies")`, yang berarti Room akan membuat tabel dalam database dengan nama "movies" yang strukturnya sesuai dengan properti kelas ini.

Pada baris [8], properti `id` dideklarasikan sebagai `Int` dan dianotasi dengan `@PrimaryKey`. Ini menetapkan `id` sebagai kunci utama tabel "movies", memastikan setiap entri film memiliki identifikasi unik dan tidak ada duplikasi berdasarkan `id`.

Pada baris [9], properti `title` dideklarasikan sebagai `String`. Ini akan menyimpan judul film. Catatan `NON-NULLABLE` menunjukkan bahwa kolom ini tidak boleh berisi nilai null di database.

Pada baris [10], properti `overview` dideklarasikan sebagai `String`. Ini akan menyimpan ringkasan atau deskripsi singkat film. Catatan `NON-NULLABLE` juga menunjukkan bahwa kolom ini tidak boleh berisi nilai null di database.

Pada baris [11], properti `posterPath` dideklarasikan sebagai `String`. Ini akan menyimpan jalur (path) relatif ke gambar poster film.

Pada baris [12], properti `backdropPath` dideklarasikan sebagai `String`. Ini akan menyimpan jalur (path) relatif ke gambar backdrop film, yang biasanya digunakan sebagai latar belakang.

Pada baris [13], properti `releaseDate` dideklarasikan sebagai `String`. Ini akan menyimpan tanggal rilis film.

Pada baris [14], properti `voteAverage` dideklarasikan sebagai `Double`. Ini akan menyimpan rata-rata rating suara atau nilai voting film.

Pada baris [15], properti `popularity` dideklarasikan sebagai `Double`. Ini akan menyimpan nilai popularitas film.

Data/local/AppDatabase.kt:

Pada baris [1], dideklarasikan package tempat file ini berada, yaitu `com.example.booklist.screens`.

Pada baris [3–8], diimport berbagai komponen yang diperlukan dari pustaka `Room` dan `Android`, yaitu `Context`, `@Database`, `Room`, `RoomDatabase`, `MovieDao`, dan `MovieEntity`.

Pada baris [10], kelas abstrak `AppDatabase` dideklarasikan sebagai turunan dari `RoomDatabase`. Kelas ini dianotasi dengan `@Database(entities = [MovieEntity::class], version = 1, exportSchema = false)`. `entities = [MovieEntity::class]` menunjukkan `MovieEntity` adalah satu-satunya entitas yang akan disertakan dalam database ini. `version = 1` menetapkan versi database. `exportSchema = false` berarti `Room` tidak akan mengeksport skema database ke dalam folder.

Pada baris [12], fungsi abstrak `movieDao()` dideklarasikan. Fungsi ini akan diimplementasikan secara otomatis oleh `Room` untuk menyediakan instance dari `MovieDao`, yang memungkinkan interaksi dengan tabel film.

Pada baris [14], blok companion object dideklarasikan. Ini adalah cara di `Kotlin` untuk mendefinisikan anggota statis yang terkait dengan kelas itu sendiri, bukan dengan instance kelas.

Pada baris [16], variabel INSTANCE dideklarasikan dengan anotasi `@Volatile` dan diinisialisasi sebagai `null`. `@Volatile` memastikan bahwa nilai variabel ini selalu up-to-date dan terlihat oleh semua thread, yang penting untuk mencegah masalah konkurensi saat membuat instance database tunggal.

Pada baris [18–29], fungsi `create()` dideklarasikan. Fungsi ini adalah metode singleton yang digunakan untuk mendapatkan atau membuat instance tunggal dari `AppDatabase`. `return INSTANCE ?: synchronized(this) { ... }` akan mengembalikan INSTANCE yang sudah ada jika tidak `null`. Jika INSTANCE adalah `null`, maka blok `synchronized(this)` akan dieksekusi. `synchronized(this)` memastikan bahwa hanya satu thread yang dapat masuk ke blok ini pada satu waktu, mencegah pembuatan beberapa instance database dalam lingkungan multi-thread. Di dalam blok `synchronized`, `Room.databaseBuilder()` digunakan untuk membangun instance database, menggunakan `context.applicationContext` sebagai konteks aplikasi, `AppDatabase::class.java` sebagai kelas database, dan `"movie_database"` sebagai nama file database. `.fallbackToDestructiveMigration()` ditambahkan ke builder, yang merupakan strategi migrasi database yang akan menghapus semua data yang ada dan membuat ulang database jika terjadi perubahan skema dan tidak ada aturan migrasi eksplisit yang ditentukan. Ini sangat berguna selama fase pengembangan untuk menghindari crash karena perubahan skema. `.build()` menyelesaikan proses pembangunan database. `INSTANCE = instance` menyimpan instance database yang baru dibuat ke variabel `INSTANCE`, dan instance mengembalikan instance database tersebut.

Data/mapper/MovieMapper.kt:

Pada baris [1], ditentukan package tempat file ini berada, yaitu `com.example.booklist`. Pada baris [3–5], diimport berbagai kelas model yang diperlukan untuk pemetaan data. `MovieEntity` diimport sebagai representasi data film di database lokal. `MovieResultDto` diimport sebagai representasi data film yang diterima dari API. `Movie` diimport sebagai representasi model domain film, yang merupakan objek data yang digunakan di lapisan domain aplikasi.

Pada baris [7–16], fungsi ekstensi `toEntity()` dideklarasikan untuk kelas `MovieResultDto`. Fungsi ini bertanggung jawab untuk mengubah objek `MovieResultDto` (data dari API) menjadi objek `MovieEntity` (data untuk database lokal). Di dalamnya, setiap properti dari `MovieResultDto` dipetakan ke properti yang sesuai di `MovieEntity`. Operator `?:` (Elvis operator) atau `.orEmpty()` digunakan untuk memberikan nilai default (0, 0.0, atau string kosong) jika properti dari `MovieResultDto` bersifat nullable, memastikan bahwa `MovieEntity` yang dihasilkan memiliki nilai yang valid untuk semua properti yang dideklarasikan sebagai non-nullable.

Pada baris [20–28], fungsi ekstensi `toDomain()` dideklarasikan untuk kelas `MovieEntity`. Fungsi ini bertanggung jawab untuk mengubah objek `MovieEntity` (data dari database lokal) menjadi objek `Movie` (model domain). Proses ini memetakan properti dari `MovieEntity` ke properti yang sesuai di `Movie`, menjadikannya siap untuk digunakan di lapisan domain aplikasi, yang merupakan representasi agnostik terhadap sumber data.

Data/remote/api/ApiConstants.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan object `ApiConstants`. Penggunaan object di Kotlin berarti bahwa `ApiConstants` adalah singleton, yang berarti hanya ada satu instance dari kelas ini sepanjang siklus hidup aplikasi. Ini sangat cocok untuk menyimpan konstanta global seperti URL dasar API dan kunci API.

Pada baris [4], properti `BASE_URL` dideklarasikan sebagai `const val` dengan nilai `"https://api.themoviedb.org/3/"`. `const val` berarti ini adalah konstanta waktu kompilasi, yang optimal untuk nilai-nilai yang tidak akan berubah. `BASE_URL` ini adalah alamat dasar dari API The Movie Database yang akan digunakan untuk semua permintaan API.

Pada baris [5], properti `API_KEY` dideklarasikan sebagai `const val`. Ini adalah kunci API yang diperlukan untuk mengautentikasi permintaan ke The Movie Database API. Kunci ini harus disertakan dalam setiap permintaan untuk mendapatkan akses ke data API.

Data/remote/api/KtorApiClient.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–12], diimport berbagai komponen yang diperlukan dari pustaka Ktor, Kotlinx Serialization, dan model data. `MovieResponseDto` diimport sebagai model data untuk respons API film. `HttpClient`, `call.*`, `Android`, `contentnegotiation.*`, `logging.*`, dan `request.*` diimport dari Ktor Client untuk membangun dan mengkonfigurasi klien HTTP. `json` dari `io.ktor.serialization.kotlinx.json` dan `Json` dari `kotlinx.serialization.json` diimport untuk menangani serialisasi dan deserialisasi JSON. Pada baris [14], dideklarasikan object `KtorApiClient`. Penggunaan object di Kotlin berarti `KtorApiClient` adalah singleton, yang berarti hanya ada satu instance dari klien HTTP ini sepanjang siklus hidup aplikasi. Ini memastikan bahwa konfigurasi klien dan koneksi jaringan digunakan secara efisien.

Pada baris [15–24], properti client dideklarasikan dan diinisialisasi sebagai `HttpClient` yang menggunakan Android engine. Di dalamnya, beberapa plugin Ktor diinstal:

Pada baris [16–18], `install(Logging)` diinstal dengan `level = LogLevel.ALL`. Plugin ini mengaktifkan logging lengkap untuk semua permintaan dan respons HTTP, yang sangat berguna untuk debugging karena menampilkan detail seperti URL, header, dan body permintaan/respons.

Pada baris [19–24], `install(ContentNegotiation)` diinstal, dengan konfigurasi `json` yang menggunakan objek `Json` dari `kotlinx.serialization.json`. Konfigurasi JSON ini disetel dengan `ignoreUnknownKeys = true` (mengabaikan kunci JSON yang tidak ada di model data, mencegah crash jika API mengembalikan bidang baru), `isLenient = true` (memungkinkan format JSON yang sedikit kurang ketat), dan `prettyPrint = true` (memformat output JSON agar mudah dibaca di log).

Pada baris [26–30], fungsi suspend `fun getPopularMovies(): MovieResponseDto` dideklarasikan. Fungsi ini adalah fungsi suspending, yang berarti ia dapat dipanggil dari dalam coroutine dan akan menjeda eksekusi tanpa memblokir thread hingga hasilnya tersedia. Fungsi ini bertanggung jawab untuk melakukan permintaan HTTP untuk mendapatkan daftar film populer.

Pada baris [27], `client.get(ApiConstants.BASE_URL + "movie/popular")` membuat permintaan GET ke URL film populer menggunakan `BASE_URL` dari `ApiConstants`. Pada baris [28], `parameter("api_key", ApiConstants.API_KEY)` menambahkan parameter kueri `api_key` ke permintaan dengan nilai dari `ApiConstants.API_KEY` untuk autentikasi.

Pada baris [29], `.body()` digunakan untuk deserialisasi respons HTTP menjadi objek `MovieResponseDto`, yang kemudian dikembalikan oleh fungsi ini.

Data/remote/model/MovieDto.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport komponen yang diperlukan dari pustaka Kotlinx Serialization. `SerializedName` diimport untuk menentukan nama bidang JSON yang berbeda dari nama properti Kotlin. `Serializable` diimport untuk menandai kelas agar dapat di-serialisasi dan de-serialisasi secara otomatis oleh `kotlinx.serialization`. Anotasi `@SuppressLint("UnsafeOptInUsageError")` digunakan untuk menekan peringatan terkait penggunaan fitur opt-in yang mungkin belum stabil, seringkali terkait dengan fitur eksperimental dalam serialisasi.

Pada baris [9–16], kelas data `MovieResponseDto` dideklarasikan. Kelas ini dianotasi dengan `@Serializable`, memungkinkan instance kelas ini diubah menjadi format data (misalnya JSON) dan sebaliknya. Kelas ini merepresentasikan struktur respons keseluruhan dari API film.

Pada baris [10], properti `page` dideklarasikan sebagai `Int?` (nullable integer), merepresentasikan nomor halaman hasil yang saat ini diambil.

Pada baris [11], properti `results` dideklarasikan sebagai `List<MovieResultDto>?` (nullable list of `MovieResultDto`), yang berisi daftar film sebenarnya yang dikembalikan oleh API.

Pada baris [12–13], properti `totalPages` dideklarasikan sebagai `Int?` dan dianotasi dengan `@SerializedName("total_pages")`. Ini merepresentasikan jumlah total halaman yang tersedia dari hasil API. `SerializedName` digunakan karena nama properti di JSON adalah `"total_pages"` sedangkan di Kotlin adalah `"totalPages"`.

Pada baris [14–15], properti `totalResults` dideklarasikan sebagai `Int?` dan dianotasi dengan `@SerializedName("total_results")`. Ini merepresentasikan jumlah total film yang cocok dengan kueri.

Pada baris [20–33], kelas data `MovieResultDto` dideklarasikan. Kelas ini juga dianotasi dengan `@Serializable`, memungkinkan serialisasi dan de-serialisasi. Kelas ini merepresentasikan detail satu film dalam daftar hasil.

Pada baris [21], properti `id` dideklarasikan sebagai `Int?` (`nullable integer`), yang merupakan ID unik dari film.

Pada baris [22], properti `title` dideklarasikan sebagai `String?` (`nullable string`), yang merupakan judul film.

Pada baris [23], properti `overview` dideklarasikan sebagai `String?` (`nullable string`), yang merupakan ringkasan atau deskripsi film.

Pada baris [24–25], properti `posterPath` dideklarasikan sebagai `String?` dan dianotasi dengan `@SerializedName("poster_path")`, yang berisi jalur (`path`) relatif ke gambar poster film.

Pada baris [26–27], properti `releaseDate` dideklarasikan sebagai `String?` dan dianotasi dengan `@SerializedName("release_date")`, yang berisi tanggal rilis film.

Pada baris [28–29], properti `voteAverage` dideklarasikan sebagai `Double?` dan dianotasi dengan `@SerializedName("vote_average")`, yang berisi rata-rata voting atau rating film.

Pada baris [30–31], properti `backdropPath` dideklarasikan sebagai `String?` dan dianotasi dengan `@SerializedName("backdrop_path")`, yang berisi jalur (`path`) relatif ke gambar backdrop film.

Pada baris [32], properti `popularity` dideklarasikan sebagai `Double?` (`nullable double`), yang berisi nilai popularitas film.

Data/repository/MovieRepositoryImpl.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–12], diimport berbagai komponen yang diperlukan untuk implementasi repositori. `NetworkResult` diimport untuk merepresentasikan status operasi jaringan.

MovieDao diimport untuk berinteraksi dengan database lokal. toDomain dan toEntity diimport sebagai fungsi mapper untuk mengubah data antar lapisan. KtorApiClient diimport untuk melakukan panggilan API. MovieResponseDto diimport sebagai model data respons dari API. Movie diimport sebagai model domain film. MovieRepository diimport sebagai antarmuka repositori yang diimplementasikan. Flow dan map dari `kotlinx.coroutines.flow` diimport untuk menangani aliran data asinkron. `IOException` diimport untuk menangani pengecualian terkait input/output.

Pada baris [14], kelas `MovieRepositoryImpl` dideklarasikan. Kelas ini mengimplementasikan antarmuka `MovieRepository` dan bertanggung jawab untuk mengelola data film dari berbagai sumber (API dan database lokal). Konstruktornya menerima `apiClient` (instance `KtorApiClient`) dan `movieDao` (instance `MovieDao`) sebagai dependensinya.

Pada baris [19–23], fungsi `getMovies()` di-override dari antarmuka `MovieRepository`. Fungsi ini mengembalikan `Flow<List<Movie>>`. Di dalamnya, `movieDao.getMovies()` dipanggil untuk mendapatkan aliran data `List<MovieEntity>` dari database lokal. Kemudian, `.map { entities -> entities.map { it.toDomain() } }` digunakan untuk mengubah setiap `MovieEntity` dalam daftar menjadi objek `Movie` (model domain) menggunakan fungsi `toDomain()` sebelum memancarkannya sebagai aliran data.

Pada baris [25–34], fungsi `refreshMovies()` di-override dari antarmuka `MovieRepository` sebagai suspend fungsi. Fungsi ini bertanggung jawab untuk memperbarui data film dari API dan menyimpannya ke database lokal, mengembalikan `NetworkResult<Unit>` untuk menunjukkan status operasi.

Pada baris [26], blok try dimulai untuk menangani potensi error selama operasi jaringan dan database.

Pada baris [27], `val response: MovieResponseDto = apiClient.getPopularMovies()` memanggil API Ktor untuk mendapatkan daftar film populer.

Pada baris [28], `val movieEntities = response.results?.map { it.toEntity() } ?: emptyList()` mengambil daftar hasil film dari respons API. Setiap `MovieResultDto` kemudian diubah menjadi `MovieEntity` menggunakan fungsi `toEntity()`. Jika `response.results` null, maka akan menghasilkan `emptyList()`.

Pada baris [29], `movieDao.clearAndInsertMovies(movieEntities)` dipanggil untuk menghapus semua film yang ada di database lokal dan kemudian menyisipkan daftar film yang baru didapatkan dari API. Operasi ini berjalan dalam satu transaksi.

Pada baris [30], jika semua operasi berhasil, `NetworkResult.Success(Unit)` dikembalikan, menunjukkan keberhasilan operasi tanpa data spesifik yang dikembalikan.

Pada baris [31–32], blok `catch (e: IOException)` menangani pengecualian `IOException`, yang biasanya terjadi karena masalah koneksi jaringan (misalnya, tidak ada internet). Dalam kasus ini, `NetworkResult.Error` dikembalikan dengan pesan error yang relevan untuk pengguna.

Pada baris [33–34], blok `catch (e: Exception)` adalah penangkap umum untuk pengecualian lain yang mungkin terjadi. Dalam kasus ini, `NetworkResult.Error` dikembalikan dengan pesan error yang berisi detail dari `e.message` untuk diagnosis lebih lanjut.

Domain/model/Movie.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], kelas data `Movie` dideklarasikan. Kelas data ini adalah representasi objek film di lapisan domain aplikasi, yang berarti ia berisi data yang relevan untuk UI tanpa terpapar detail implementasi dari sumber data (baik API maupun database).

Pada baris [4–10], dideklarasikan properti-properti yang membentuk objek `Movie`. Properti `id` dideklarasikan sebagai `Int`, yang merupakan ID unik untuk film. Properti `title` dideklarasikan sebagai `String`, yang menyimpan judul film. Properti `overview` dideklarasikan sebagai `String`, yang menyimpan ringkasan atau sinopsis film. Properti `posterPath` dideklarasikan sebagai `String?` (`nullable String`), yang menyimpan jalur relatif ke gambar poster film. Ini `nullable` karena mungkin tidak selalu tersedia. Properti `backdropPath` dideklarasikan sebagai `String?` (`nullable String`), yang menyimpan jalur relatif ke gambar latar belakang (`backdrop`) film. Ini juga `nullable`. Properti `voteAverage` dideklarasikan sebagai `Double`, yang menyimpan nilai rata-rata vote atau

rating film. Properti `releaseDate` dideklarasikan sebagai `String?` (nullable `String`), yang menyimpan tanggal rilis film. Ini nullable.

Pada baris [11–13], fungsi `getPosterUrl()` dideklarasikan. Fungsi ini mengembalikan URL lengkap dari gambar poster film. Ia memeriksa apakah `posterPath` tidak null dan tidak kosong, jika ya, ia akan menggabungkan `BASE_URL` dari The Movie Database untuk gambar (<https://image.tmdb.org/t/p/w500>) dengan `posterPath` yang tersedia. Jika `posterPath` null atau kosong, fungsi ini mengembalikan string kosong.

Pada baris [15–17], fungsi `getBackdropUrl()` dideklarasikan. Mirip dengan `getPosterUrl()`, fungsi ini mengembalikan URL lengkap dari gambar backdrop film. Ia menggunakan ukuran yang berbeda (`w780`) dan juga memeriksa `backdropPath` untuk memastikan tidak null atau kosong, mengembalikan string kosong jika tidak ada.

Pada baris [19–21], fungsi `getTmdbUrl()` dideklarasikan. Fungsi ini mengembalikan URL langsung ke halaman film di situs web The Movie Database, menggunakan `id` film untuk membuat tautan yang spesifik.

Domain/repository/MovieRepository.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport berbagai komponen yang diperlukan. `NetworkResult` diimport untuk merepresentasikan status operasi jaringan. `Movie` diimport sebagai model domain film. `Flow` diimport dari `kotlinx.coroutines.flow` untuk mendukung pengumpulan data asinkron. Pada baris [7], antarmuka `MovieRepository` dideklarasikan. Antarmuka ini mendefinisikan kontrak untuk operasi data film, memisahkan lapisan domain dari detail implementasi data. Pada baris [8], fungsi `getMovies()` dideklarasikan. Fungsi ini mengembalikan `Flow<List<Movie>>`, yang berarti ia akan menyediakan aliran data asinkron berisi daftar objek `Movie`. Ini digunakan untuk mengambil data film yang akan diobservasi oleh lapisan presentasi. Pada baris [9], fungsi `refreshMovies()` dideklarasikan sebagai suspend fungsi. Fungsi ini bertanggung jawab untuk memperbarui data film, biasanya dengan mengambil data terbaru dari sumber eksternal (misalnya API). Ia mengembalikan

NetworkResult<Unit>, yang menunjukkan status operasi (sukses, error, atau loading) tanpa mengembalikan data spesifik (hanya Unit sebagai penanda keberhasilan).

Domain/usecase/GetMoviesUseCase.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–6], diimport berbagai komponen yang diperlukan. NetworkResult diimport untuk merepresentasikan status operasi jaringan. Movie diimport sebagai model domain film. MovieRepository diimport sebagai antarmuka repositori untuk mendapatkan data film. Flow diimport dari kotlinx.coroutines.flow untuk mendukung pengumpulan data asinkron.

Pada baris [8], kelas GetMoviesUseCase dideklarasikan. Kelas ini merupakan bagian dari lapisan domain dan mengimplementasikan logika bisnis untuk mengambil dan menyegarkan data film. Konstruktornya menerima movieRepository sebagai dependensinya, yang memungkinkan use case ini berinteraksi dengan lapisan data.

Pada baris [9], fungsi getMoviesFromDb() dideklarasikan. Fungsi ini mengembalikan Flow<List<Movie>>. Ia secara langsung mendelegasikan panggilan ke movieRepository.getMovies(), yang berarti use case ini menyediakan cara bagi lapisan presentasi untuk mendapatkan aliran data film dari database lokal.

Pada baris [11], fungsi refreshMoviesFromApi() dideklarasikan sebagai suspend fungsi. Fungsi ini juga mendelegasikan panggilan ke movieRepository.refreshMovies(). Ini adalah fungsi yang bertanggung jawab untuk memicu pembaruan data film dari API dan menyimpannya ke database, mengembalikan NetworkResult<Unit> untuk menunjukkan status operasi (sukses, error, atau loading).

presentation/ui/component/MovieCard.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–19], diimport berbagai komponen yang diperlukan dari Jetpack Compose dan pustaka lainnya. androidx.compose.foundation diimport untuk elemen UI dasar seperti tata letak dan interaksi. androidx.compose.material3 diimport untuk komponen

Material Design 3 seperti Card, Button, Text, MaterialTheme, CardDefaults, dan ExperimentalMaterial3Api. `androidx.compose.runtime` diimport untuk fungsionalitas Composable. `androidx.compose.ui` diimport untuk modifier UI seperti Modifier, Alignment, `draw.clip`, `layout.ContentScale`, `platform.LocalContext`, dan `text.font.FontWeight`, `text.style.TextOverflow`. `androidx.compose.ui.unit.dp` diimport untuk unit densitas piksel. `AsyncImage` dan `ImageRequest` dari pustaka Coil diimport untuk memuat gambar secara asinkron. `Movie` diimport sebagai model domain film.

Pada baris [21], anotasi `@OptIn(ExperimentalMaterial3Api::class)` digunakan untuk mengizinkan penggunaan API eksperimental dari Material3, seperti `CardDefaults.cardElevation`.

Pada baris [22], fungsi `MovieCard` dideklarasikan sebagai Composable. Fungsi ini menerima objek `Movie`, sebuah modifier opsional, sebuah lambda `onMovieClick` yang akan dipanggil ketika kartu film diklik, dan sebuah lambda `onTmdbClick` yang akan dipanggil ketika tombol TMDB diklik.

Pada baris [27], variabel `context` dideklarasikan dan diinisialisasi dengan `LocalContext.current`, yang menyediakan akses ke konteks Android saat ini, berguna untuk membangun permintaan gambar Coil.

Pada baris [29–35], komponen Card dideklarasikan sebagai elemen dasar untuk menampilkan informasi film. modifier diterapkan untuk mengatur lebar kartu, membuatnya dapat diklik (`clickable { onMovieClick(movie) }`), dan memberikan padding. `shape = RoundedCornerShape(12.dp)` memberikan sudut membulat pada kartu, dan `elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)` memberikan efek bayangan yang mengangkat kartu dari latar belakang.

Pada baris [36–69], `Row` dideklarasikan di dalam Card untuk mengatur tata letak horizontal konten film. modifier disetel untuk mengisi lebar penuh dan memberikan padding.

Pada baris [40–47], `AsyncImage` digunakan untuk memuat dan menampilkan gambar poster film secara asinkron. model dibangun menggunakan `ImageRequest.Builder` dengan konteks lokal dan URL poster film yang didapatkan dari `movie.getPosterUrl()`. `crossfade(true)` menambahkan efek fade-in saat gambar dimuat. `contentDescription`

diatur ke judul film untuk aksesibilitas. modifier mengatur ukuran gambar, memberinya sudut membulat, dan `contentScale = ContentScale.Crop` memastikan gambar dipangkas agar sesuai dengan batasnya.

Pada baris [49], `Spacer` dideklarasikan untuk memberikan ruang horizontal sebesar 12.dp antara gambar poster dan detail teks.

Pada baris [51–68], `Column` dideklarasikan untuk mengatur tata letak vertikal detail teks film. modifier memberikan bobot 1f agar kolom ini mengambil sisa ruang horizontal dan `align(Alignment.CenterVertically)` menyejajarkan konten kolom secara vertikal di tengah.

Pada baris [53–57], `Text` digunakan untuk menampilkan judul film. `text` disetel ke `movie.title`, `style` menggunakan `MaterialTheme.typography.titleMedium`, `fontWeight` disetel ke `Bold`, `maxLines = 2` membatasi judul hingga dua baris, dan `overflow = TextOverflow.Ellipsis` menambahkan elipsis jika judul terlalu panjang.

Pada baris [58], `Spacer` dideklarasikan untuk memberikan ruang vertikal sebesar 4.dp.

Pada baris [60–62], `Text` digunakan untuk menampilkan rating film, diformat menjadi satu desimal. `style` menggunakan `MaterialTheme.typography.bodySmall`.

Pada baris [63], `Spacer` dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp.

Pada baris [65–68], `Row` dideklarasikan untuk menampung tombol-tombol aksi, dengan `horizontalArrangement = Arrangement.End` untuk menempatkan tombol di sisi kanan.

Pada baris [69–72], `Button` pertama dideklarasikan. `onClick` diatur untuk memanggil `onTmdbClick` dengan URL TMDB film yang didapatkan dari `movie.getTmdbUrl()`. modifier memberikan `padding` di sisi kanan. `Text("TMDB")` ditampilkan sebagai label tombol.

Pada baris [74–76], `Button` kedua dideklarasikan. `onClick` diatur untuk memanggil `onMovieClick` dengan objek `movie`, memungkinkan navigasi ke detail film. `Text("Detail")` ditampilkan sebagai label tombol.

presentation/ui/navigation/Screen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], dideklarasikan sealed class Screen dengan sebuah properti route bertipe String. Sealed class ini digunakan untuk merepresentasikan semua kemungkinan tujuan (destinasi) dalam navigasi aplikasi secara aman dan type-safe. Properti route akan digunakan oleh Jetpack Compose Navigation untuk mengidentifikasi layar yang berbeda.

Pada baris [4], MovieList dideklarasikan sebagai data object yang merupakan turunan dari Screen. Ini merepresentasikan layar daftar film dan memiliki route "movie_list".

Pada baris [5], MovieDetail dideklarasikan sebagai data object yang juga merupakan turunan dari Screen. Ini merepresentasikan layar detail film dan memiliki route "movie_detail". Penggunaan data object adalah cara yang efisien untuk mendefinisikan objek tunggal (singleton) dalam Kotlin, yang sangat cocok untuk merepresentasikan rute navigasi yang tidak memerlukan parameter tambahan dalam definisi rutenya.

presentation/ui/screens/detail/MovieDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–19], diimport berbagai komponen yang diperlukan dari Jetpack Compose dan pustaka lainnya. androidx.compose.foundation diimport untuk elemen UI dasar seperti tata letak, scrolling, dan state. androidx.compose.material.icons.Icons dan androidx.compose.material.icons.filled.Star diimport untuk ikon bintang. androidx.compose.material3 diimport untuk komponen Material Design 3 seperti Text, Icon, MaterialTheme, dan Scaffold (meskipun Scaffold tidak digunakan secara langsung di Composable ini, komponen Material3 lainnya digunakan). androidx.compose.runtime diimport untuk fungsionalitas Composable. androidx.compose.ui diimport untuk modifier UI seperti Modifier, Alignment, draw.clip, graphics.Color, layout.ContentScale, platform.LocalContext, text.font.FontWeight, dan unit.dp untuk unit densitas piksel. AsyncImage dan ImageRequest dari pustaka Coil diimport untuk memuat gambar secara asinkron. Movie diimport sebagai model domain film.

Pada baris [21], fungsi MovieDetailScreen dideklarasikan sebagai Composable. Fungsi ini menerima objek movie yang berisi detail film yang akan ditampilkan.

Pada baris [22–27], Column dideklarasikan sebagai tata letak vertikal utama untuk seluruh konten layar detail film. modifier diterapkan untuk mengisi seluruh ukuran layar (`fillMaxSize()`), menambahkan padding 16.dp di sekelilingnya, dan membuatnya dapat di-scroll secara vertikal (`verticalScroll(rememberScrollState())`) jika konten melebihi tinggi layar.

Pada baris [28–36], `AsyncImage` digunakan untuk memuat dan menampilkan gambar backdrop film secara asinkron. model dibangun menggunakan `ImageRequest.Builder` dengan konteks lokal dan URL backdrop film yang didapatkan dari `movie.getBackdropUrl()`. `crossfade(true)` menambahkan efek fade-in saat gambar dimuat. `contentDescription` diatur ke judul film untuk aksesibilitas. modifier mengatur gambar untuk mengisi lebar penuh, memiliki tinggi 220.dp, dan memiliki sudut membulat sesuai bentuk tema Material (`clip(MaterialTheme.shapes.medium)`), sementara `contentScale = ContentScale.Crop` memastikan gambar dipangkas agar sesuai dengan batasnya.

Pada baris [37], `Spacer` dideklarasikan untuk memberikan ruang vertikal sebesar 16.dp. Pada baris [39–42], `Text` digunakan untuk menampilkan judul film. `text` disetel ke `movie.title`, `style` menggunakan `MaterialTheme.typography.headlineMedium`, dan `fontWeight` disetel ke `FontWeight.Bold` untuk menonjolkan judul.

Pada baris [43], `Spacer` dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp. Pada baris [45–56], `Row` dideklarasikan untuk mengatur tata letak horizontal ikon bintang dan rating. `verticalAlignment = Alignment.CenterVertically` memastikan elemen di tengah secara vertikal.

Pada baris [46–51], `Icon` digunakan untuk menampilkan ikon bintang. `imageVector` disetel ke `Icons.Filled.Star`, `contentDescription` diatur sebagai "Rating" untuk aksesibilitas, `tint` disetel ke warna kuning (`Color(0xFFFFDD835)`), dan modifier mengatur ukuran ikon menjadi 24.dp.

Pada baris [52], `Spacer` dideklarasikan untuk memberikan ruang horizontal sebesar 4.dp.

Pada baris [53–56], Text digunakan untuk menampilkan rating film yang diformat menjadi satu desimal dari `movie.voteAverage`. style menggunakan `MaterialTheme.typography.titleMedium`.

Pada baris [57], Spacer dideklarasikan untuk memberikan ruang vertikal sebesar 8.dp. Pada baris [59–62], blok `movie.releaseDate?.let { ... }` digunakan untuk menampilkan tanggal rilis film hanya jika `releaseDate` tidak null. Di dalamnya, Text menampilkan tanggal rilis, dan Spacer memberikan ruang vertikal sebesar 8.dp setelahnya.

Pada baris [64–67], Text digunakan untuk menampilkan deskripsi atau overview film. text disetel ke `movie.overview`, dan style menggunakan `MaterialTheme.typography.bodyLarge`.

presentation/ui/screens/list/MovieListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–24], diimport berbagai komponen yang diperlukan dari Android, Jetpack Compose, dan pustaka lainnya. Intent dan Uri diimport untuk tujuan deep linking atau membuka URL eksternal. `androidx.compose.foundation.layout` diimport untuk komponen tata letak. `androidx.compose.foundation.lazy.LazyColumn`, `LazyVerticalGrid`, `GridCells`, dan `items` diimport untuk menampilkan daftar item yang efisien. Ikon-ikon `Icons.Filled.CheckCircle`, `Icons.Filled.Menu`, dan `Icons.AutoMirrored.Filled.List` diimport untuk tombol action di top bar. `androidx.compose.material3` diimport untuk komponen Material Design 3 seperti Scaffold, TopAppBar, Text, IconButton, Icon, CircularProgressIndicator, dan ExperimentalMaterial3Api. `androidx.compose.runtime` diimport untuk fungsionalitas Composable dan state management seperti `rememberCoroutineScope`, `collectAsState`, dan `getValue`. `androidx.compose.ui` diimport untuk Modifier dan Alignment. `androidx.compose.ui.platform.LocalContext` diimport untuk mendapatkan konteks Android. `androidx.compose.ui.unit.dp` diimport untuk unit densitas piksel. NavController dari `androidx.navigation.compose` diimport untuk navigasi. MovieCard diimport sebagai komponen UI kustom. LayoutType dan SettingsManager diimport

untuk pengelolaan pengaturan aplikasi. Gson diimport untuk serialisasi objek. launch dari `kotlinx.coroutines` diimport untuk meluncurkan coroutine.

Pada baris [26], anotasi `@OptIn(ExperimentalMaterial3Api::class)` digunakan untuk mengizinkan penggunaan API eksperimental dari Material3, seperti `TopAppBar`.

Pada baris [27], fungsi `MovieListScreen` dideklarasikan sebagai `Composable`. Fungsi ini menerima `viewModel` (instance `MovieListViewModel`), `settingsManager` (instance `SettingsManager`), dan `navController` (instance `NavController`) sebagai parameter.

Pada baris [33], variabel `uiState` diobservasi dari `viewModel.uiState.collectAsState()`, yang akan memicu rekomposisi UI setiap kali status UI berubah.

Pada baris [34], variabel `layoutType` diobservasi dari `settingsManager.layoutType.collectAsState()`, dengan nilai awal `LayoutType.LIST`. Ini akan mengontrol apakah film ditampilkan dalam tata letak daftar atau grid.

Pada baris [35], variabel `isDarkMode` diobservasi dari `settingsManager.isDarkMode.collectAsState()`, dengan nilai awal `false`. Ini akan mengontrol tema aplikasi.

Pada baris [36], variabel `context` diinisialisasi dengan `LocalContext.current` untuk mendapatkan konteks Android saat ini, yang diperlukan untuk memulai Intent.

Pada baris [37], variabel `scope` diinisialisasi dengan `rememberCoroutineScope()`, yang menyediakan `CoroutineScope` yang terkait dengan siklus hidup `Composable`, digunakan untuk meluncurkan coroutine.

Pada baris [39–98], `Scaffold` dideklarasikan sebagai komponen Material Design 3 yang menyediakan slot untuk top bar, bottom bar, snackbar, dan konten utama.

Pada baris [40–64], blok `topBar` dari `Scaffold` didefinisikan.

Pada baris [41–63], `TopAppBar` dideklarasikan sebagai bilah atas aplikasi. `title` disetel ke `Text("Popular Movies")`.

Pada baris [42–62], blok `actions` dari `TopAppBar` didefinisikan untuk menampung tombol aksi di sisi kanan.

Pada baris [43–50], `IconButton` pertama dideklarasikan. Ketika diklik, ia meluncurkan coroutine menggunakan `scope.launch` yang kemudian memanggil `settingsManager.toggleDarkMode()` untuk mengubah tema aplikasi antara terang dan

gelap. Icon ditampilkan dengan ikon CheckCircle yang sama untuk kedua mode (meskipun secara umum ikon akan berubah untuk menunjukkan status).

Pada baris [52–61], IconButton kedua dideklarasikan. Ketika diklik, ia meluncurkan coroutine menggunakan scope.launch yang kemudian menghitung newLayout (mengubah dari LIST ke GRID atau sebaliknya) dan memanggil settingsManager.setLayoutType(newLayout) untuk mengubah tata letak film. Icon ditampilkan dengan ikon Menu jika tata letak saat ini LIST dan List (dari AutoMirrored.Filled) jika tata letak saat ini GRID.

Pada baris [64], paddingValues adalah parameter yang disediakan oleh Scaffold untuk memastikan konten tidak tumpang tindih dengan top bar.

Pada baris [65–97], Box dideklarasikan sebagai wadah untuk konten utama layar, dengan modifier mengisi seluruh ruang dan menerapkan paddingValues dari Scaffold. Pada baris [67–68], kondisi if (uiState.isLoading && uiState.movies.isEmpty()) memeriksa apakah aplikasi sedang memuat data dan daftar film masih kosong. Jika ya, CircularProgressIndicator ditampilkan di tengah layar.

Pada baris [69–75], kondisi else if (uiState.errorMessage != null && uiState.movies.isEmpty()) memeriksa apakah ada pesan error dan daftar film masih kosong. Jika ya, Text ditampilkan dengan pesan error (atau "Unknown Error" jika pesan null), dengan warna error, di tengah layar.

Pada baris [76–96], blok else dieksekusi jika aplikasi tidak dalam kondisi loading atau error yang membuat daftar kosong, yang berarti data film siap ditampilkan.

Pada baris [77–95], kondisi if (layoutType == LayoutType.LIST) memeriksa jenis tata letak yang dipilih.

Pada baris [78–90], jika layoutType adalah LIST, LazyColumn digunakan untuk menampilkan daftar film secara vertikal. modifier mengisi seluruh ukuran, contentPadding memberikan padding di sekeliling daftar, dan verticalArrangement memberikan jarak antar item.

Pada baris [81–89], items(uiState.movies) mengulang melalui daftar film dan untuk setiap film, MovieCard ditampilkan. modifier diatur untuk mengisi lebar penuh. onMovieClick lambda mendefinisikan navigasi ke layar detail film: objek

selectedMovie di-serialisasi ke JSON, di-encode URL, lalu navController.navigate dipanggil dengan rute "movie_detail/" diikuti oleh JSON film. onTmdbClick lambda mendefinisikan aksi untuk membuka URL TMDB di browser eksternal: sebuah Intent dibuat dengan ACTION_VIEW dan Uri yang diparse dari tmdbUrl, lalu context.startActivity dipanggil.

Pada baris [91–95], jika layoutType adalah GRID, LazyVerticalGrid digunakan untuk menampilkan daftar film dalam tata letak grid dengan 2 kolom tetap. modifier mengisi seluruh ukuran, contentPadding memberikan padding, dan verticalArrangement serta horizontalArrangement memberikan jarak antar item.

Pada baris [96–104], items(uiState.movies) mengulang melalui daftar film dalam grid, dan untuk setiap film, MovieCard ditampilkan dengan logika onMovieClick dan onTmdbClick yang sama seperti pada LazyColumn.

presentation/ui/screens/list/MovieListViewModel.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–10], diimport berbagai komponen yang diperlukan dari pustaka Jetpack AndroidX, Kotlin Coroutines, dan model data aplikasi. ViewModel dan viewModelScope diimport untuk fungsionalitas ViewModel dan manajemen coroutine. NetworkResult diimport untuk merepresentasikan status hasil jaringan. Movie diimport sebagai model domain film. GetMoviesUseCase diimport sebagai use case untuk mendapatkan data film. MutableStateFlow, StateFlow, asStateFlow, catch, onStart, dan launch diimport dari kotlinx.coroutines.flow dan kotlinx.coroutines untuk manajemen aliran data reaktif dan konkurensi.

Pada baris [12–16], kelas data MovieListUiState dideklarasikan. Kelas ini merepresentasikan status UI dari layar daftar film. Ini mencakup movies (daftar objek Movie yang ditampilkan, default kosong), isLoading (boolean yang menunjukkan apakah data sedang dimuat, default false), dan errorMessage (string opsional untuk pesan error, default null).

Pada baris [18], kelas MovieListViewModel dideklarasikan sebagai turunan dari ViewModel. ViewModel ini bertanggung jawab untuk menyiapkan data untuk UI,

menangani logika bisnis, dan berinteraksi dengan use case dan repositori. Konstruktornya menerima `getMoviesUseCase` sebagai dependensinya.

Pada baris [22], variabel `_uiState` dideklarasikan sebagai `MutableStateFlow` yang menyimpan instance `MovieListUiState` dengan nilai awal default. `MutableStateFlow` adalah aliran data yang dapat diubah dan memegang nilai terakhir yang dipancarkan.

Pada baris [23], variabel `uiState` dideklarasikan sebagai `StateFlow` yang merupakan versi read-only dari `_uiState` (`_uiState.asStateFlow()`). Ini adalah aliran yang akan diobservasi oleh UI Composable.

Pada baris [25–28], blok `init` dideklarasikan. Blok ini akan dieksekusi segera setelah instance `MovieListViewModel` dibuat.

Pada baris [26], `observeMovies()` dipanggil untuk memulai pengumpulan data film dari database lokal.

Pada baris [27], `refreshData()` dipanggil untuk memicu pengambilan data film terbaru dari API.

Pada baris [30–44], fungsi privat `observeMovies()` dideklarasikan. Fungsi ini bertanggung jawab untuk mengamati aliran data film dari database lokal.

Pada baris [31], `viewModelScope.launch` digunakan untuk meluncurkan coroutine dalam cakupan `ViewModel`, memastikan coroutine dibatalkan ketika `ViewModel` dihapus.

Pada baris [32], `getMoviesUseCase.getMoviesFromDb()` dipanggil untuk mendapatkan `Flow` dari daftar film dari database.

Pada baris [33], `.onStart { _uiState.value = _uiState.value.copy(isLoading = true) }` adalah operator aliran yang akan dieksekusi sebelum aliran mulai mengumpulkan item. Ini mengatur `isLoading` di `_uiState` menjadi `true` untuk menunjukkan bahwa proses pemuatan dimulai.

Pada baris [34–38], `.catch { e -> ... }` adalah operator aliran yang akan menangkap pengecualian apa pun yang terjadi di dalam aliran. Jika error terjadi saat memuat dari database, `isLoading` diatur menjadi `false` dan `errorMessage` diatur dengan pesan error yang sesuai.

Pada baris [39–43], `.collect { movies -> ... }` adalah operator yang mengumpulkan item yang dipancarkan oleh aliran. Setiap kali daftar film dari database berubah, `isLoading` diatur menjadi `false` dan `movies` di `_uiState` diperbarui dengan daftar film yang baru.

Pada baris [46–63], fungsi `refreshData()` dideklarasikan. Fungsi ini bertanggung jawab untuk mengambil data film terbaru dari API.

Pada baris [47], `viewModelScope.launch` digunakan untuk meluncurkan coroutine.

Pada baris [48], `_uiState.value = _uiState.value.copy(isLoading = true, errorMessage = null)` mengatur `isLoading` menjadi `true` dan menghapus `errorMessage` sebelumnya untuk menunjukkan bahwa proses pembaruan data dimulai.

Pada baris [49], `when (val result = getMoviesUseCase.refreshMoviesFromApi())` memanggil use case untuk menyegarkan data dari API dan kemudian memproses hasilnya berdasarkan jenis `NetworkResult`.

Pada baris [50–52], jika `result` adalah `NetworkResult.Success`, `isLoading` diatur kembali ke `false`.

Pada baris [53–57], jika `result` adalah `NetworkResult.Error`, `isLoading` diatur ke `false` dan `errorMessage` diatur ke pesan yang diterima dari `result`.

Pada baris [58], `else -> {}` menangani kasus lain dari `NetworkResult` (misalnya, `Loading`, meskipun biasanya `Loading` tidak akan diproses di sini karena `refreshMoviesFromApi()` segera mengembalikan `Success` atau `Error`).

presentation/ui/screens/list/MovieListViewModelFactory.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–5], diimport komponen yang diperlukan dari pustaka Jetpack AndroidX. `ViewModel` dan `ViewModelProvider` diimport untuk fungsionalitas `ViewModel` dan kemampuan untuk membuat instance `ViewModel`. `GetMoviesUseCase` diimport sebagai use case yang akan disuntikkan ke dalam `ViewModel`.

Pada baris [7], kelas `MovieListViewModelFactory` dideklarasikan. Kelas ini mengimplementasikan antarmuka `ViewModelProvider.Factory`. Pola `Factory` ini diperlukan ketika `ViewModel` memiliki dependensi (dalam hal ini, `GetMoviesUseCase`) yang perlu disediakan pada saat pembuatan.

Konstruktor kelas ini menerima sebuah instance `getMoviesUseCase` sebagai parameter privat. Ini adalah dependensi yang akan diberikan kepada `MovieListViewModel` ketika dibuat.

Pada baris [11–17], fungsi `create()` di-override dari antarmuka `ViewModelProvider.Factory`. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance `ViewModel` yang sesuai.

Pada baris [12], `if (modelClass.isAssignableFrom(MovieListViewModel::class.java))` memeriksa apakah `modelClass` yang diminta (yaitu, kelas `ViewModel` yang ingin dibuat) adalah `MovieListViewModel` atau salah satu subclass-nya.

Pada baris [14], jika kelas yang diminta adalah `MovieListViewModel`, `MovieListViewModel(getMoviesUseCase)` dipanggil untuk membuat instance baru dari `MovieListViewModel`, dengan menyuntikkan `getMoviesUseCase` yang telah diterima oleh factory ini.

Pada baris [15], `@Suppress("UNCHECKED_CAST")` digunakan untuk menekan peringatan kompilasi karena casting dari `MovieListViewModel` ke `T` tidak dapat diperiksa secara ketat pada waktu kompilasi. Ini aman karena pemeriksaan `isAssignableFrom` sudah memastikan jenis yang benar.

Pada baris [17], jika `modelClass` yang diminta bukan `MovieListViewModel`, `throw IllegalArgumentException("Unknown ViewModel class")` akan dilemparkan, mengindikasikan bahwa factory ini tidak tahu cara membuat `ViewModel` untuk kelas yang diminta.

Tautan Git

Berikut adalah tautan untuk semua source code yang telah dibuat.

https://github.com/Bukhrii/Praktikum_Mobile